

计算机视觉

第7章 目标检测

软件学院：刘春

Ref: C280, Computer Vision
Prof. Trevor Darrell

本章内容

- 7.1.目标检测定义
- 7.2.R-CNN
- 7.3.Fast/Faster R-CNN

计算机视觉主要任务

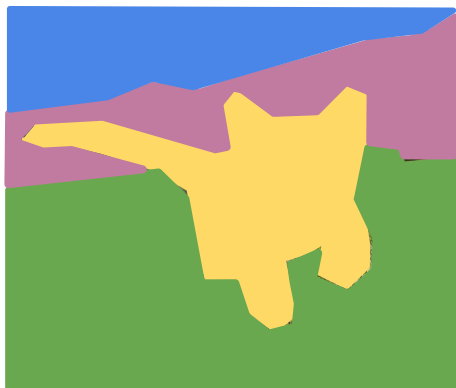
分类



CAT

不考虑局部区域

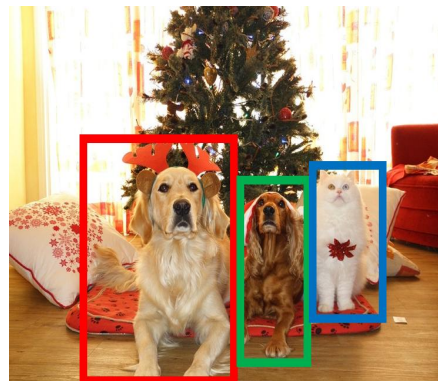
语义分割



GRASS, CAT, TREE,
SKY

没有目标, 只考虑像素

目标检测



DOG, DOG, CAT

多目标

实例分割



DOG, DOG, CAT

[This image is CC0 public domain](#)

目标检测

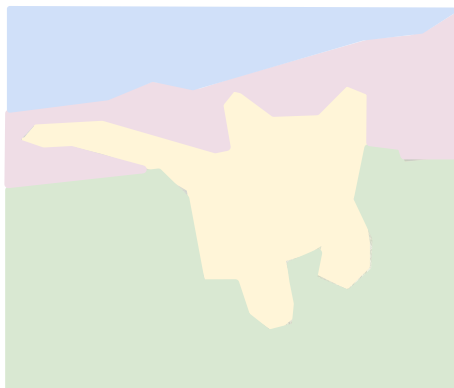
Classification



CAT

No spatial extent

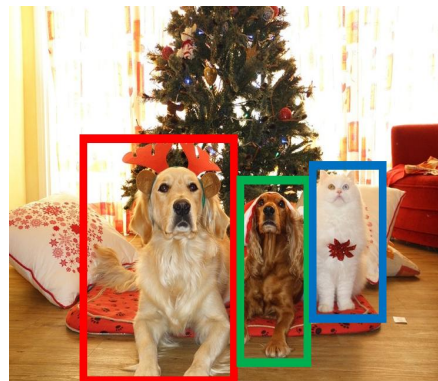
Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

目标检测



DOG, DOG, CAT

多个目标

Instance Segmentation



DOG, DOG, CAT

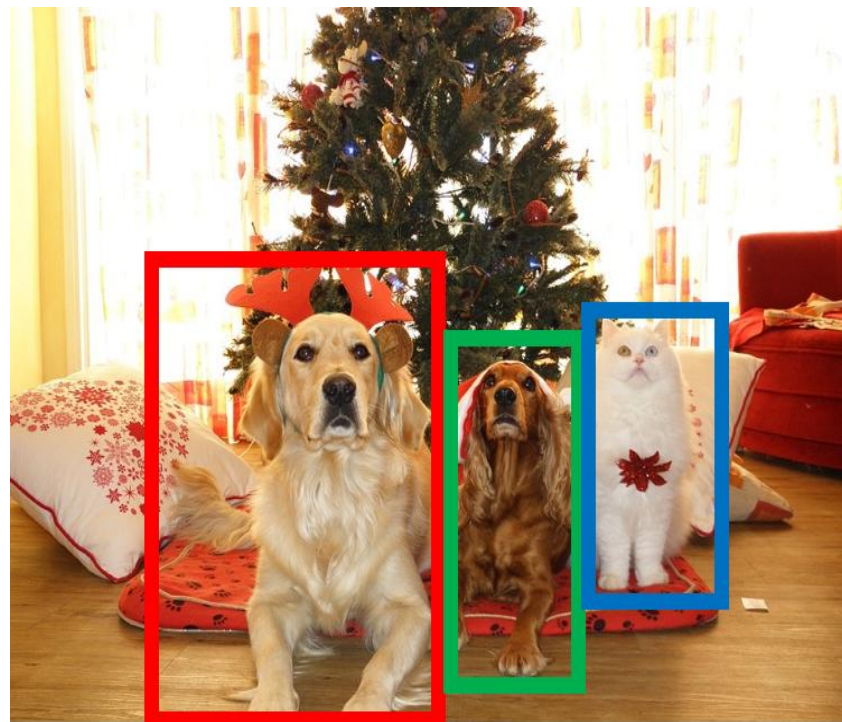
[This image is CC0 public domain](#)

目标检测: 定义

输入: 单张RGB图像

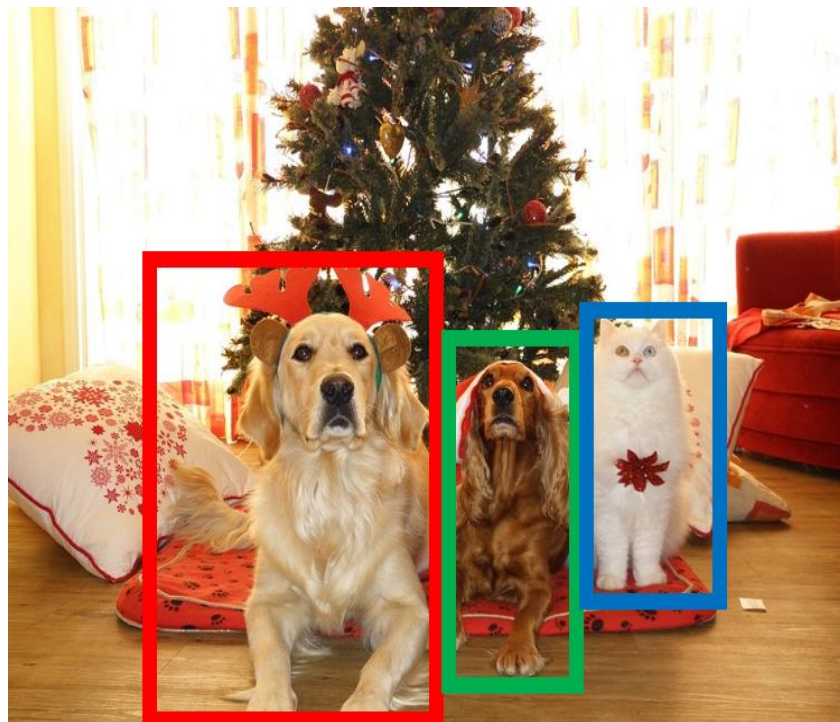
输出: 检测目标集合;
对于每个检测目标:

1. 类别标签(已知的类别标签中的一个)
2. 外接矩形(4个数: x , y , width, height)



目标检测: 挑战

- **多个输出:** 每张图片需要输出若干个目标
- **多种类型输出:** 既要预测是什么, 又要预测在哪里
- **图像大:** 图像分类尺寸为 224×224 ; 检测需要更高分辨率图像, 如 $\sim 800 \times 600$



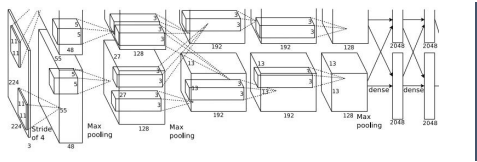
本章内容

- 7.1.目标检测定义
- 7.2.R-CNN
- 7.3.Fast/Faster R-CNN

检测单一目标



[This image is CC0 public domain](#)

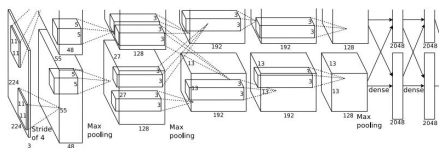


向量:
4096

检测单一目标



[This image is CC0 public domain](#)



向量:
4096

Fully
Connected:
4096 to 1000

“What”

Class分值

Cat: 0.9
Dog: 0.05
Car: 0.01
...

正确类别:
Cat

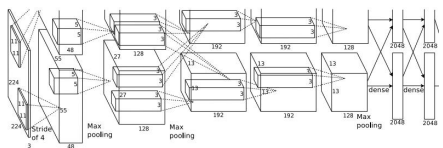
Softmax
Loss

检测单一目标

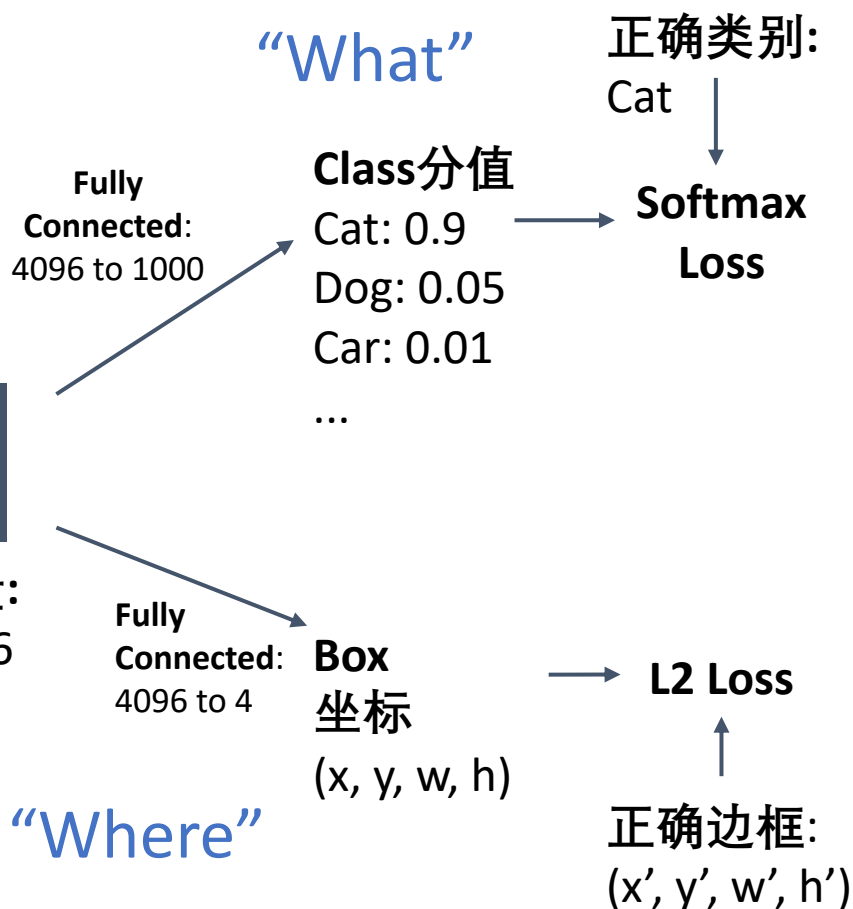


[This image is CC0 public domain](#)

定位被看成一个
回归问题!



向量:
4096

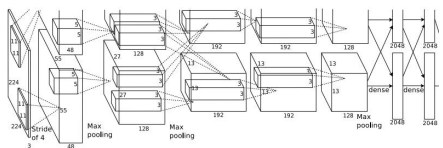


检测单一目标

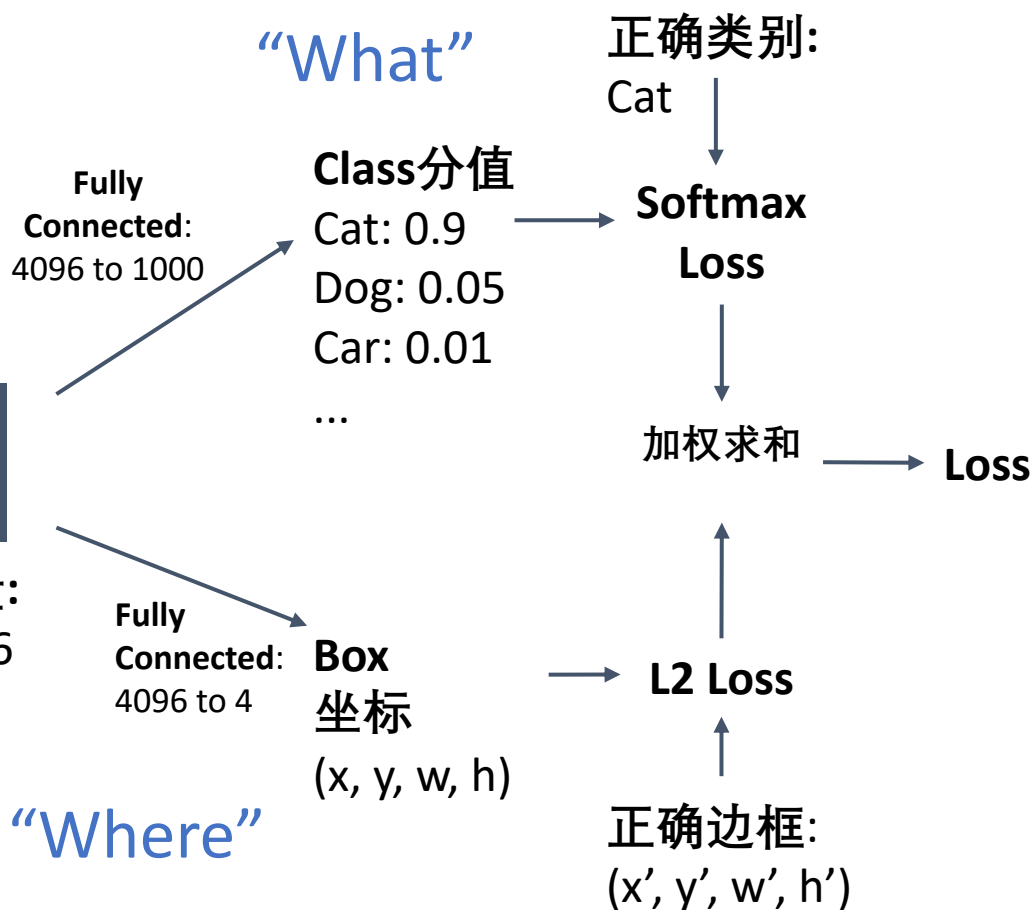


[This image is CC0 public domain](#)

定位被看成一个
回归问题!



向量:
4096

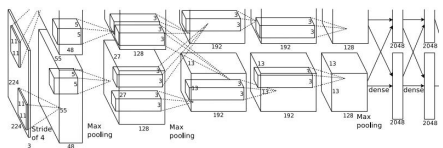


检测单一目标

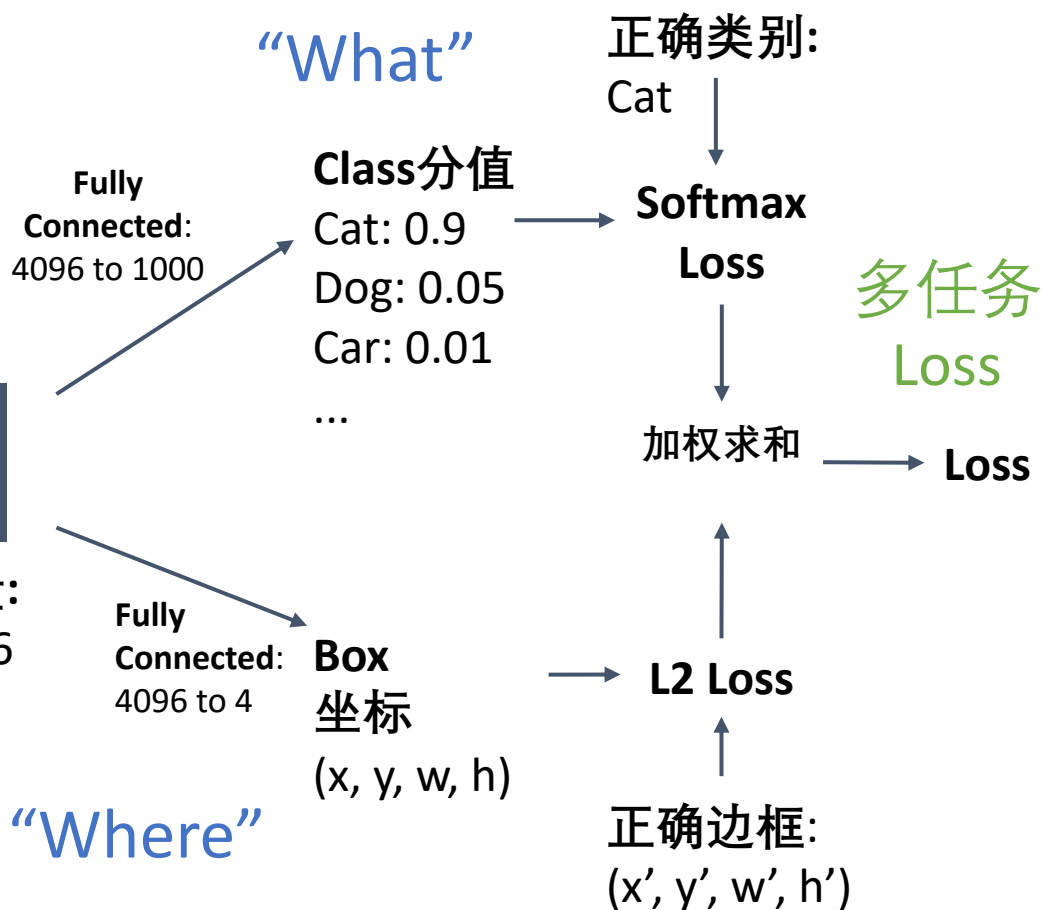


[This image is CC0 public domain](#)

定位被看成一个
回归问题!

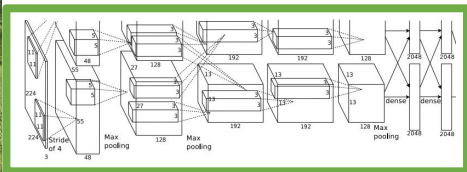


向量:
4096



检测单一目标

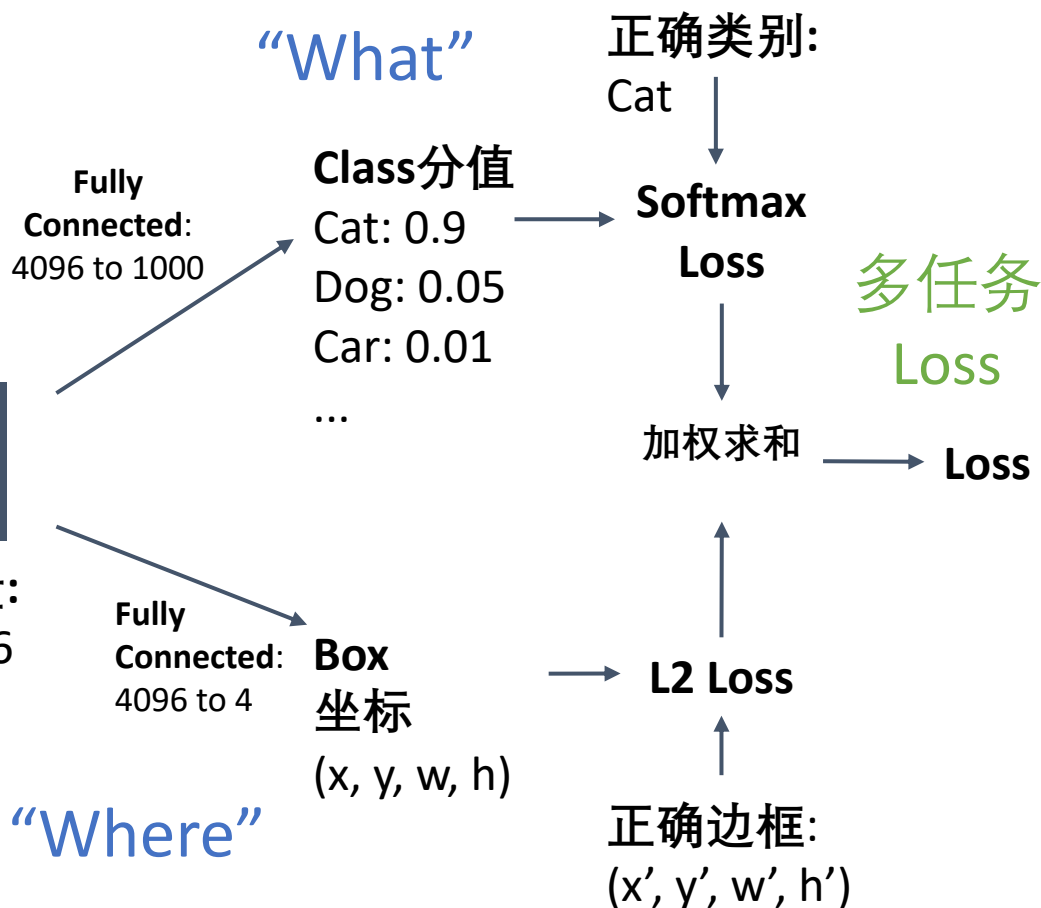
经常通过
ImageNet预训练
获得(迁移学习)



[This image is CC0 public domain](#)

定位被看成一个
回归问题!

向量:
4096



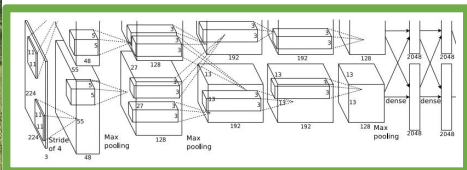
检测单一目标

经常通过
ImageNet预训练
获得(迁移学习)

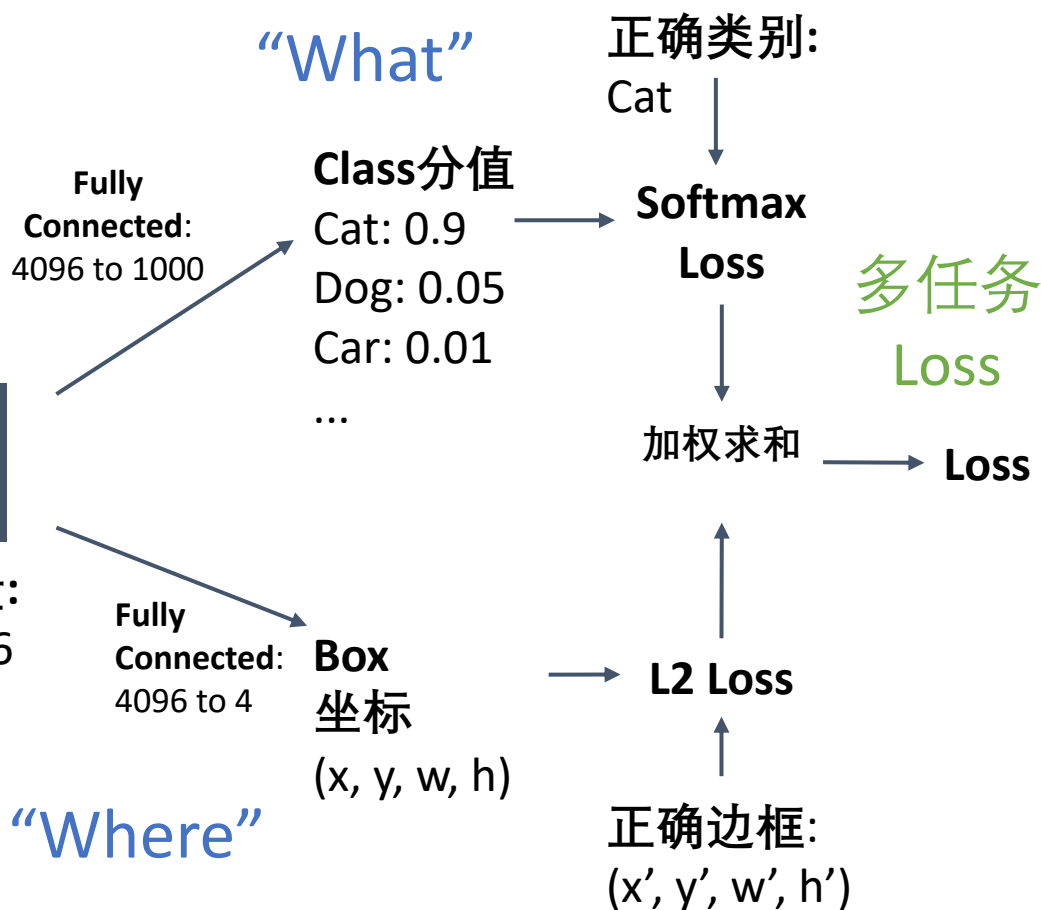


This image is CC0 public domain

定位被看成一个
回归问题!



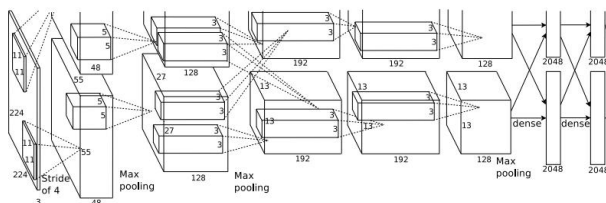
向量:
4096



问题: 图像不只有一个
目标!

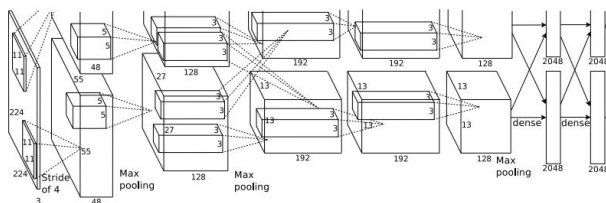
检测多个目标

每张图片需要多个不同的输出



CAT: (x, y, w, h)

4 个

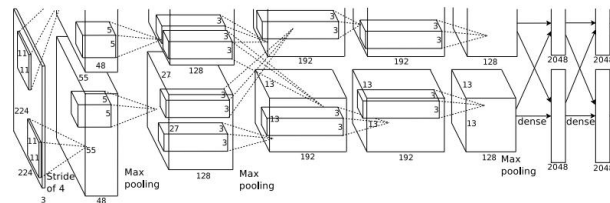


DOG: (x, y, w, h)

DOG: (x, y, w, h)

CAT: (x, y, w, h)

12 个



DUCK: (x, y, w, h)

DUCK: (x, y, w, h)

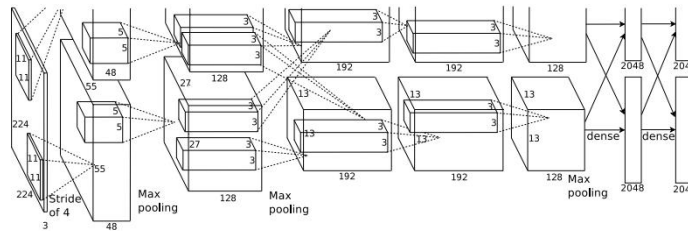
许多个!

....

[Duck image](#) is free to use under the [Pixabay license](#)

检测多个目标: 滑动窗

对图像截取的不同子区域使用
CNN分类器分类, CNN 将子区域
分类为目标或背景



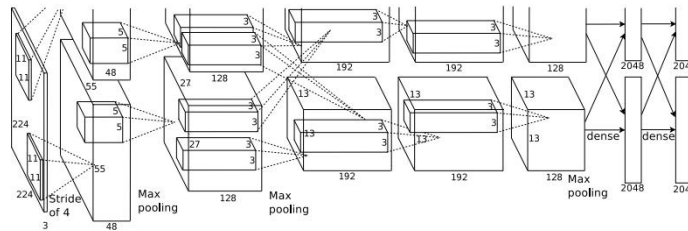
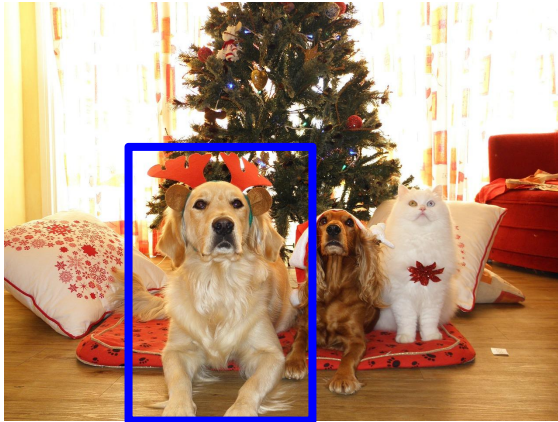
Dog? **NO**

Cat? **NO**

Background? **YES**

检测多个目标: 滑动窗

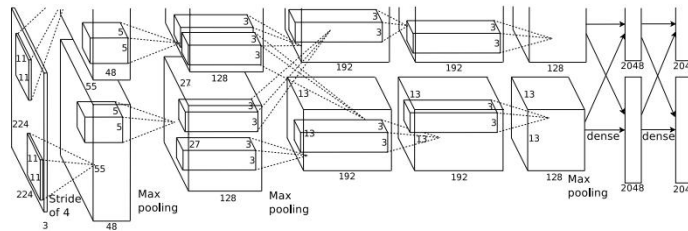
对图像截取的不同子区域使用
CNN分类器分类, CNN 将子区域
分类为目标或背景



Dog? **YES**
Cat? **NO**
Background? **NO**

检测多个目标: 滑动窗

对图像截取的不同子区域使用
CNN分类器分类, CNN 将子区域
分类为目标或背景



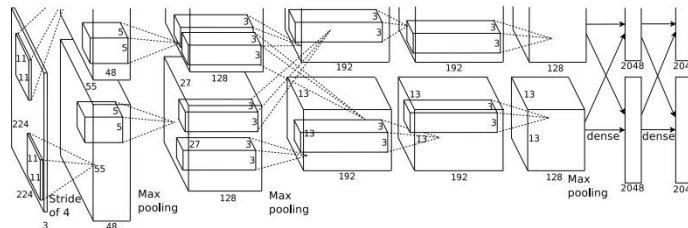
Dog? **YES**

Cat? **NO**

Background? **NO**

检测多个目标: 滑动窗

对图像截取的不同子区域使用
CNN分类器分类, CNN 将子区域
分类为目标或背景



Dog? **NO**
Cat? **YES**
Background? **NO**

检测多个目标: 滑动窗

对图像截取的不同子区域使用
CNN分类器分类, CNN 将子区域
分类为目标或背景

问题: 有多少个大小为 $H \times W$ 的
可能候选子区域?



检测多个目标: 滑动窗

对图像截取的不同子区域使用
CNN分类器分类, CNN 将子区域
分类为目标或背景



问题: 有多少个大小为 $H \times W$ 的可能候选子区域?

考虑大小为 $h \times w$ 的框:

x 可能位置: $W - w + 1$

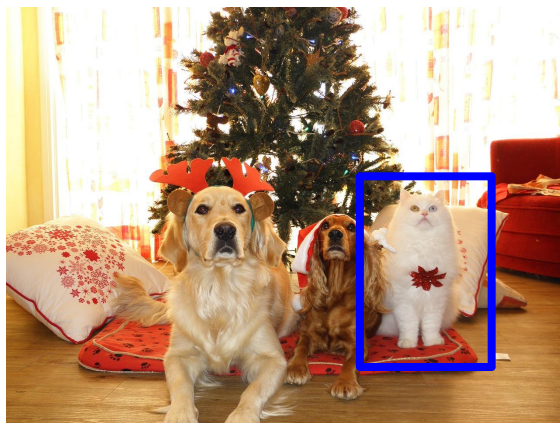
y 可能位置: $H - h + 1$

可能位置:

$(W - w + 1) * (H - h + 1)$

检测多个目标: 滑动窗

对图像截取的不同子区域使用
CNN分类器分类, CNN 将子区域
分类为目标或背景



问题: 有多少个大小为 $H \times W$ 的可能候选子区域?

考虑大小为 $h \times w$ 的框:

x 可能位置: $W - w + 1$

y 可能位置: $H - h + 1$

可能位置:

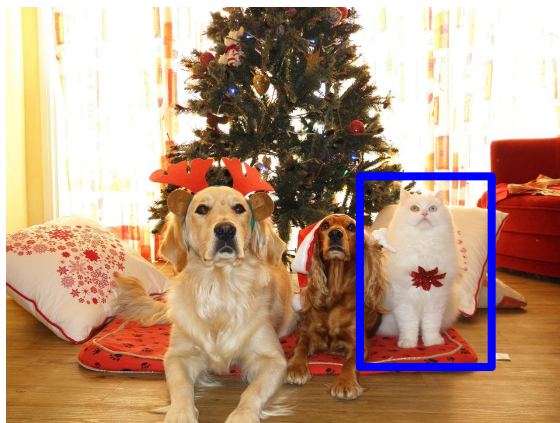
$(W - w + 1) * (H - h + 1)$

总数:

$$\sum_{h=1}^H \sum_{w=1}^W (W - w + 1)(H - h + 1)$$

$$= \frac{H(H+1)}{2} \frac{W(W+1)}{2}$$

检测多个目标: 滑动窗



对图像截取的不同子区域使用
CNN分类器分类, CNN 将子区域
分类为目标或背景

问题: 有多少个大小为 $H \times W$ 的可
能候选子区域?

考虑大小为 $h \times w$ 的框:

x 可能位置: $W - w + 1$

y 可能位置: $H - h + 1$

可能位置:

$(W - w + 1) * (H - h + 1)$

800 x 600 大小图
像有 ~58M 个框!
全部计算不可实
现

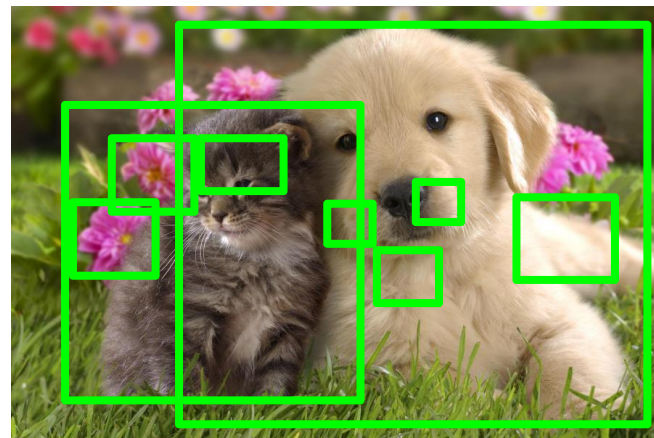
总数:

$$\sum_{h=1}^H \sum_{w=1}^W (W - w + 1)(H - h + 1)$$

$$= \frac{H(H+1)}{2} \frac{W(W+1)}{2}$$

候选区域(Region Proposals)

- 确定少量候选框尽可能覆盖所有目标
- 经常基于启发式方法: e.g. 寻找“blob-like” 图像区域
- 相对快速运行的方法; e.g. 在几秒钟中选择性搜索出2000 个候选区域



Alexe et al, "Measuring the objectness of image windows", TPAMI 2012
Uijlings et al, "Selective Search for Object Recognition", IJCV 2013
Cheng et al, "BING: Binarized normed gradients for objectness estimation at 300fps", CVPR 2014
Zitnick and Dollar, "Edge boxes: Locating object proposals from edges", ECCV 2014

R-CNN: Region-Based CNN

输入
图像



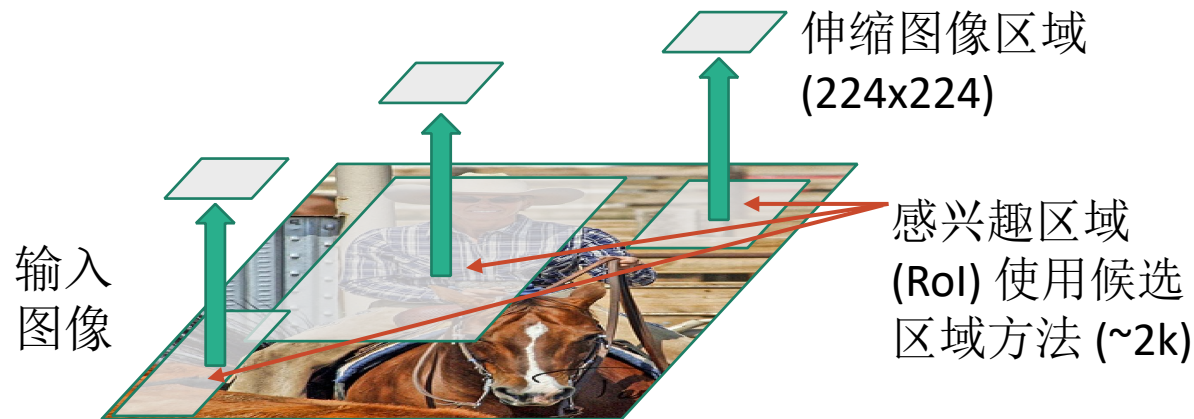
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN: Region-Based CNN



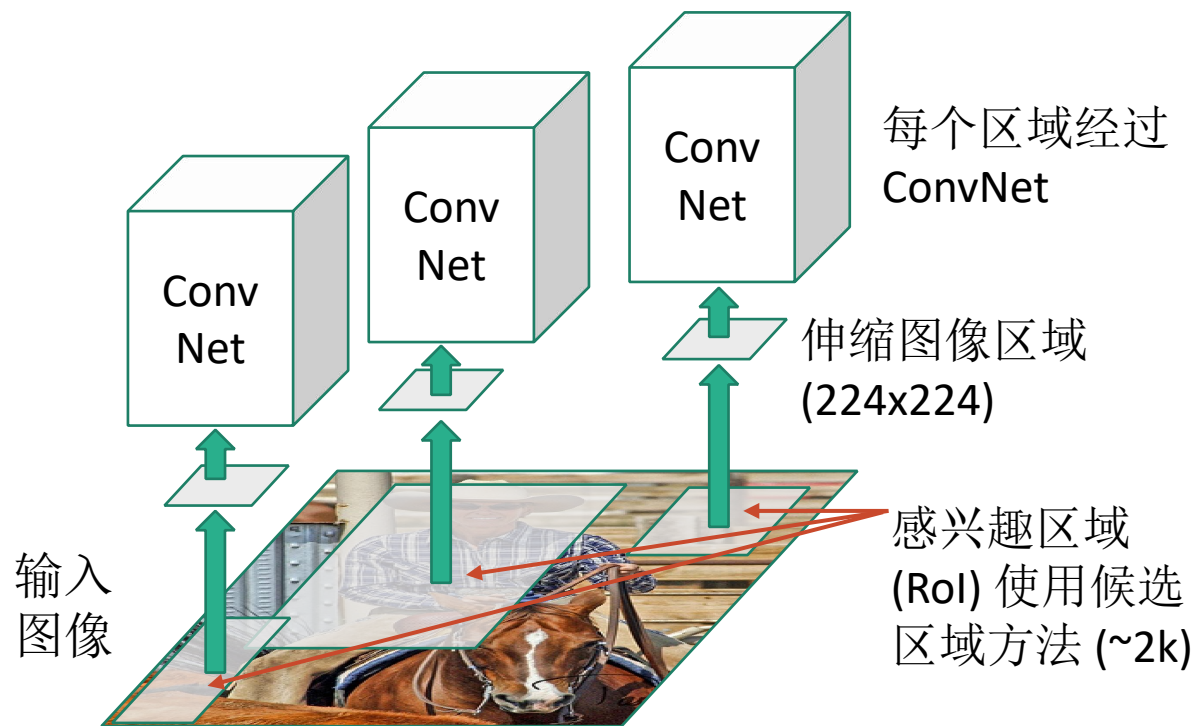
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN: Region-Based CNN



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

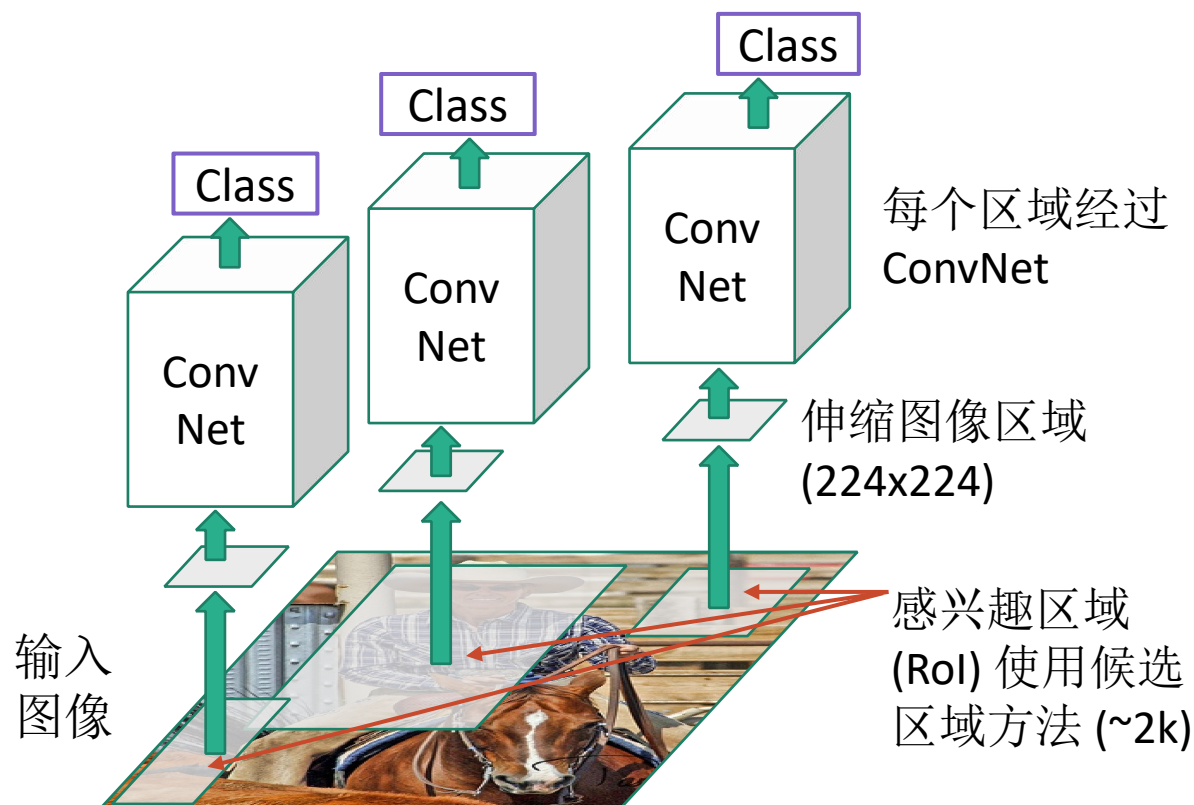
R-CNN: Region-Based CNN



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

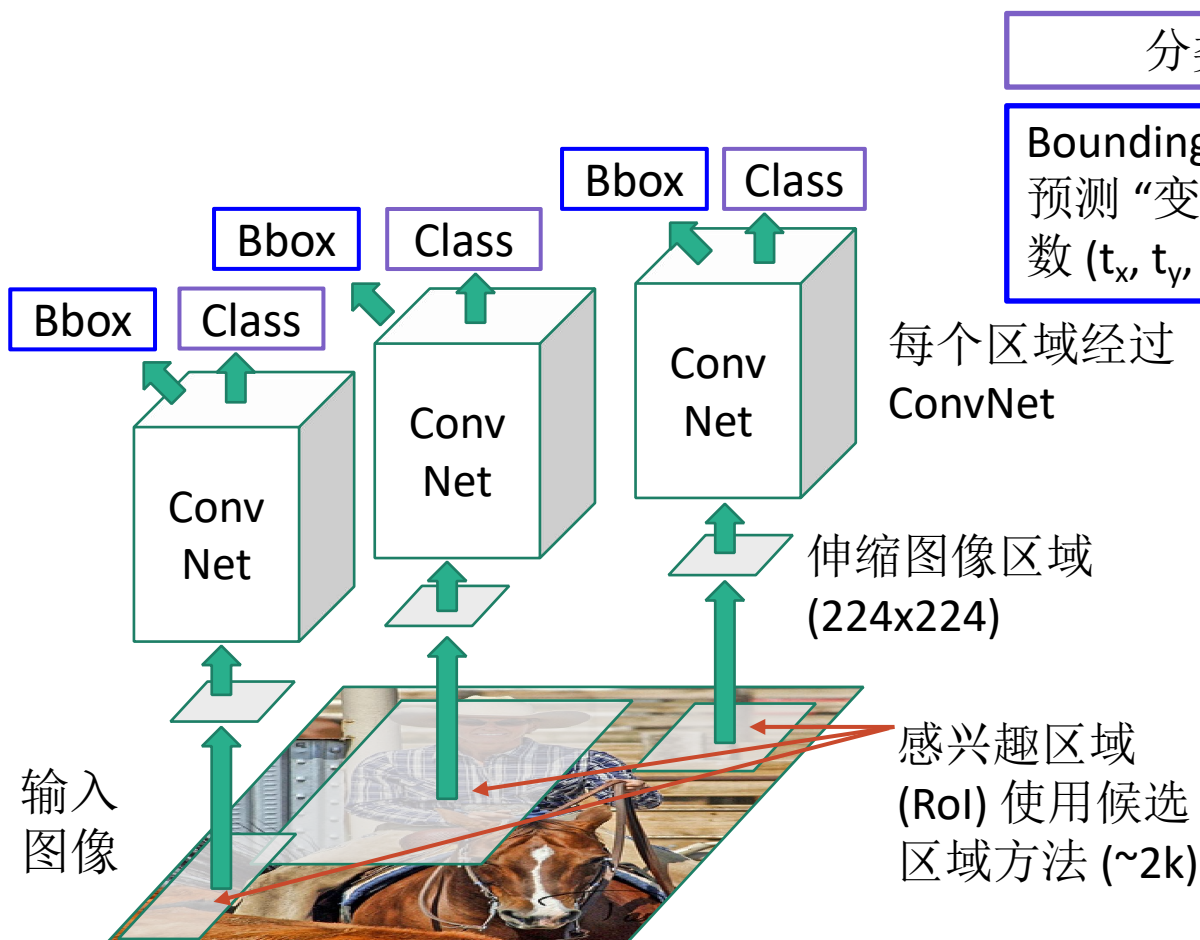
R-CNN: Region-Based CNN

分类每个候选子区域



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN: Region-Based CNN



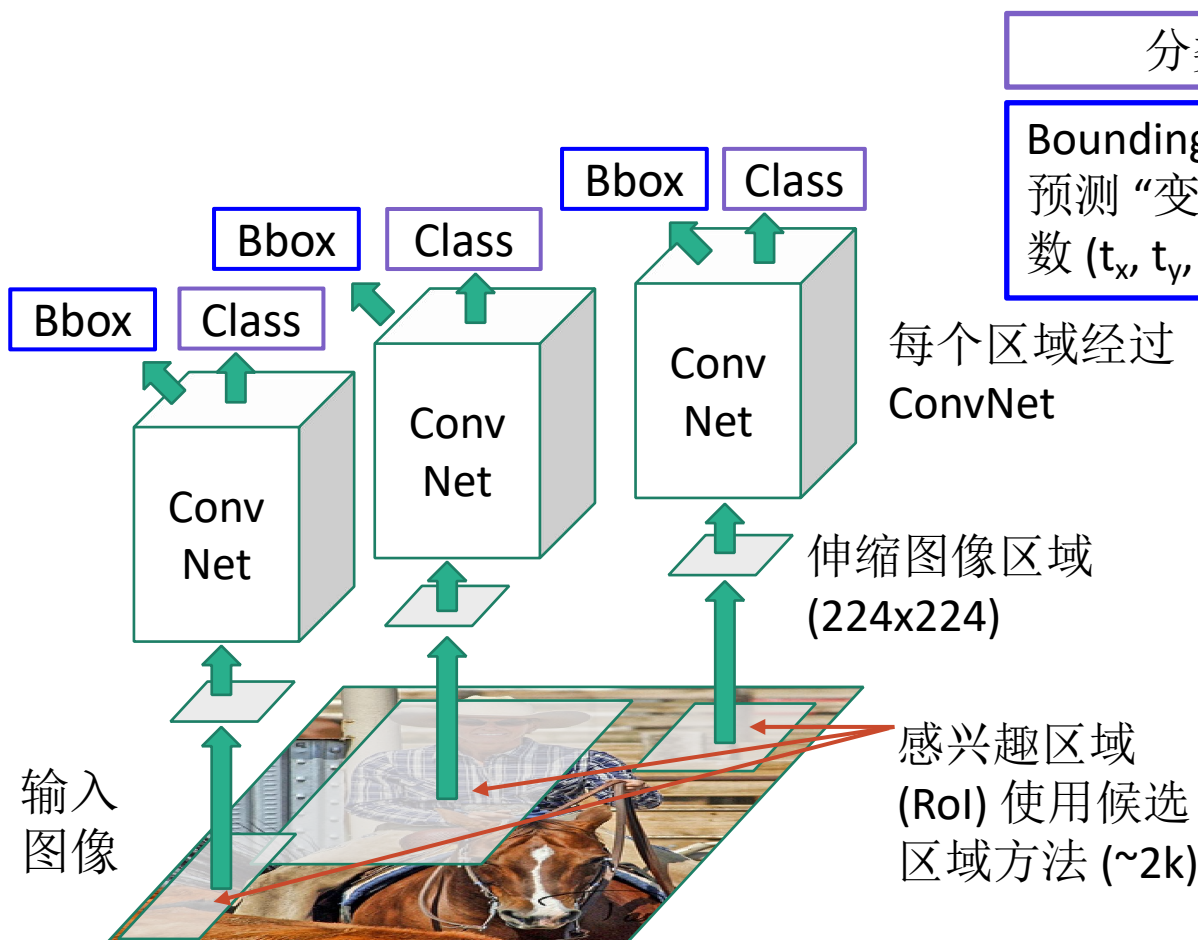
分类每个候选子区域

Bounding box 回归:

预测“变换参数”纠正RoI: 4个参数 (t_x, t_y, t_h, t_w)

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN: Region-Based CNN



分类每个候选子区域

Bounding box 回归:

预测“变换参数”纠正RoI: 4个参数 (t_x, t_y, t_h, t_w)

候选区域: (p_x, p_y, p_h, p_w)

变换: (t_x, t_y, t_h, t_w)

输出框: (b_x, b_y, b_h, b_w)

框的变换:

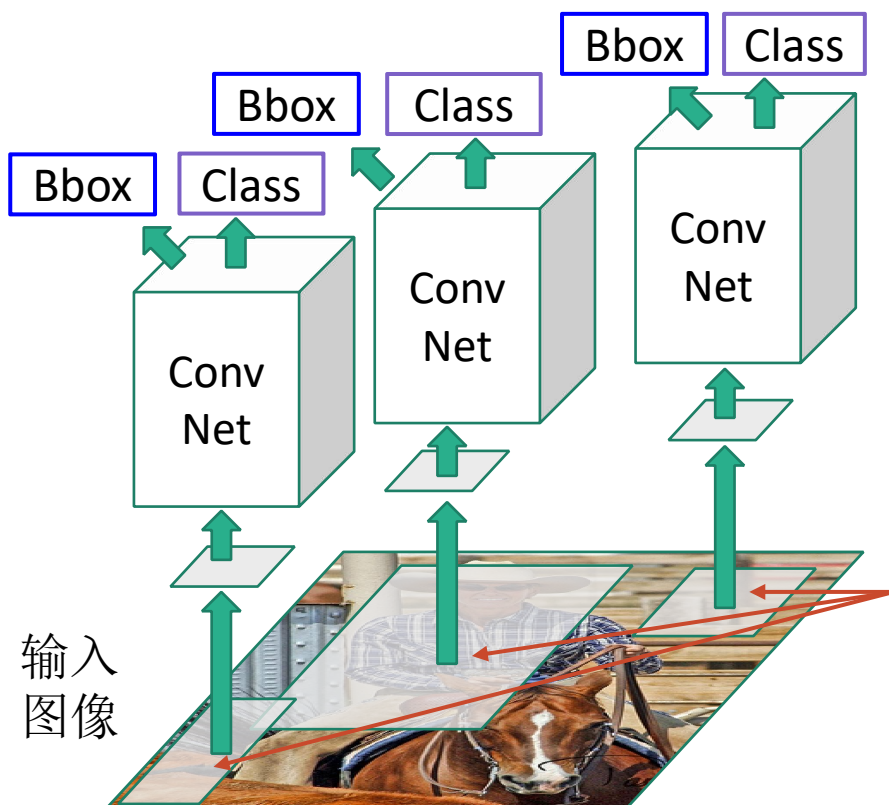
$$b_x = p_x + p_w t_x \quad b_y = p_y + p_h t_y$$

Log-尺度空间变换:

$$b_w = p_w \exp(t_w) \quad b_h = p_h \exp(t_h)$$

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN: 测试时间



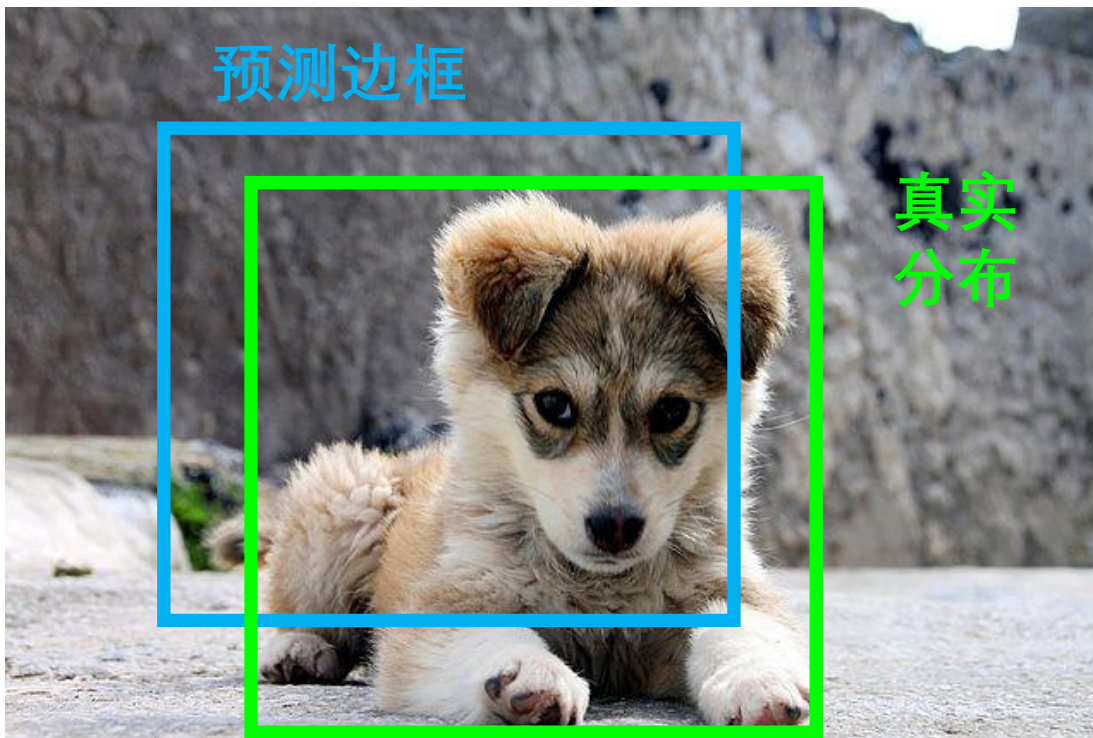
输入: 单RGB图像

1. 运行候选区域算法获取~2000 候选区域
2. 伸缩每个区域到224x224 并各自独立的经过CNN 预测类别分值和 bbox 变换参数
3. 根据分值选择候选区域输出
(选择方法: 阈值, 或按类别? 或取最大的 K 候选区域?)
4. 与真实分布框比较

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

检测框比较: 交叠率Intersection over Union (IoU)

如何计算检测边框和真实边框
差异?



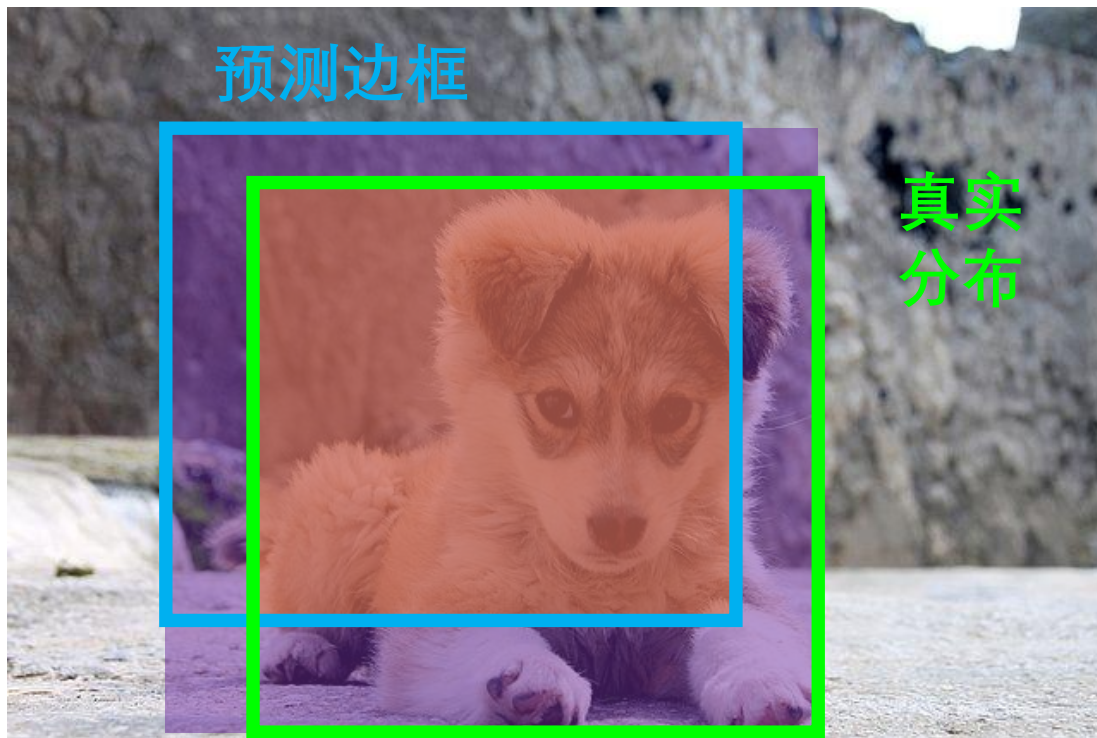
[Puppy image](#) is licensed under [CC-A 2.0 Generic license](#). Bounding boxes and text added by Justin Johnson.

检测框比较: 交叠率Intersection over Union (IoU)

如何计算检测边框和真实边框差异?

Intersection over Union (IoU)
(也称为“Jaccard 相似系数” or
“Jaccard 指数”):

$$\frac{\text{相交的面积}}{\text{合并的面积}}$$



[Puppy image](#) is licensed under [CC-A 2.0 Generic license](#). Bounding boxes and text added by Justin Johnson.

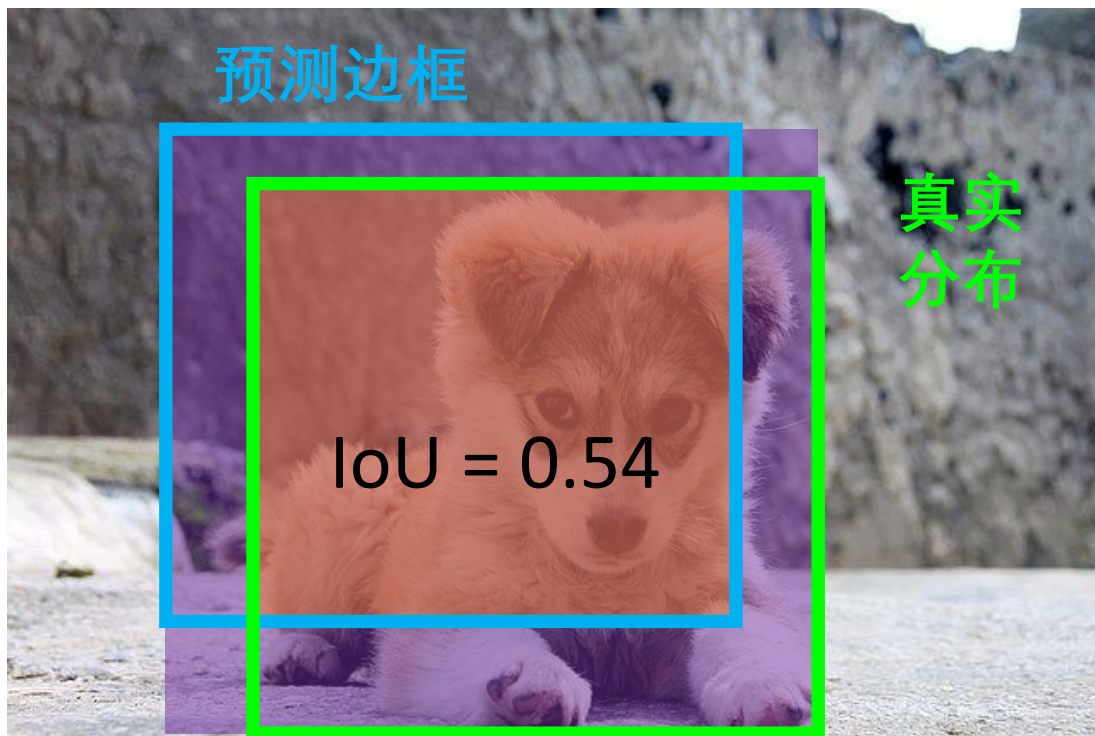
检测框比较: 交叠率Intersection over Union (IoU)

如何计算检测边框和真实边框差异?

Intersection over Union (IoU)
(也称为“Jaccard 相似系数” or “Jaccard 指数”):

$$\frac{\text{相交的面积}}{\text{合并的面积}}$$

$\text{IoU} > 0.5$ 是可接受的



Puppy image is licensed under [CC-A 2.0 Generic license](#). Bounding boxes and text added by Justin Johnson.

检测框比较: 交叠率Intersection over Union (IoU)

如何计算检测边框和真实边框差异?

Intersection over Union (IoU)
(也称为“Jaccard 相似系数” or “Jaccard 指数”):

$$\frac{\text{相交的面积}}{\text{合并的面积}}$$

$\text{IoU} > 0.5$ 是可接受的,
 $\text{IoU} > 0.7$ 是足够好的,



[Puppy image](#) is licensed under [CC-A 2.0 Generic license](#). Bounding boxes and text added by Justin Johnson.

检测框比较: 交叠率Intersection over Union (IoU)

如何计算检测边框和真实边框差异?

Intersection over Union (IoU)
(也称为“Jaccard 相似系数” or “Jaccard 指数”):

$$\frac{\text{相交的面积}}{\text{合并的面积}}$$

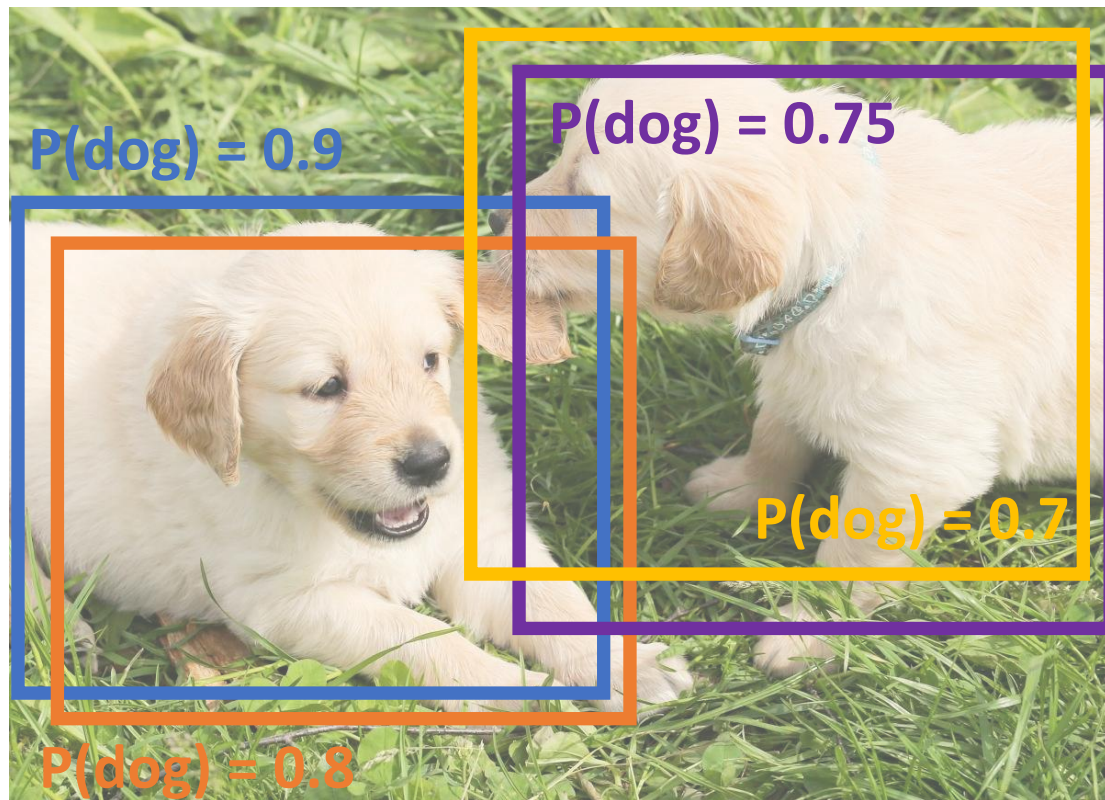
IoU > 0.5是可接受的,
IoU > 0.7是足够好的,
IoU > 0.9是几乎完美的



[Puppy image](#) is licensed under [CC-A 2.0 Generic license](#). Bounding boxes and text added by Justin Johnson.

重叠框

问题: 目标检测子经常出现许多重叠结果:



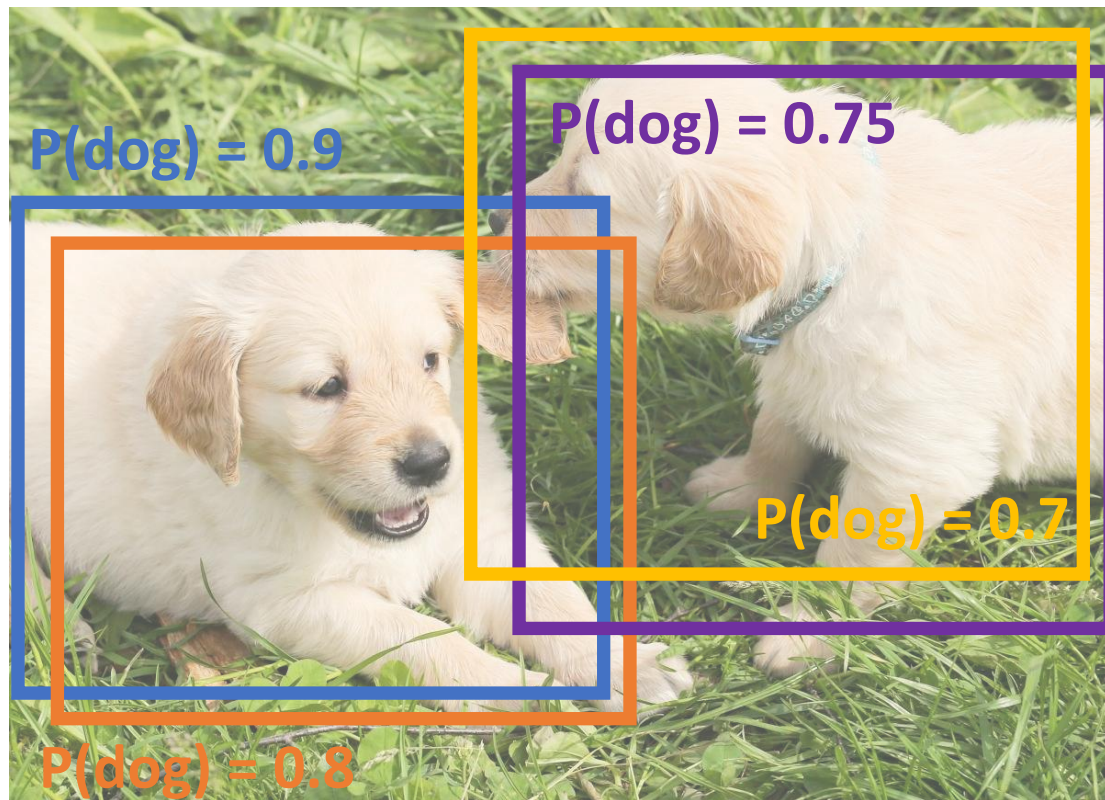
Puppy image is CC0 Public Domain

重叠框: 非最大化抑制(NMS)

问题: 目标检测子经常出现许多重叠结果:

方案: 对粗检测结果进行非最大化抑制(NMS)

1. 选择下一个高分值边框
2. 去除IoU>阈值(e.g. 0.7)的低分值边框
3. 如果仍然存在满足条件的边框, GOTO 1



Puppy image is CC0 Public Domain

重叠框: 非最大化抑制(NMS)

问题: 目标检测子经常出现许多重叠结果:

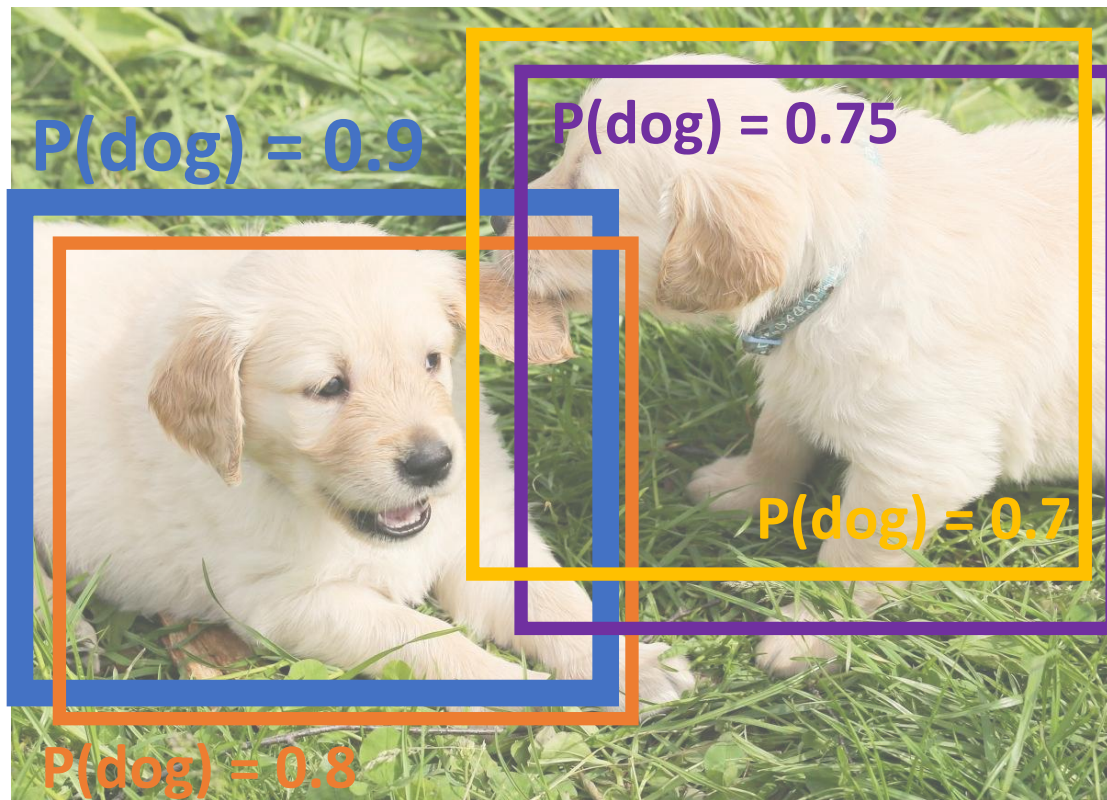
方案: 对粗检测结果进行非最大化抑制(NMS)

1. 选择下一个高分值边框
2. 去除IoU>阈值(e.g. 0.7)的低分值边框
3. 如果仍然存在满足条件的边框, GOTO 1

$$\text{IoU}(\text{blue}, \text{orange}) = \mathbf{0.78}$$

$$\text{IoU}(\text{blue}, \text{purple}) = 0.05$$

$$\text{IoU}(\text{blue}, \text{yellow}) = 0.07$$



Puppy image is CC0 Public Domain

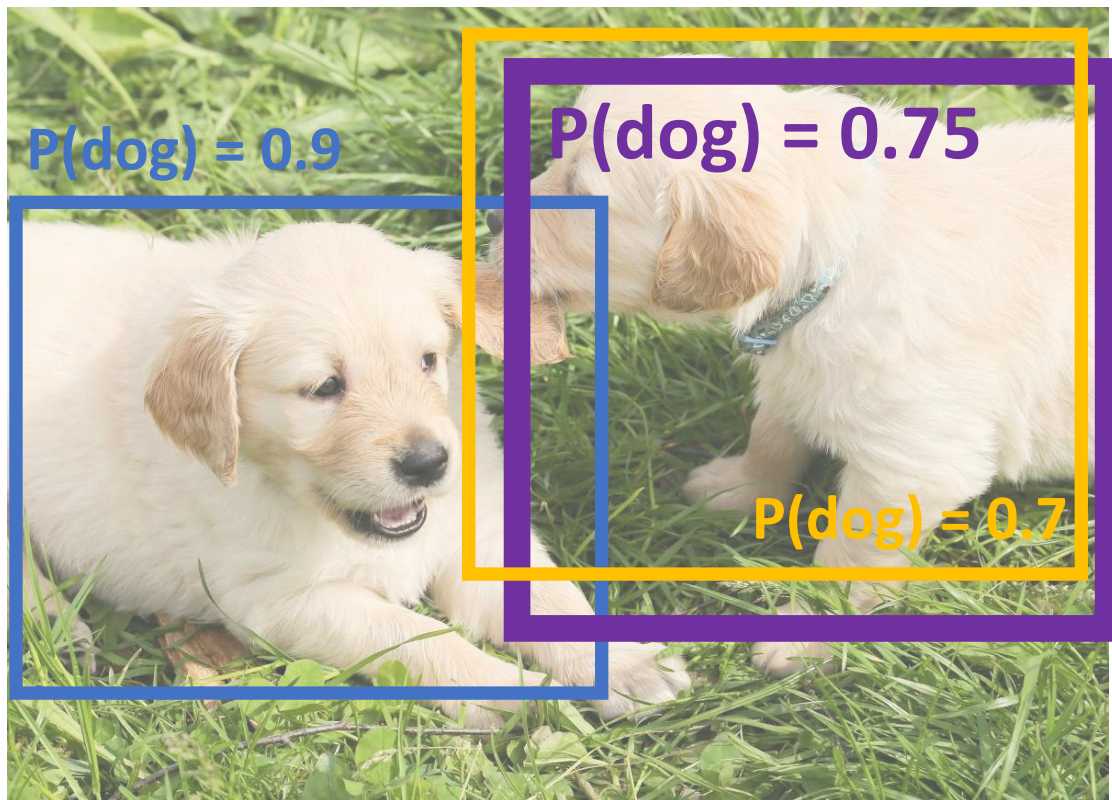
重叠框: 非最大化抑制(NMS)

问题: 目标检测子经常出现许多重叠结果:

方案: 对粗检测结果进行非最大化抑制(NMS)

1. 选择下一个高分值边框
2. 去除IoU>阈值(e.g. 0.7)的低分值边框
3. 如果仍然存在满足条件的边框, GOTO 1

$$\text{IoU}(\text{■}, \text{■}) = 0.74$$



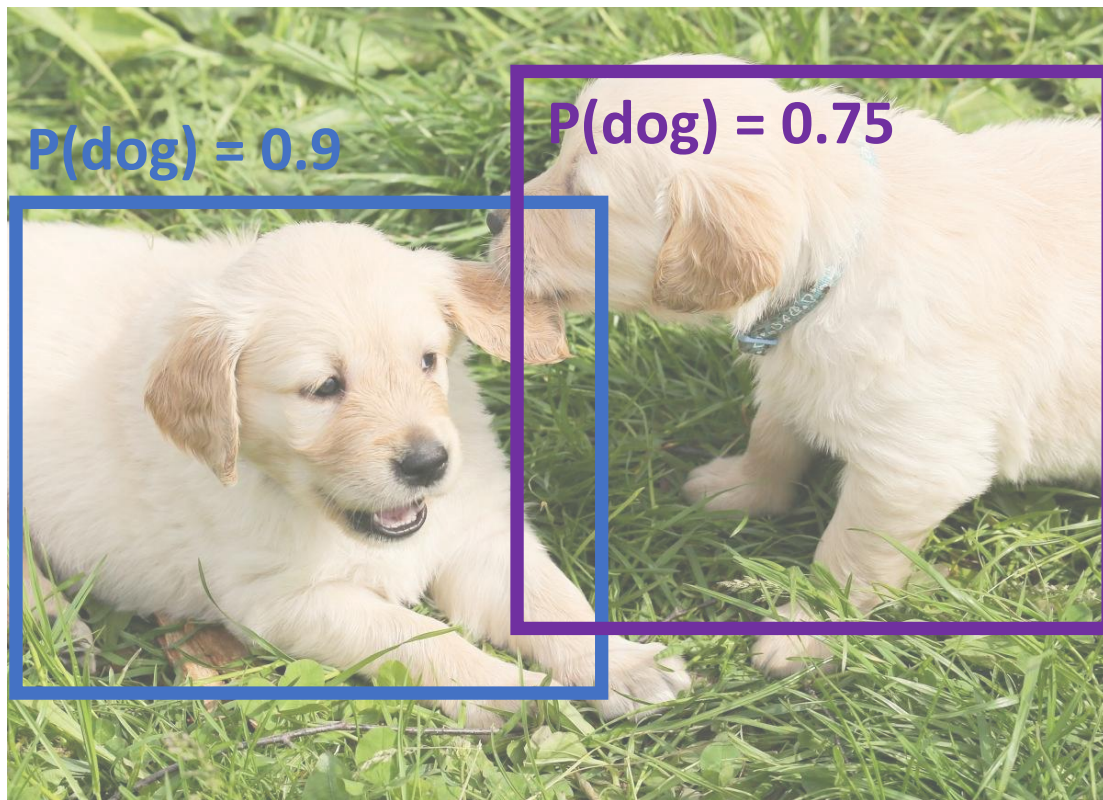
Puppy image is CC0 Public Domain

重叠框: 非最大化抑制(NMS)

问题: 目标检测子经常出现许多重叠结果:

方案: 对粗检测结果进行非最大化抑制(NMS)

1. 选择下一个高分值边框
2. 去除IoU>阈值(e.g. 0.7)的低分值边框
3. 如果仍然存在满足条件的边框, GOTO 1



Puppy image is CC0 Public Domain

重叠框: 非最大化抑制(NMS)

问题: 目标检测子经常出现许多重叠结果:

方案: 对粗检测结果进行非最大化抑制(NMS)

1. 选择下一个高分值边框
2. 去除IoU>阈值(e.g. 0.7)的低分值边框
3. 如果仍然存在满足条件的边框, GOTO 1

问题: 如果目标本身高度重叠, NMS 可能将“好”边框抑制掉了... 没有好的方案



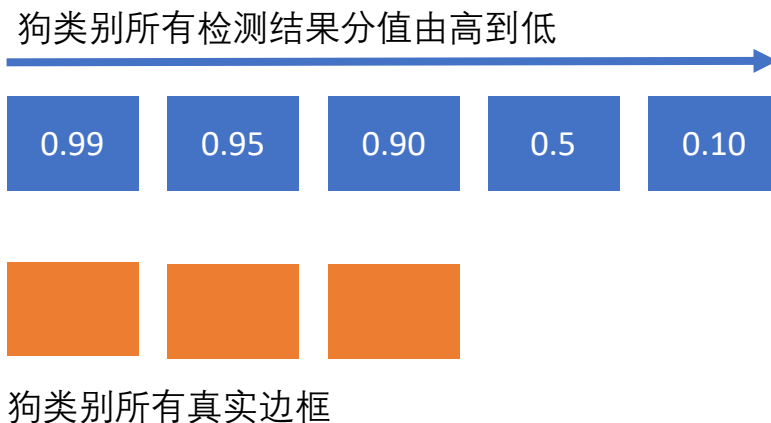
[Crowd image](#) is free for commercial use under the [Pixabay license](#)

目标检测子评估: 均值平均精确率Mean Average Precision (mAP)

1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积

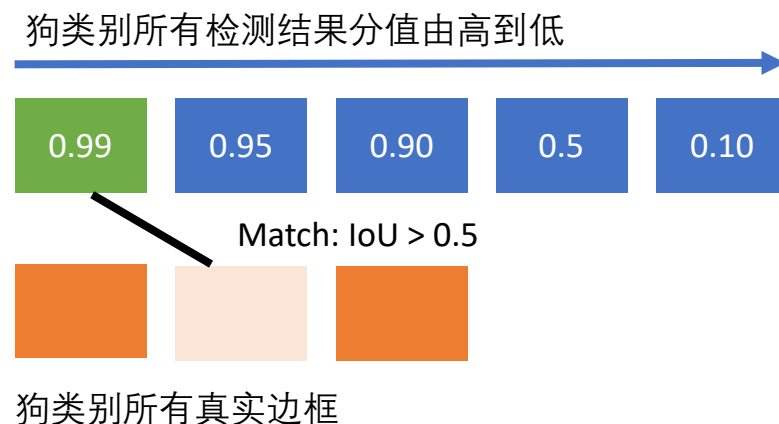
目标检测子评估: 均值平均精确率Mean Average Precision (mAP)

1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积
 1. 对于每次检测 (从高分值到低分值)



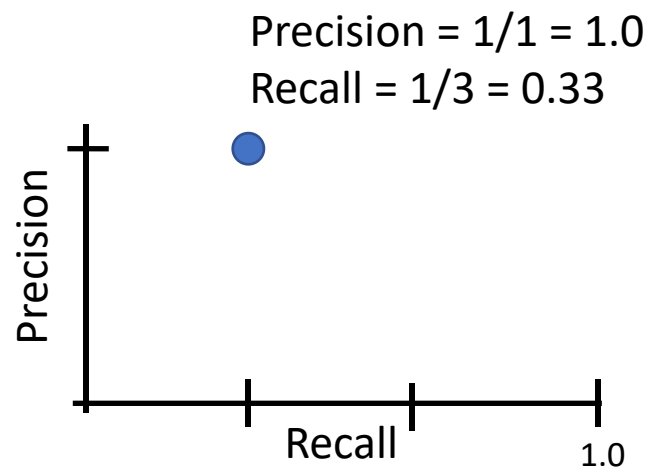
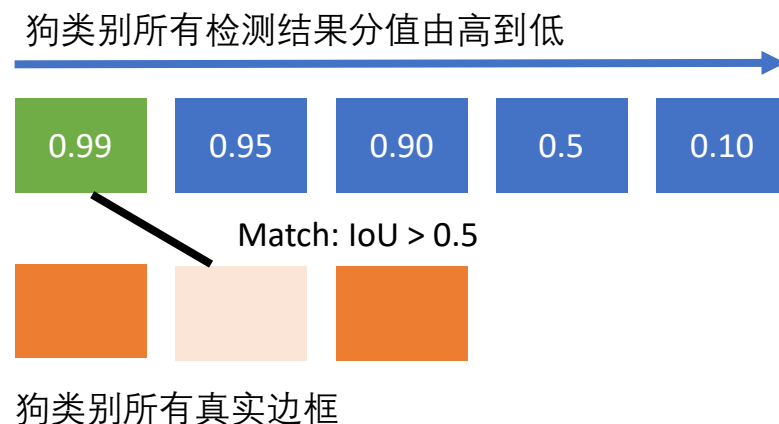
目标检测子评估: 均值平均精确率Mean Average Precision (mAP)

1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积
 1. 对于每次检测 (从高分值到低分值)
 1. 如果其与某个GT边框 $\text{IoU} > 0.5$, 标记其为正并且抑制这个GT
 2. 否则, 标记其为负



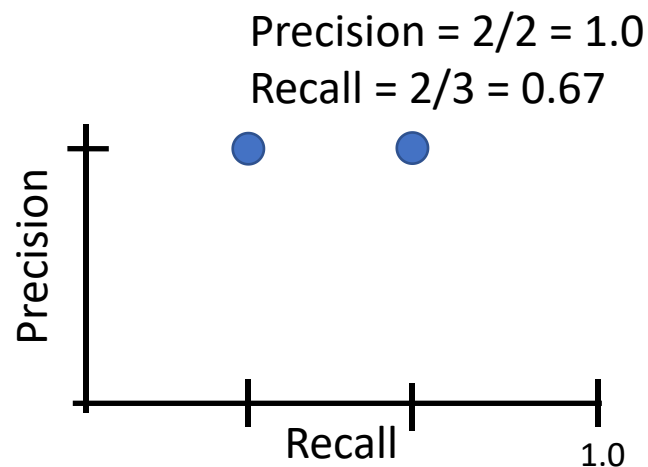
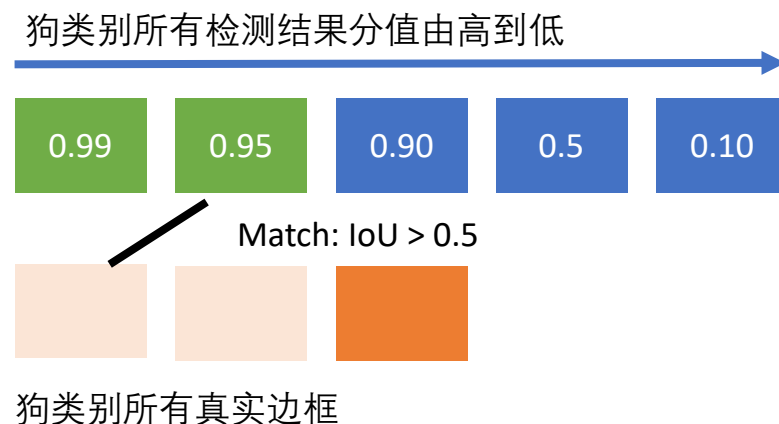
目标检测子评估: 均值平均精确率Mean Average Precision (mAP)

1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积
 1. 对于每次检测 (从高分值到低分值)
 1. 如果其与某个GT边框 $\text{IoU} > 0.5$, 标记其为正并且抑制这个GT
 2. 否则, 标记其为负
 3. 在PR曲线下画一个点



目标检测子评估: 均值平均精确率Mean Average Precision (mAP)

1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积
 1. 对于每次检测 (从高分值到低分值)
 1. 如果其与某个GT边框 $\text{IoU} > 0.5$, 标记其为正并且抑制这个GT
 2. 否则, 标记其为负
 3. 在PR曲线下画一个点



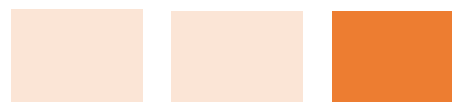
目标检测子评估: 均值平均精确率 Mean Average Precision (mAP)

1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积
 1. 对于每次检测 (从高分值到低分值)
 1. 如果其与某个GT边框 $\text{IoU} > 0.5$, 标记其为正并且抑制这个GT
 2. 否则, 标记其为负
 3. 在PR曲线下画一个点

狗类别所有检测结果分值由高到低



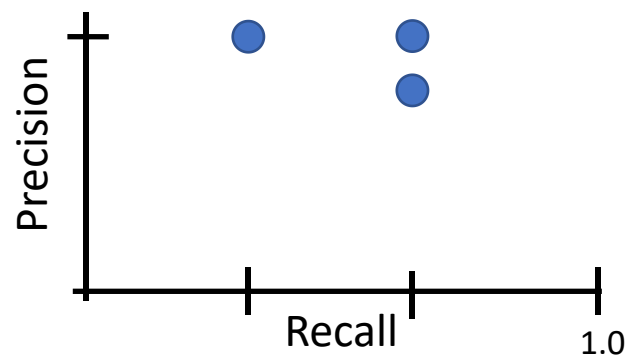
No match > 0.5 IoU with GT



狗类别所有真实边框

$$\text{Precision} = 2/3 = 0.67$$

$$\text{Recall} = 2/3 = 0.67$$



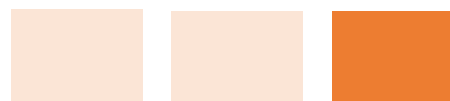
目标检测子评估: 均值平均精确率 Mean Average Precision (mAP)

1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积
 1. 对于每次检测 (从高分值到低分值)
 1. 如果其与某个GT边框 $\text{IoU} > 0.5$, 标记其为正并且抑制这个GT
 2. 否则, 标记其为负
 3. 在PR曲线下画一个点

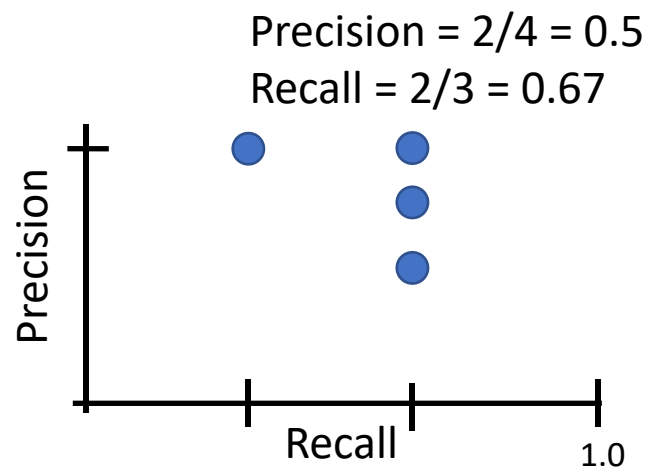
狗类别所有检测结果分值由高到低



No match > 0.5 IoU with GT



狗类别所有真实边框



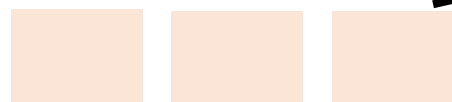
目标检测子评估: 均值平均精确率 Mean Average Precision (mAP)

1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积
 1. 对于每次检测 (从高分值到低分值)
 1. 如果其与某个GT边框 $\text{IoU} > 0.5$, 标记其为正并且抑制这个GT
 2. 否则, 标记其为负
 3. 在PR曲线下画一个点

狗类别所有检测结果分值由高到低



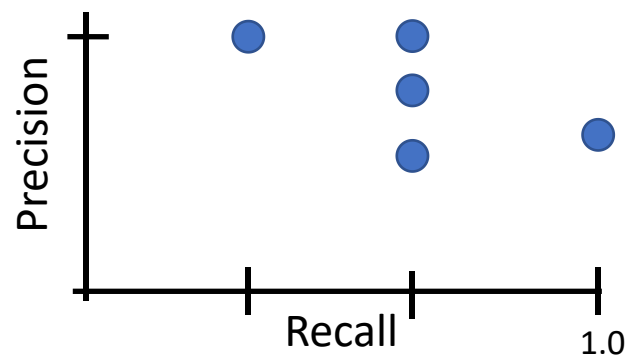
Match: > 0.5 IoU



狗类别所有真实边框

$$\text{Precision} = 3/5 = 0.6$$

$$\text{Recall} = 3/3 = 1.0$$



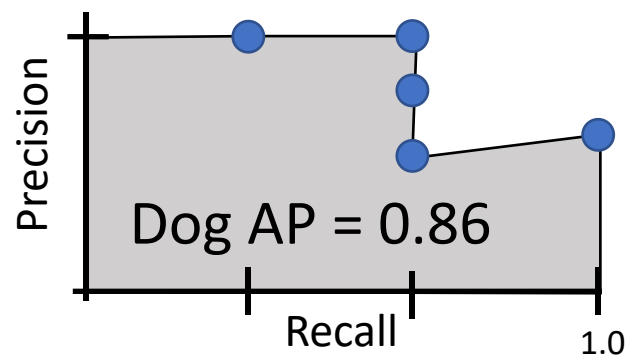
目标检测子评估: 均值平均精确率Mean Average Precision (mAP)

1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积
 1. 对于每次检测 (从高分值到低分值)
 1. 如果其与某个GT边框 $\text{IoU} > 0.5$, 标记其为正并且抑制这个GT
 2. 否则, 标记其为负
 3. 在PR曲线下画一个点
 2. 平均精确率(AP) = PR曲线下的面积

狗类别所有检测结果分值由高到低



狗类别所有真实边框



目标检测子评估: 均值平均精确率 Mean Average Precision (mAP)

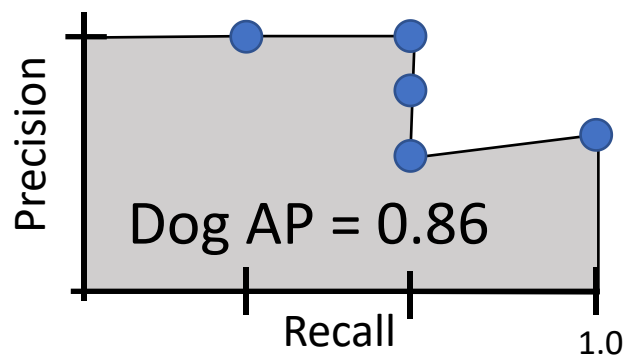
1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积
 1. 对于每次检测 (从高分值到低分值)
 1. 如果其与某个GT边框 $\text{IoU} > 0.5$, 标记其为正并且抑制这个GT
 2. 否则, 标记其为负
 3. 在PR曲线下画一个点
 2. 平均精确率(AP) = PR曲线下的面积

如何得到 $\text{AP} = 1.0$: 对于所有 $\text{IoU} > 0.5$ 的GT边框, 没有“虚警” 分值高于“正确结果”

狗类别所有检测结果分值由高到低



狗类别所有真实边框



目标检测子评估: 均值平均精确率Mean Average Precision (mAP)

1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积
 1. 对于每次检测 (从高分值到低分值)
 1. 如果其与某个GT边框 $\text{IoU} > 0.5$, 标记其为正并且抑制这个GT
 2. 否则, 标记其为负
 3. 在PR曲线下画一个点
 2. 平均精确率(AP) = PR曲线下的面积
3. 均值平均精确率(mAP) = 平均各个类别的AP

Car AP = 0.65

Cat AP = 0.80

Dog AP = 0.86

mAP@0.5 = 0.77

目标检测子评估: 均值平均精确率 Mean Average Precision (mAP)

1. 对所有测试图像进行目标检测 (带NMS)
2. 对每个类别, 计算平均精确率(AP) = PR曲线下的面积
 1. 对于每次检测 (从高分值到低分值)
 1. 如果其与某个GT边框 $\text{IoU} > 0.5$, 标记其为正并且抑制这个GT
 2. 否则, 标记其为负
 3. 在PR曲线下画一个点
 2. 平均精确率(AP) = PR曲线下的面积
3. 均值平均精确率(mAP) = 平均各个类别的AP
4. 对于 “COCO mAP”: 在不同的IoU阈值(0.5, 0.55, 0.6, ..., 0.95)下分别计算 mAP@thresh 并取平均

$$\text{mAP@0.5} = 0.77$$

$$\text{mAP@0.55} = 0.71$$

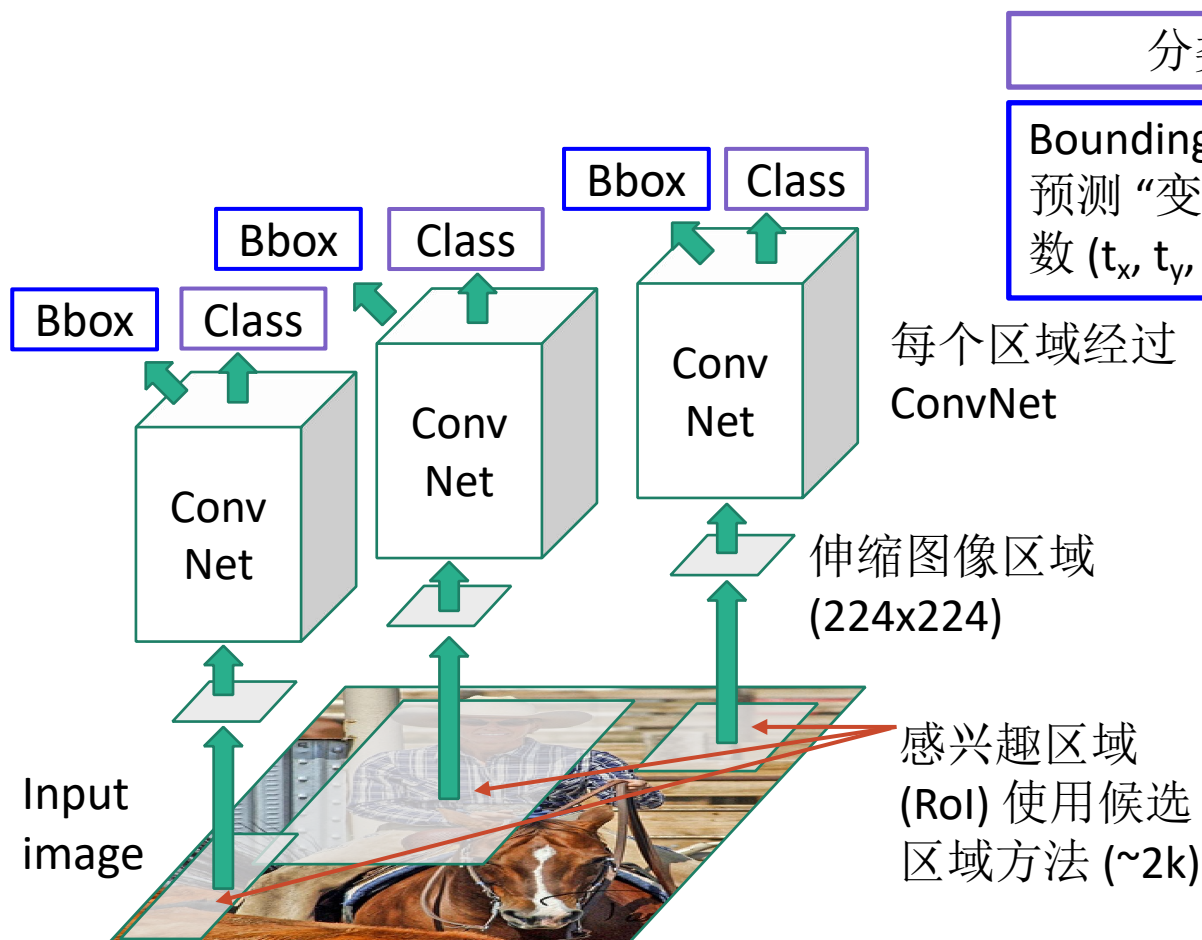
$$\text{mAP@0.60} = 0.65$$

...

$$\text{mAP@0.95} = 0.2$$

$$\text{COCO mAP} = 0.4$$

R-CNN: Region-Based CNN



分类每个候选子区域

Bounding box 回归:

预测“变换参数”纠正RoI: 4 个参数 (t_x, t_y, t_h, t_w)

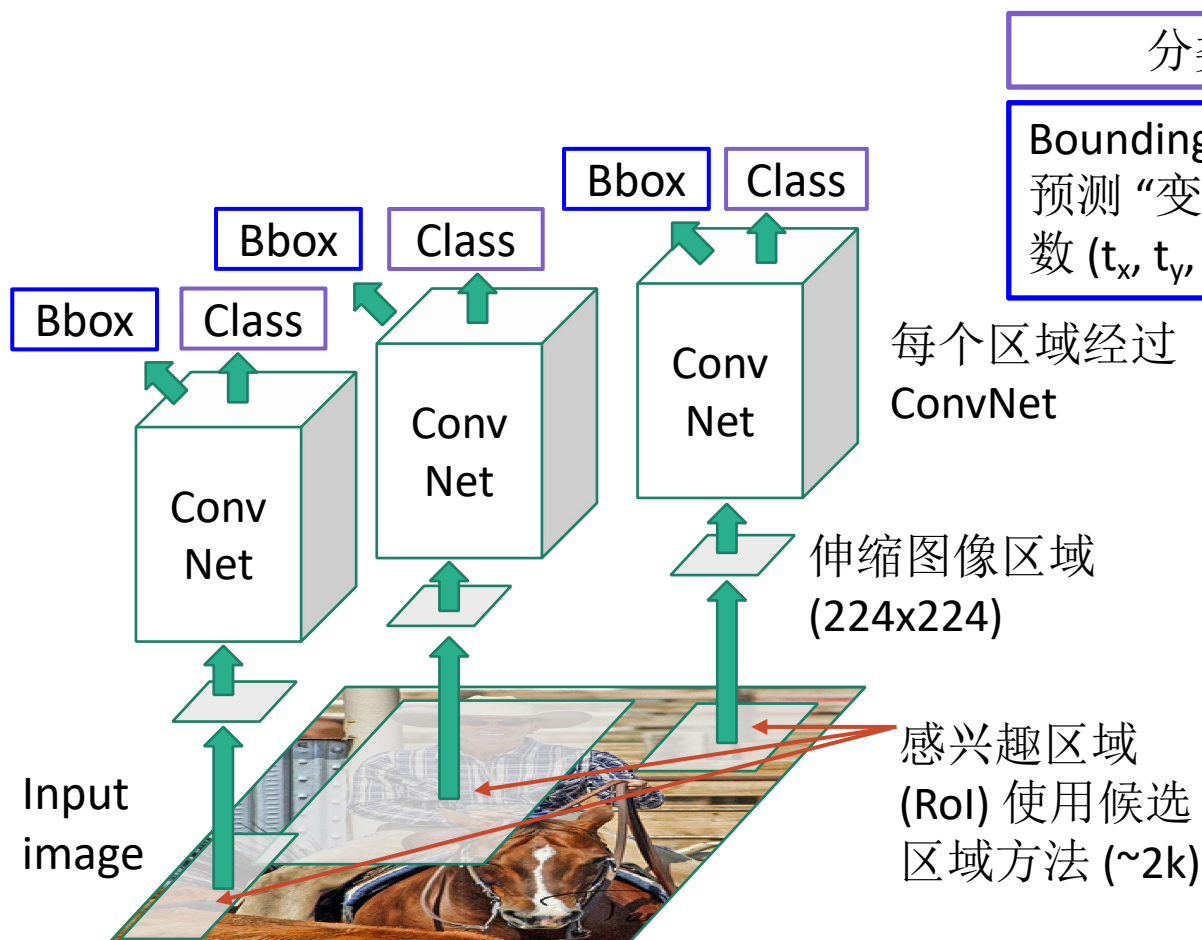
每个区域经过
ConvNet

伸缩图像区域
(224x224)

感兴趣区域
(RoI) 使用候选
区域方法 (~2k)

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN: Region-Based CNN



分类每个候选子区域

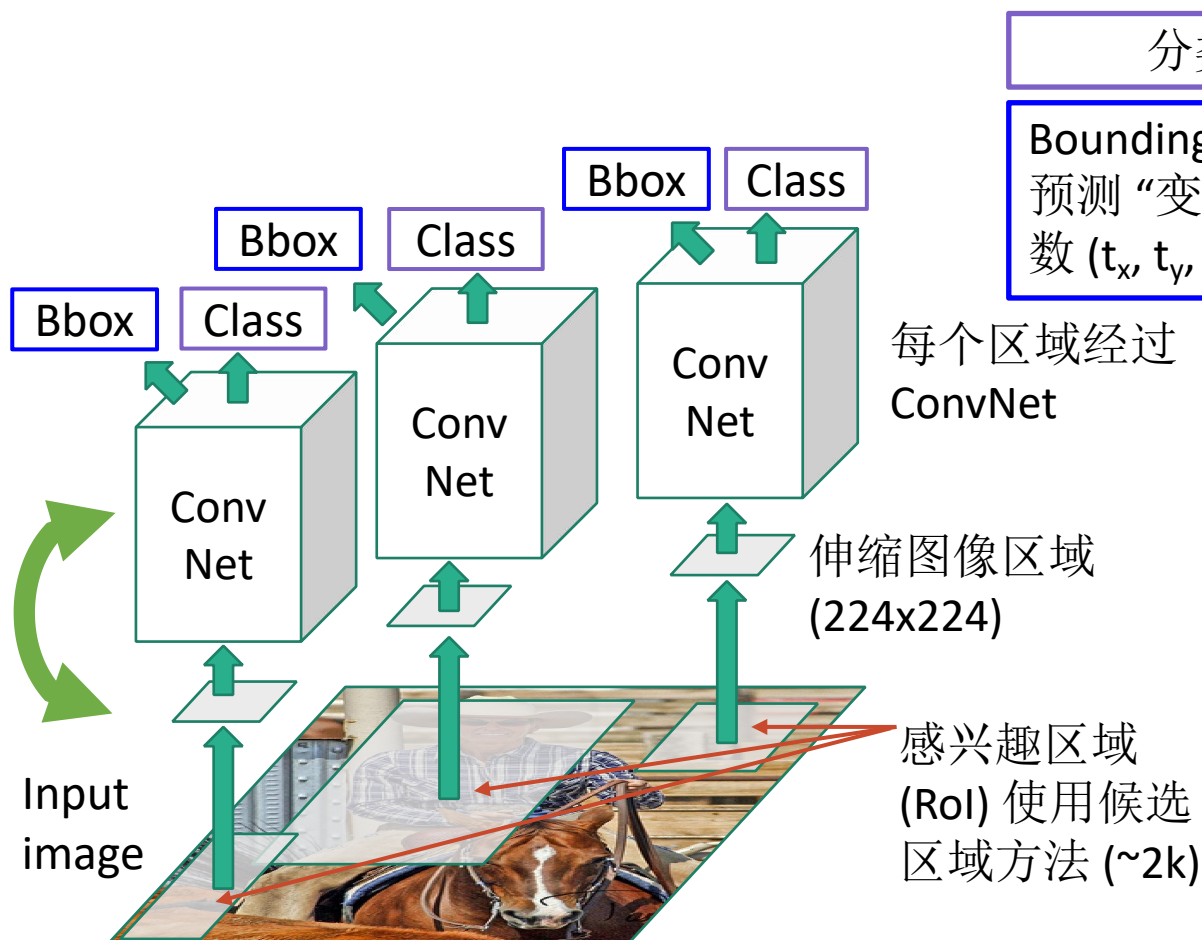
Bounding box 回归:

预测“变换参数”纠正RoI: 4个参数 (t_x, t_y, t_h, t_w)

问题: 非常缓慢! 每张图片需要做~2k次前向卷积计算!

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN: Region-Based CNN



分类每个候选子区域

Bounding box 回归:

预测“变换参数”纠正RoI: 4个参数 (t_x, t_y, t_h, t_w)

问题: 非常缓慢! 每张图片需要做~2k次前向卷积计算!

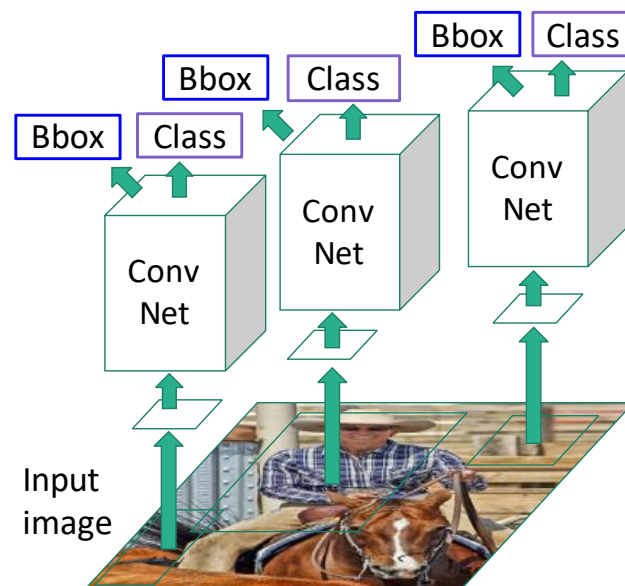
方案: 运行CNN在伸缩之前!

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

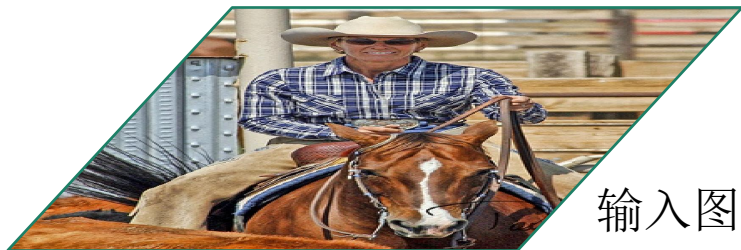
本章内容

- 7.1.目标检测定义
- 7.2.R-CNN
- 7.3.Fast/Faster R-CNN

“Slow” R-CNN
每个候选区域独立
处理

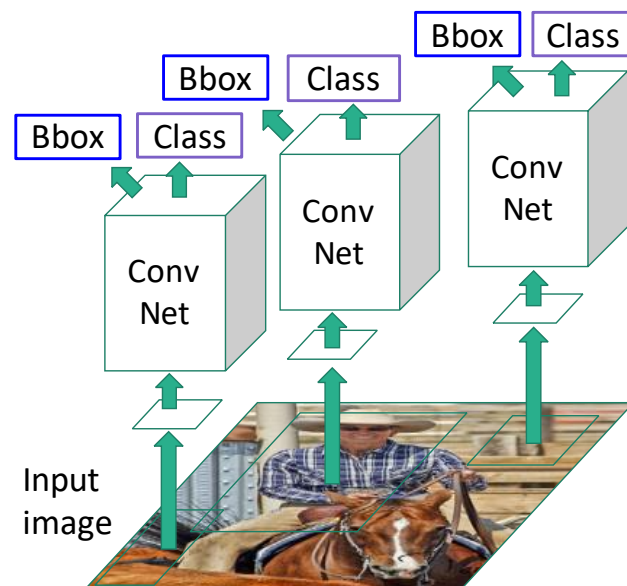


Fast R-CNN



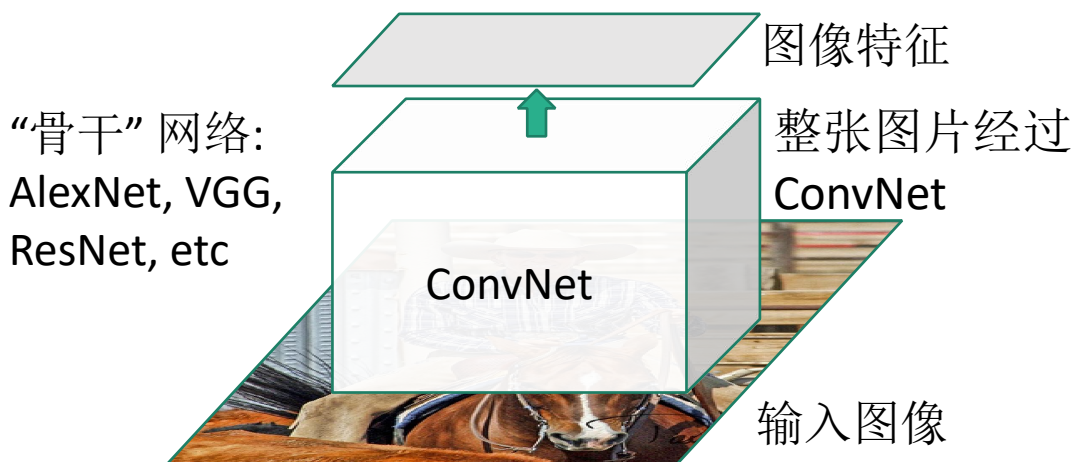
输入图像

“Slow” R-CNN
每个候选区域独立
处理

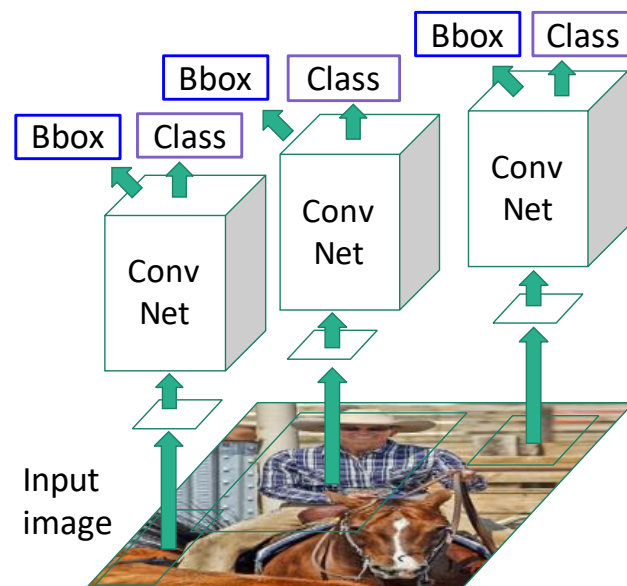


Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Fast R-CNN



“Slow” R-CNN
每个候选区域独立
处理

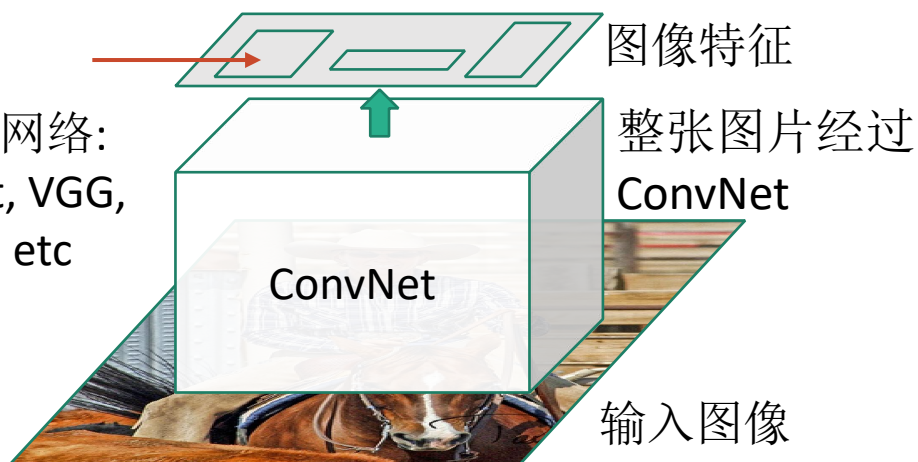


Girshick, “Fast R-CNN”, ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

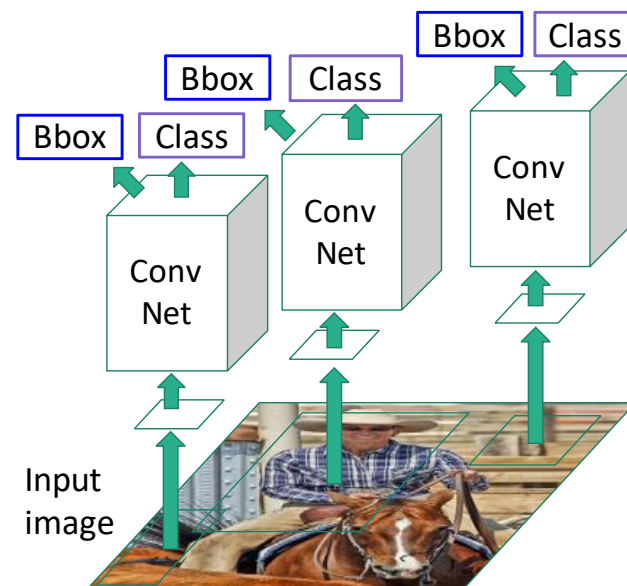
Fast R-CNN

候选区域方法
确定感兴趣区
域(RoIs)

“骨干”网络：
AlexNet, VGG,
ResNet, etc



“Slow” R-CNN
每个候选区域独立
处理

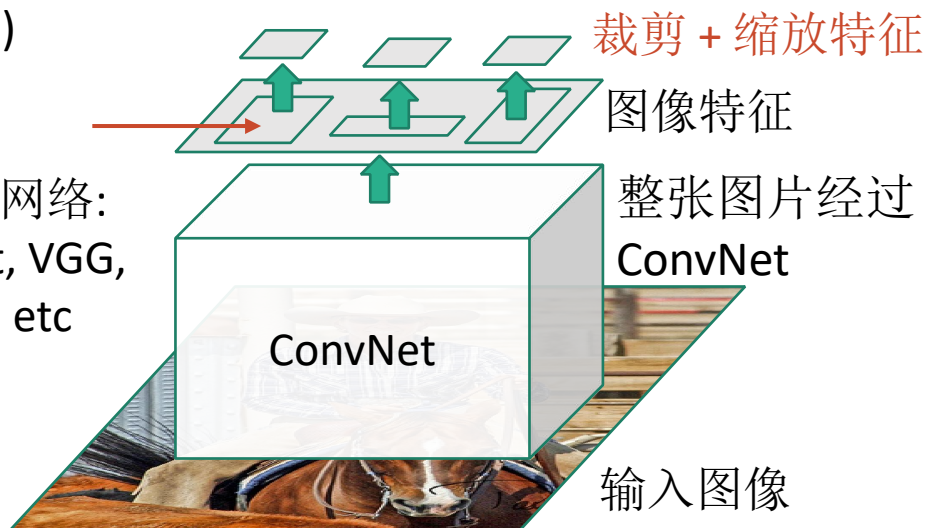


Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

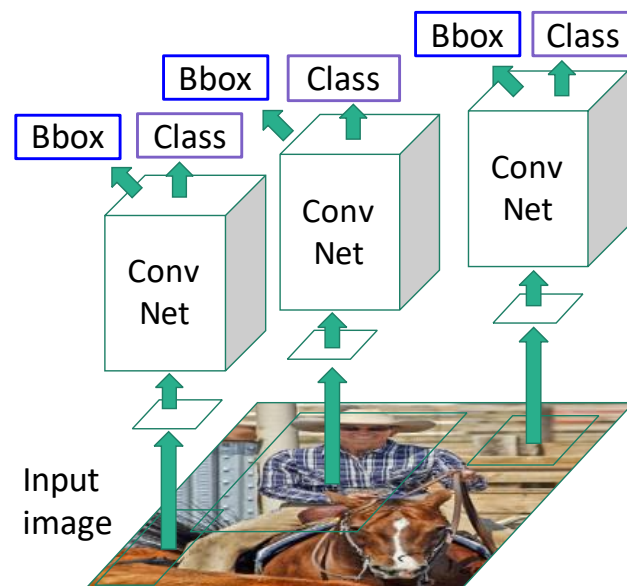
Fast R-CNN

候选区域方法
确定感兴趣区域(RoIs)

“骨干”网络:
AlexNet, VGG,
ResNet, etc



“Slow” R-CNN
每个候选区域独立
处理

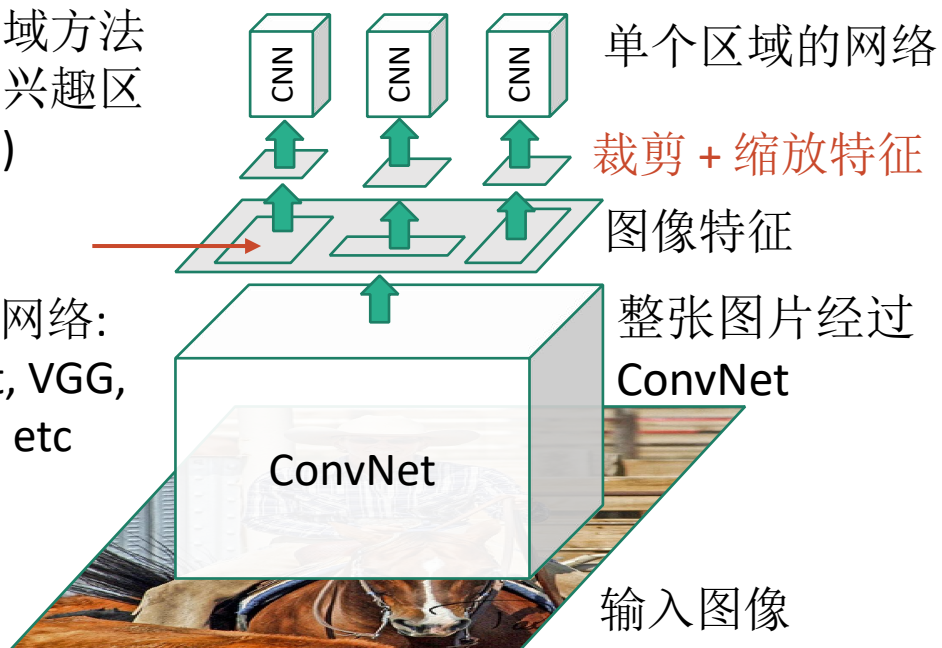


Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

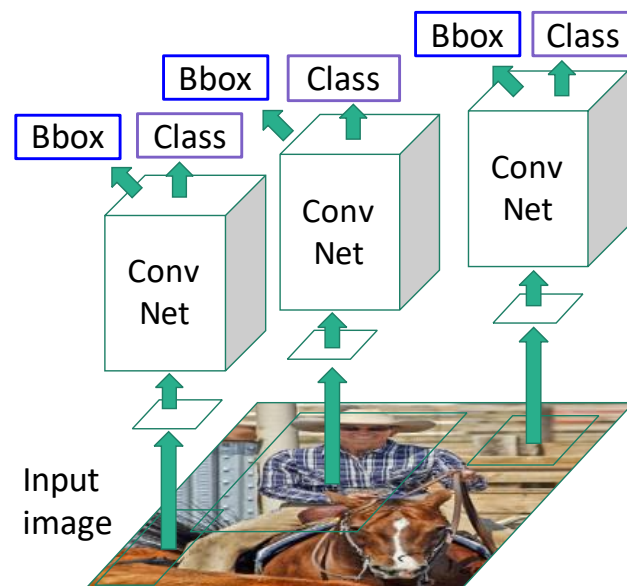
Fast R-CNN

候选区域方法
确定感兴趣区
域(RoIs)

“骨干”网络：
AlexNet, VGG,
ResNet, etc

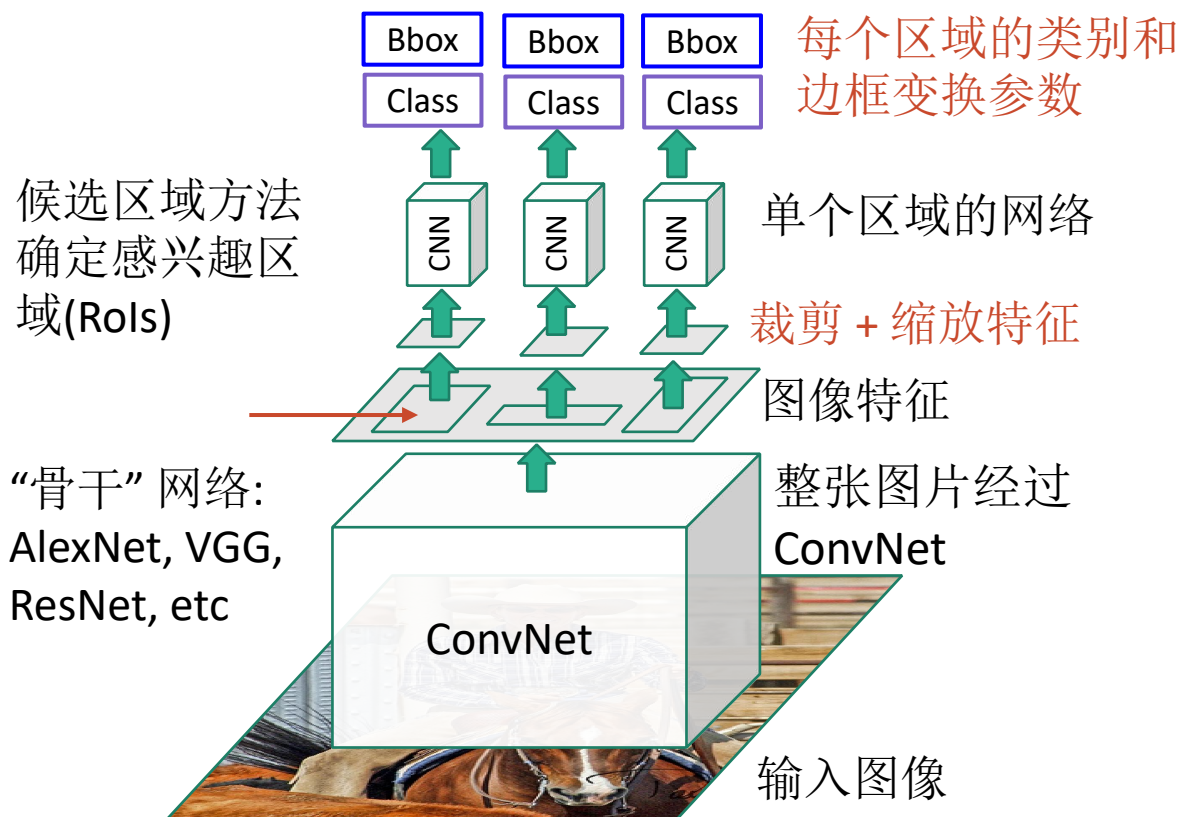


“Slow” R-CNN
每个候选区域独立
处理

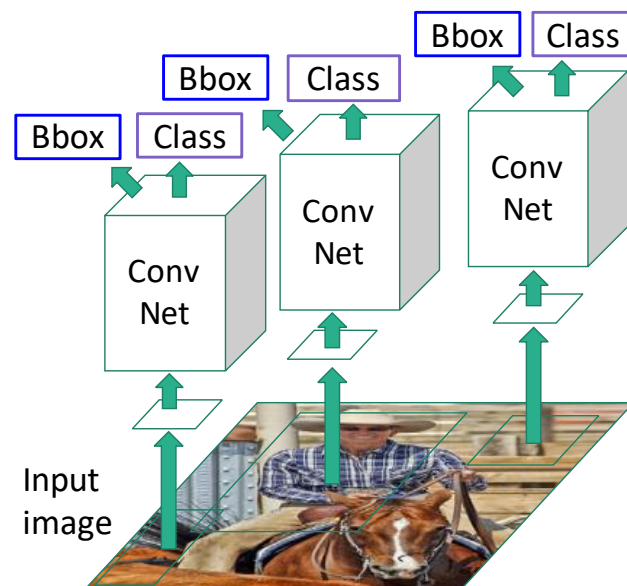


Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Fast R-CNN

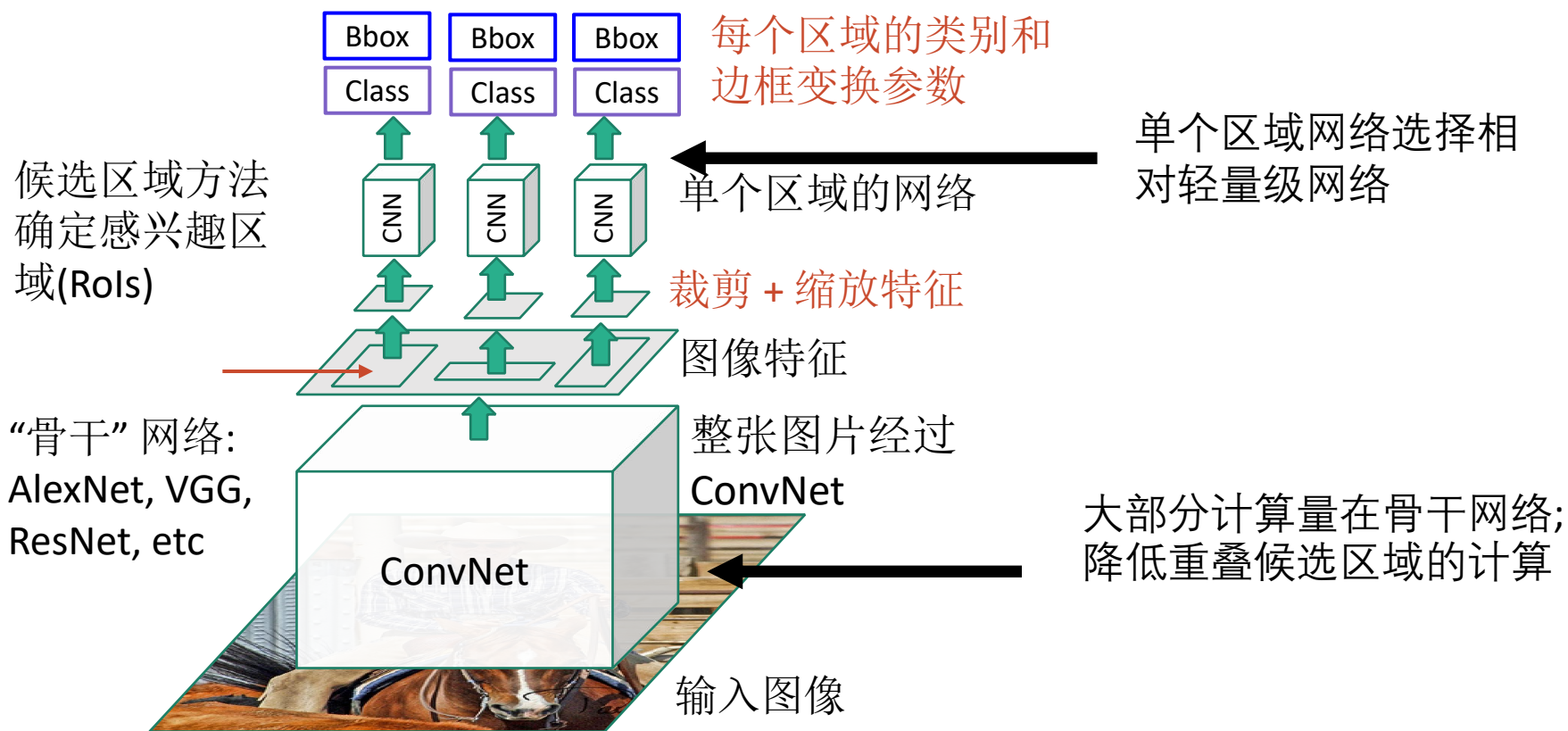


“Slow” R-CNN
每个候选区域独立
处理



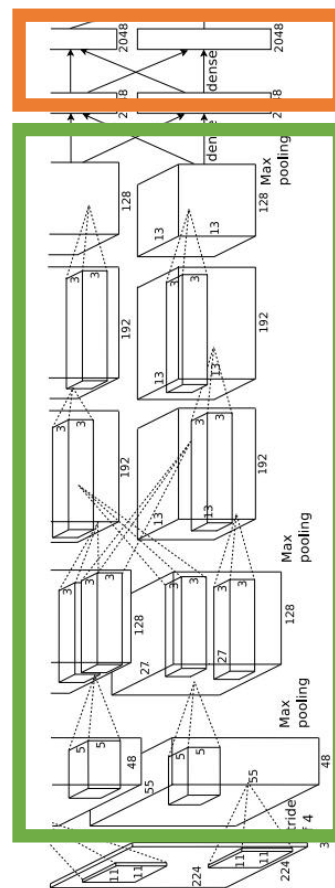
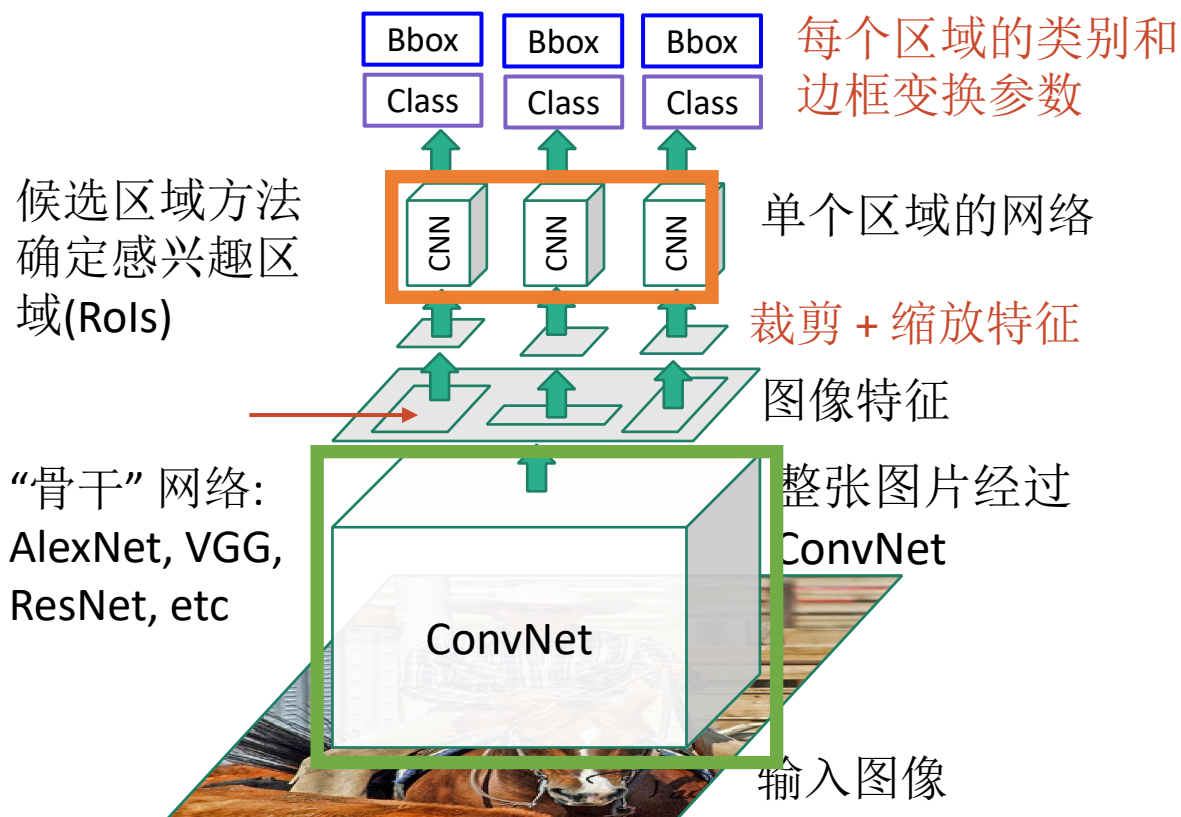
Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Fast R-CNN



Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

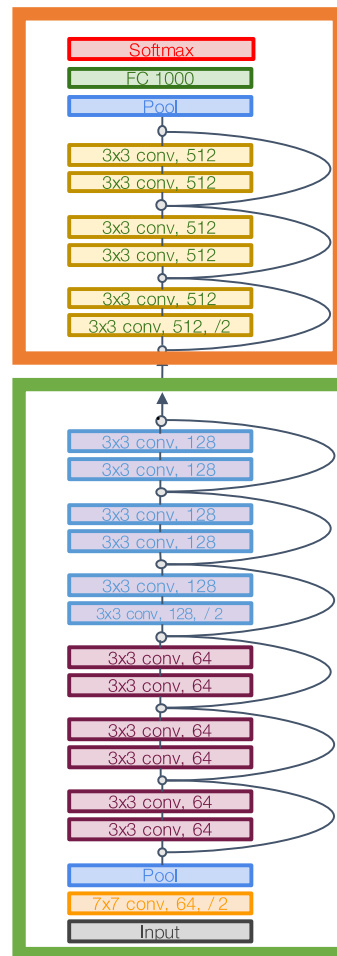
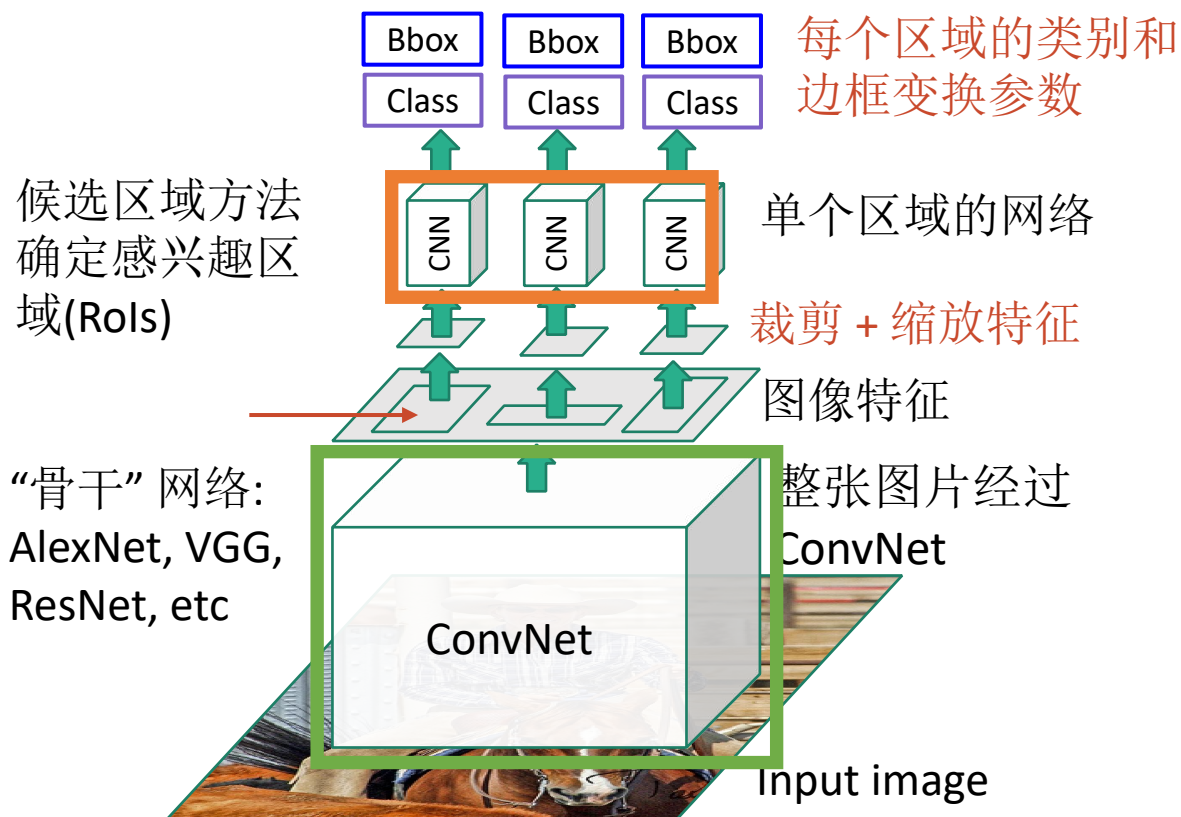
Fast R-CNN



举例:
当使用AlexNet用于检测, 五层conv用于骨干网络而两层FC用于单个区域网络

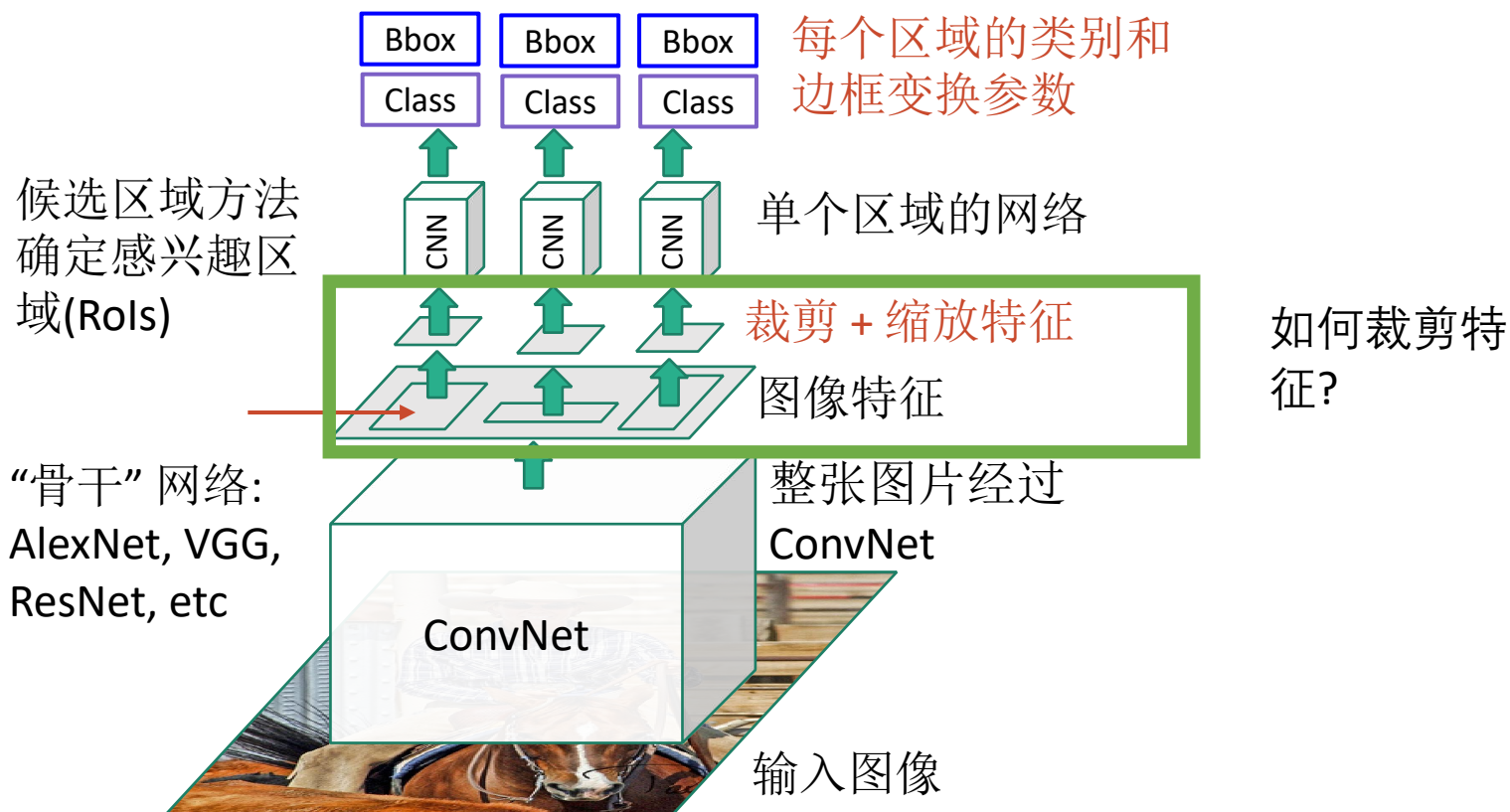
Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Fast R-CNN



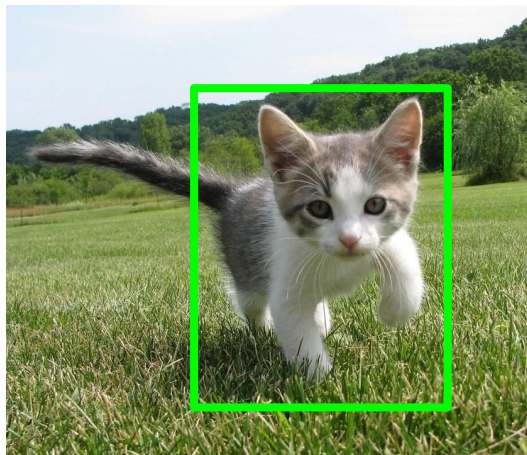
Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Fast R-CNN



Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

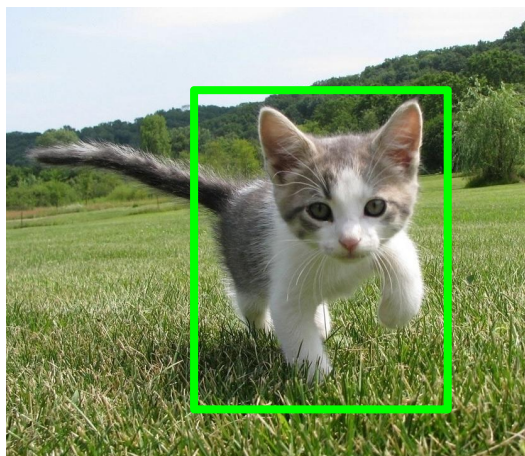
特征裁剪: RoI 池化



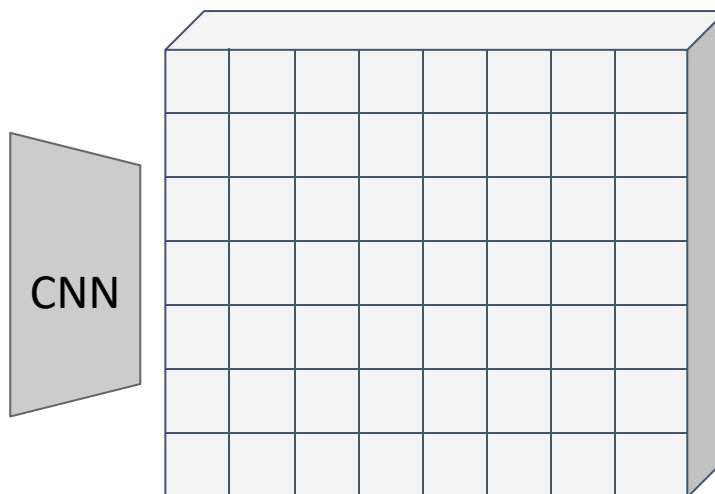
输入图像
(e.g. 3 x 640 x 480)

Girshick, "Fast R-CNN", ICCV 2015.

特征裁剪: RoI 池化



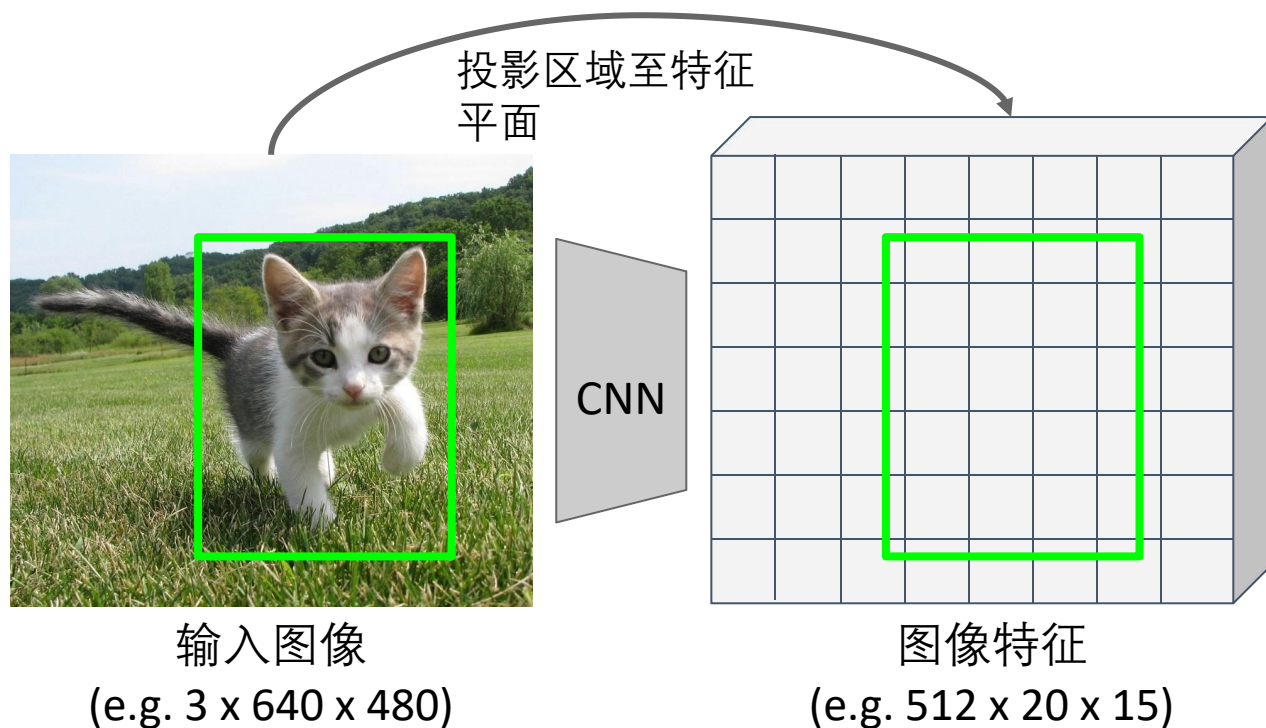
输入图像
(e.g. 3 x 640 x 480)



图像特征
(e.g. 512 x 20 x 15)

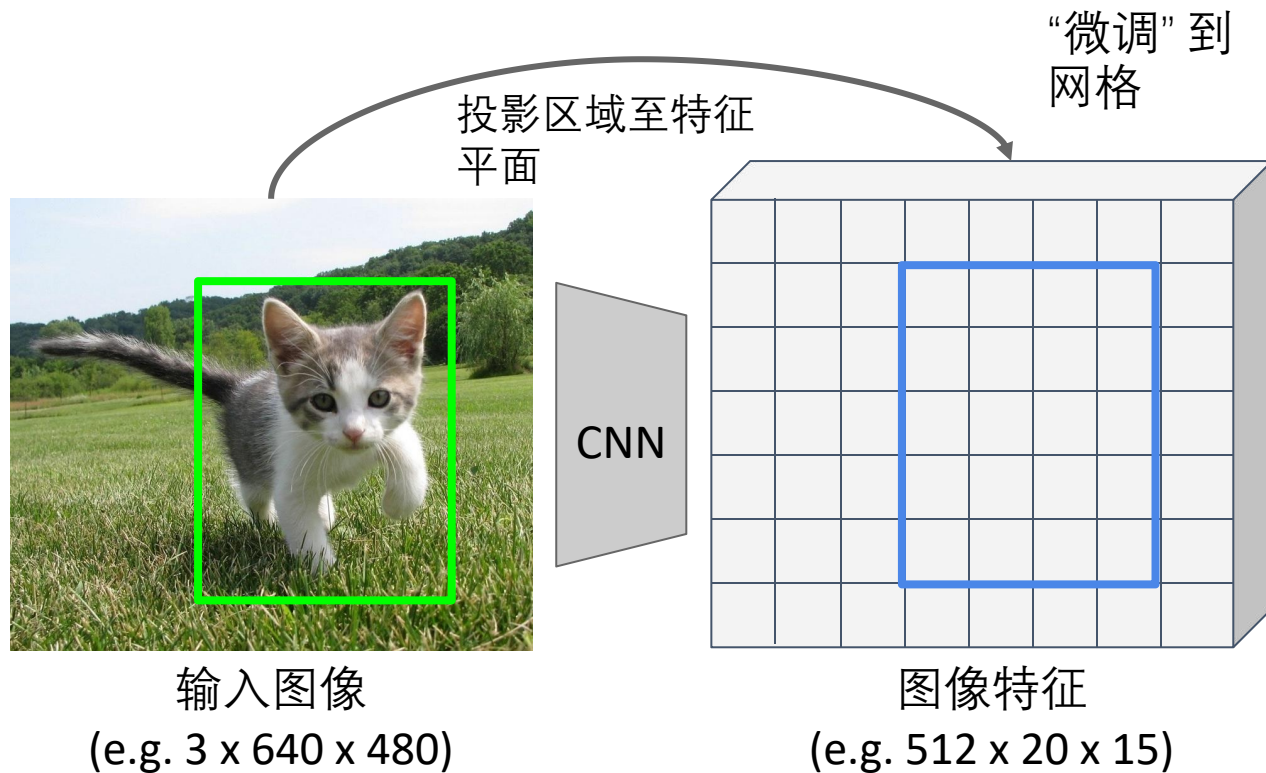
Girshick, "Fast R-CNN", ICCV 2015.

特征裁剪: RoI 池化



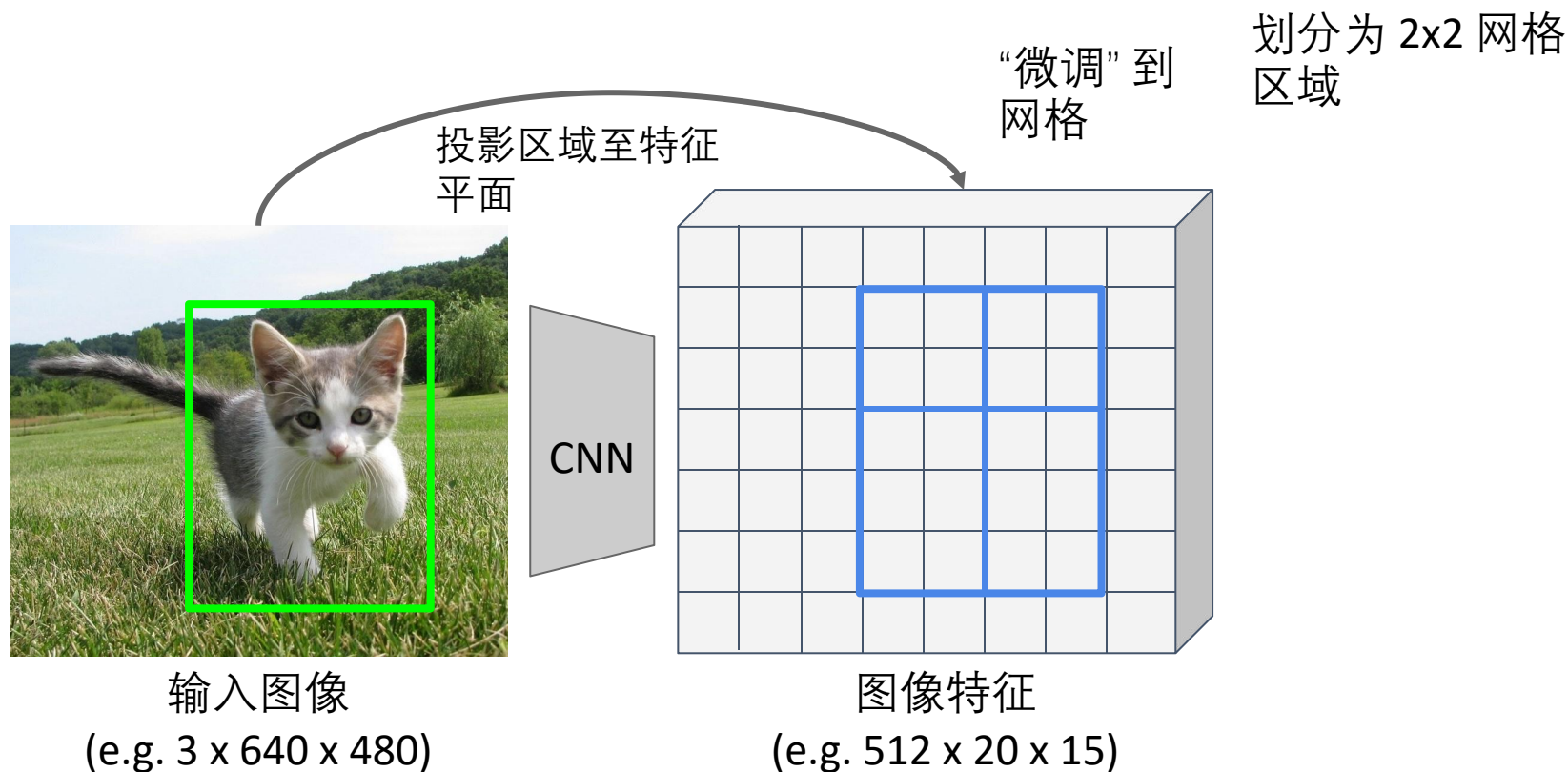
Girshick, "Fast R-CNN", ICCV 2015.

特征裁剪: RoI 池化



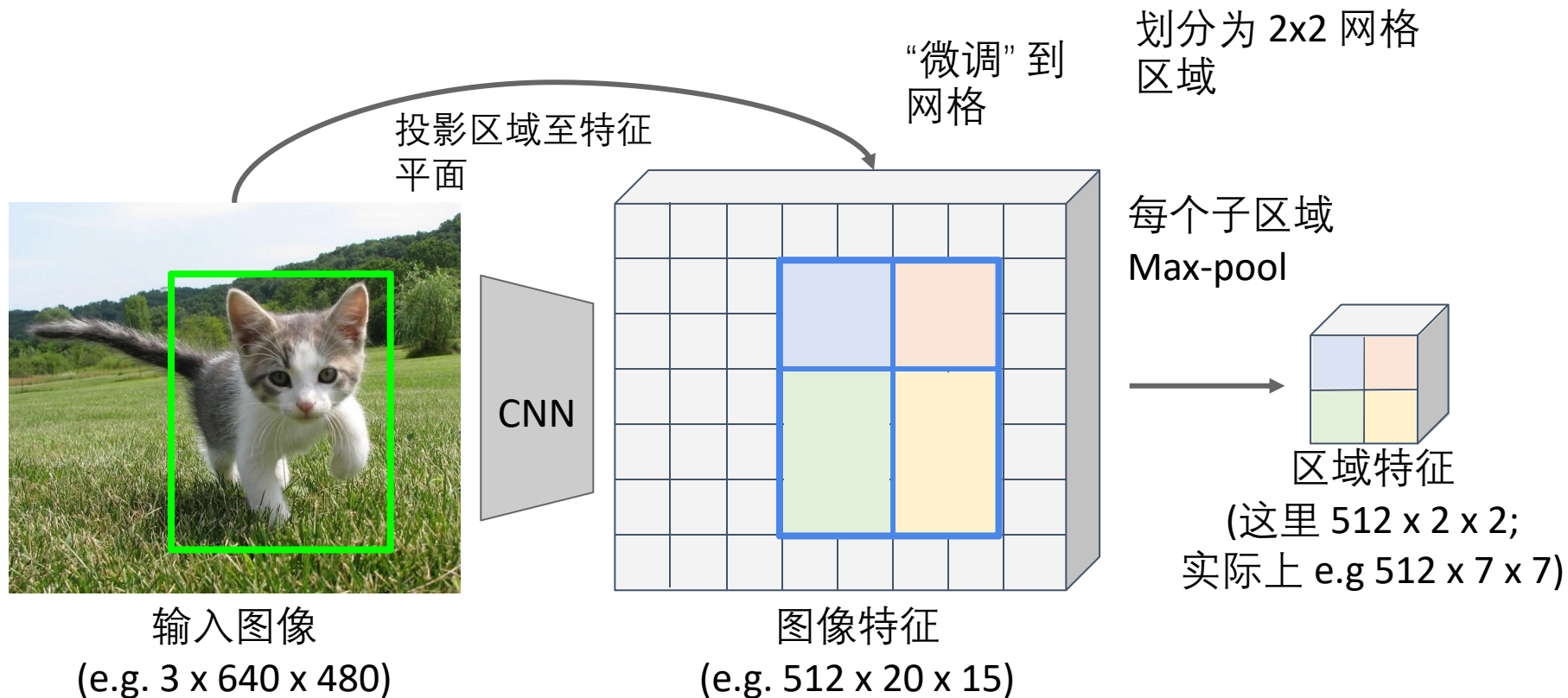
Girshick, “Fast R-CNN”, ICCV 2015.

特征裁剪: RoI 池化



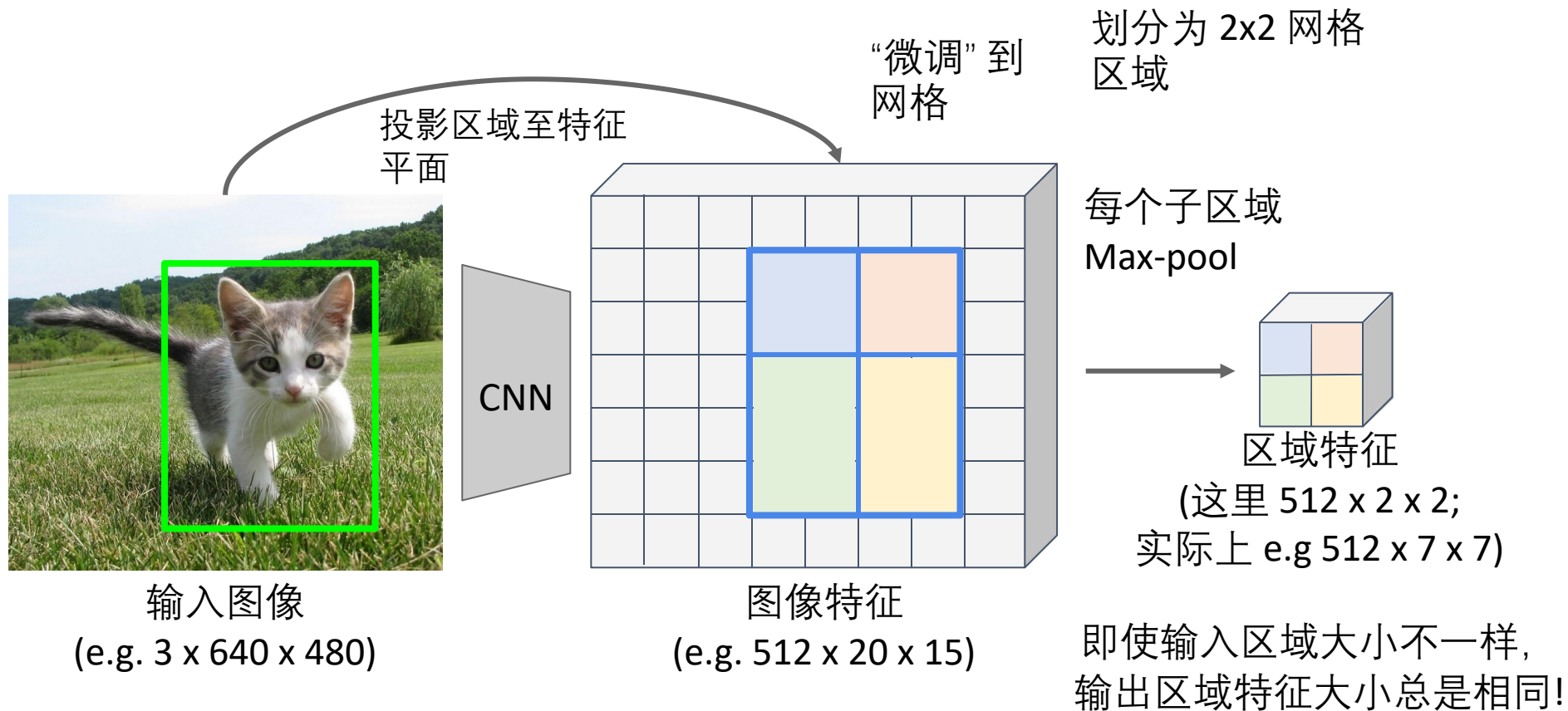
Girshick, "Fast R-CNN", ICCV 2015.

特征裁剪: RoI 池化



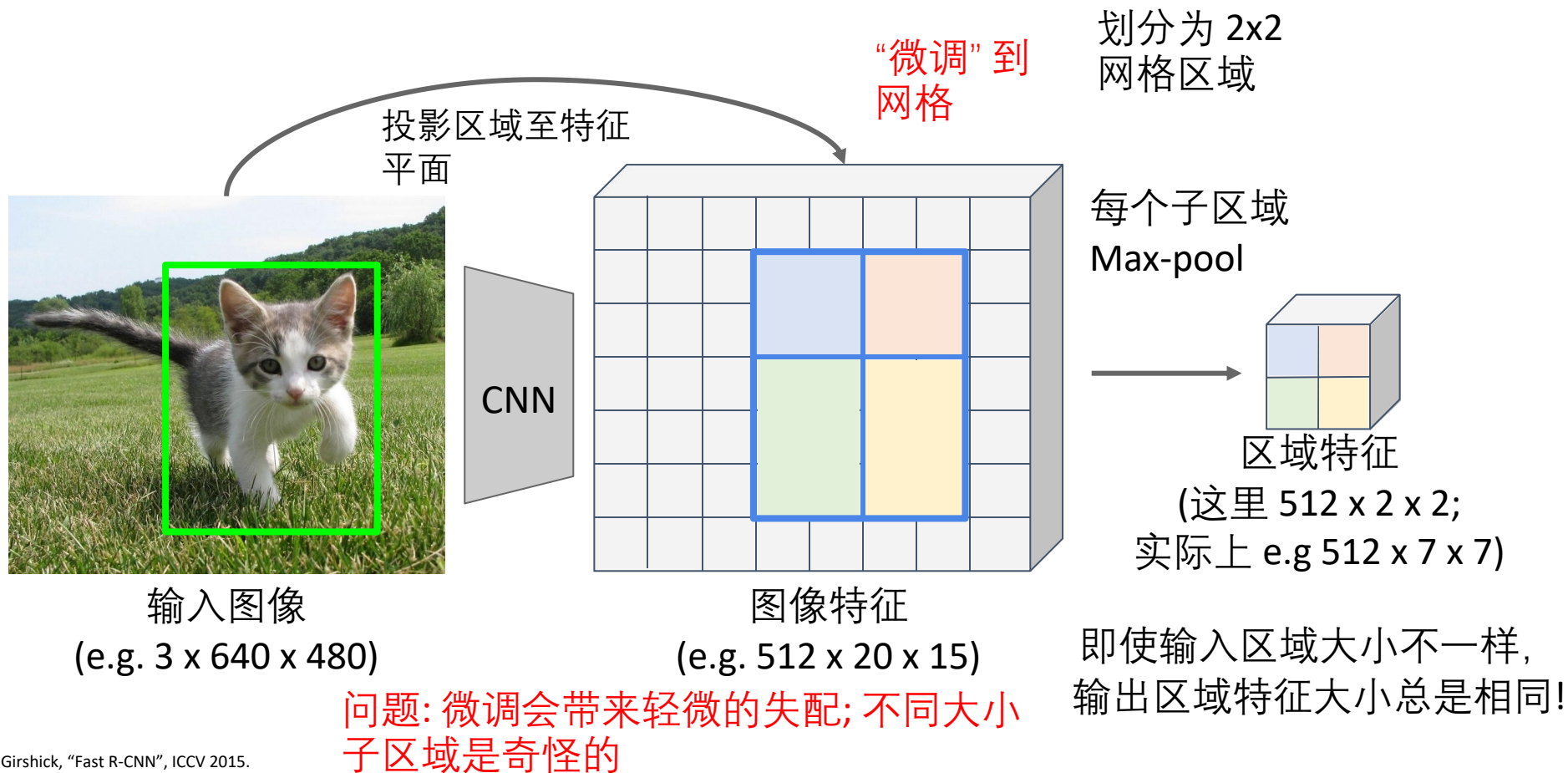
Girshick, "Fast R-CNN", ICCV 2015.

特征裁剪: RoI 池化



Girshick, "Fast R-CNN", ICCV 2015.

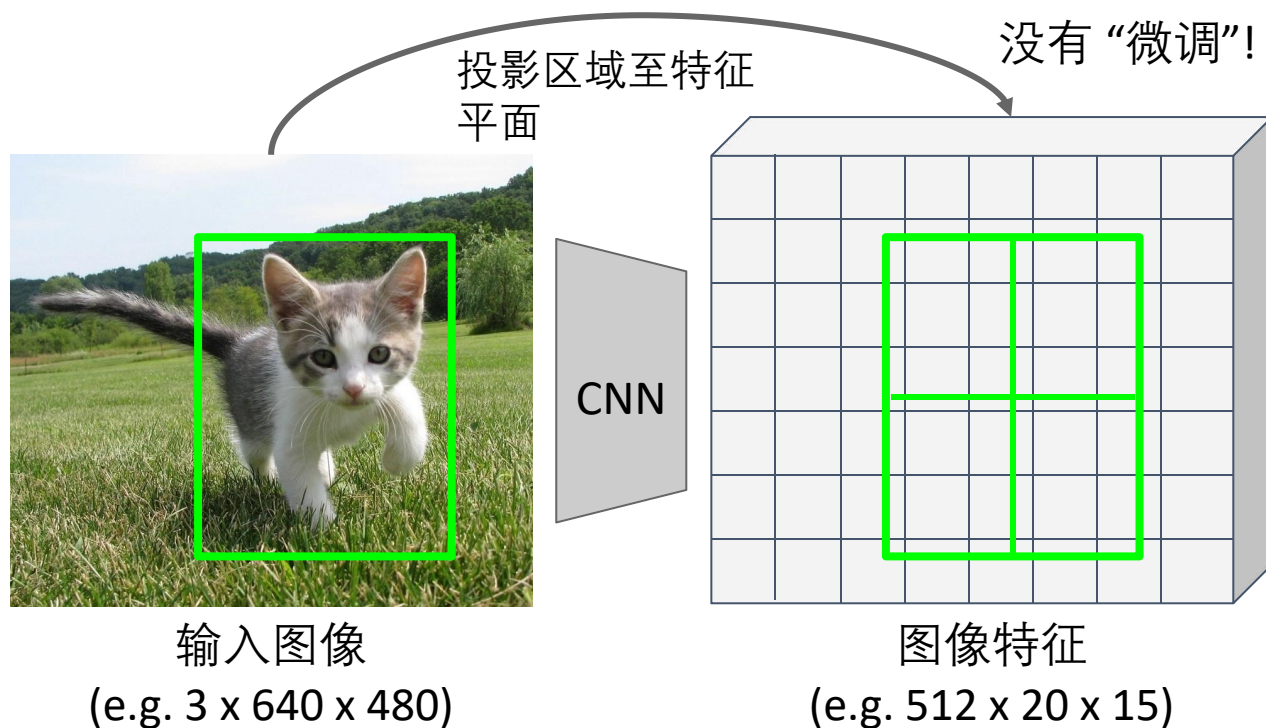
特征裁剪: RoI 池化



Girshick, "Fast R-CNN", ICCV 2015.

特征裁剪: RoI 对齐

划分为 2×2 区域
(不对齐到网格!)



He et al, "Mask R-CNN", ICCV
2017

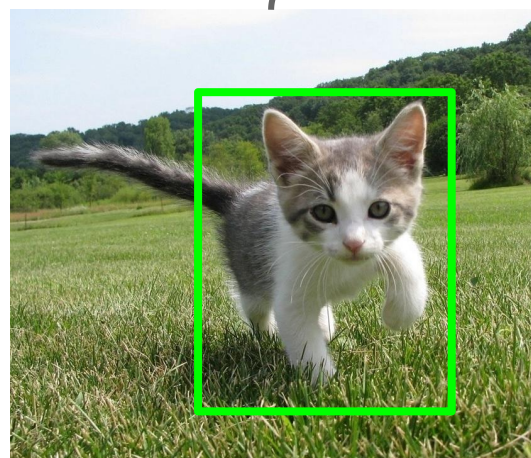
特征裁剪: RoI 对齐

划分为 2×2 区域
(不对齐到网格!)

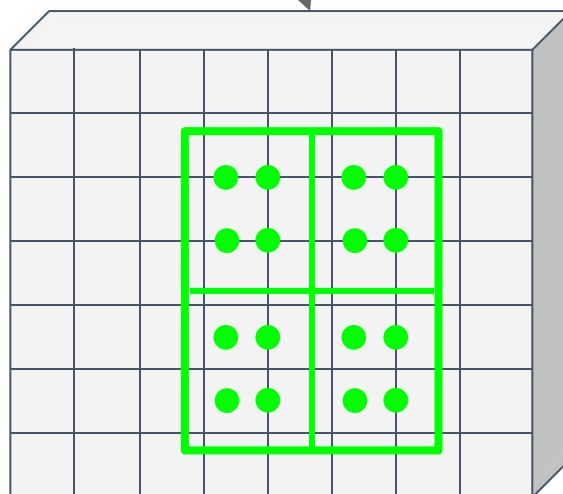
没有“微调”!

在每个子区域使用双
线性插值得到等间隔
采样特征点

投影区域至特征
平面

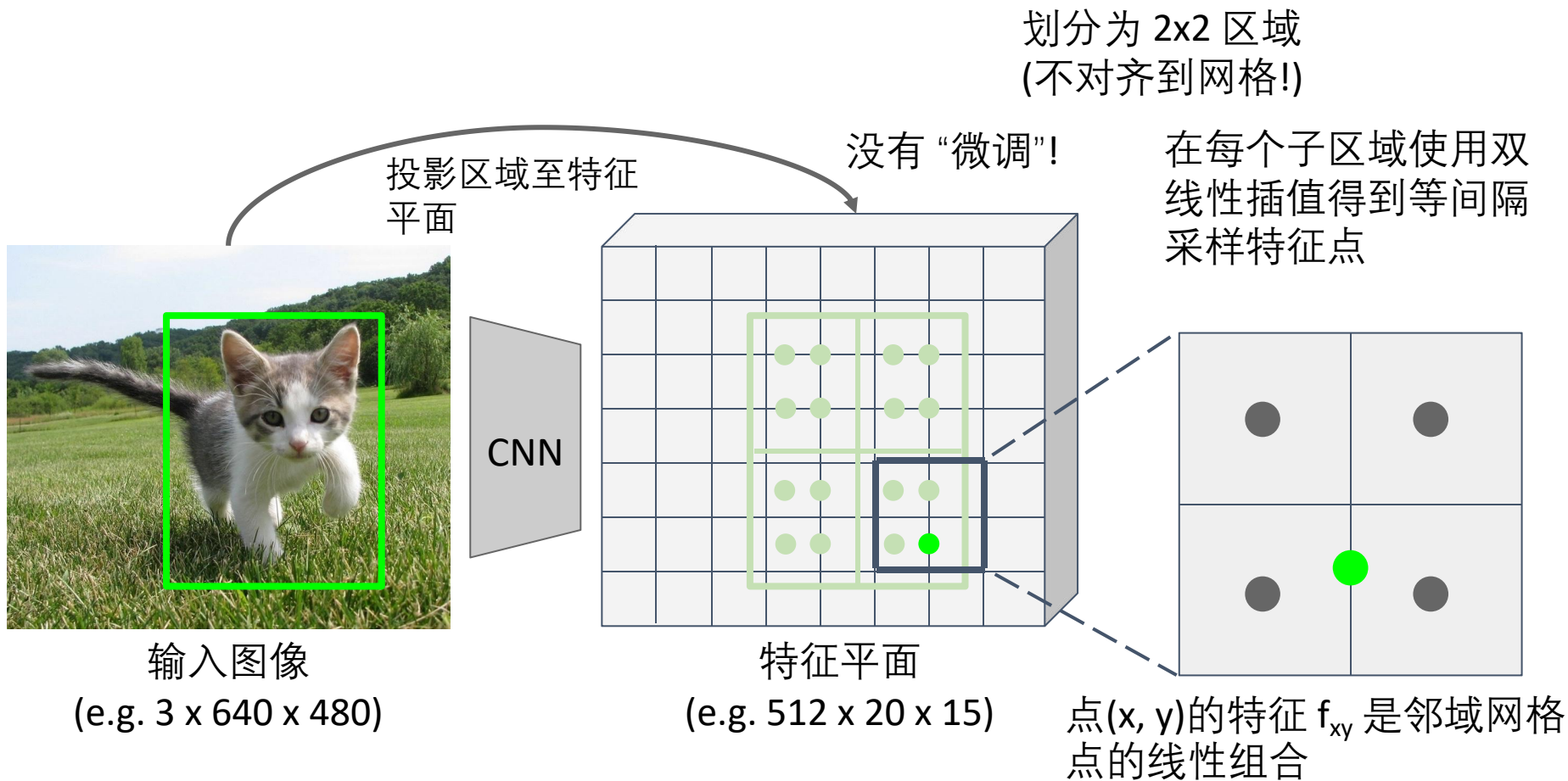


输入图像
(e.g. $3 \times 640 \times 480$)



图像特征
(e.g. $512 \times 20 \times 15$)

特征裁剪: RoI 对齐



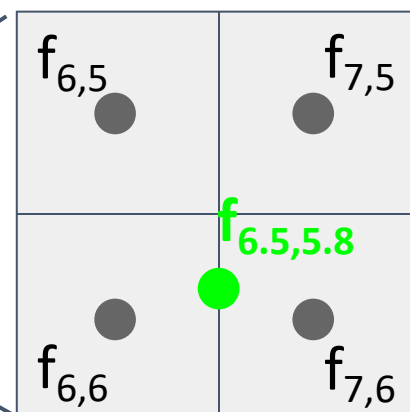
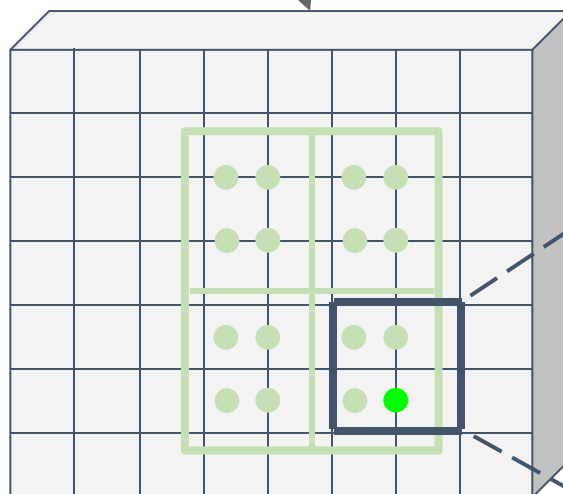
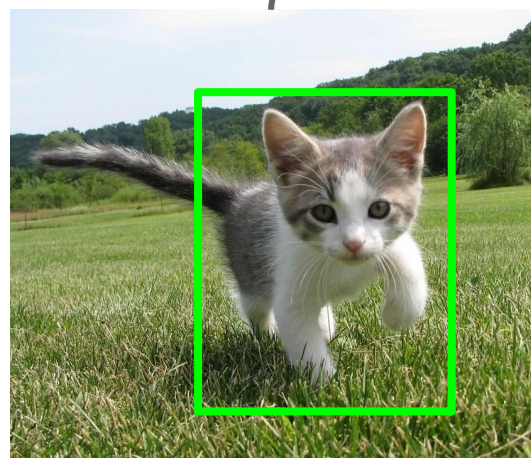
特征裁剪: RoI 对齐

划分为 2x2 区域
(不对齐到网格!)

没有“微调”!

在每个子区域使用双
线性插值得到等间隔
采样特征点

投影区域至特征
平面



$$f_{xy} = \sum_{i,j=1}^2 f_{i,j} \max(0, 1 - |x - x_i|) \max(0, 1 - |y - y_j|)$$

点(x, y)的特征 f_{xy} 是邻域网格
点的线性组合

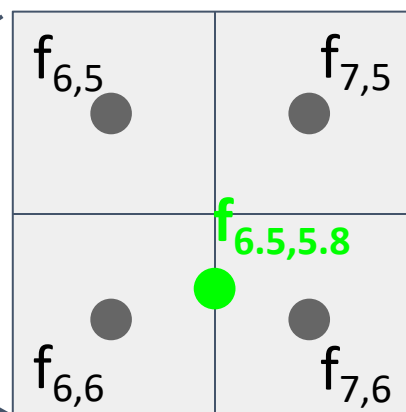
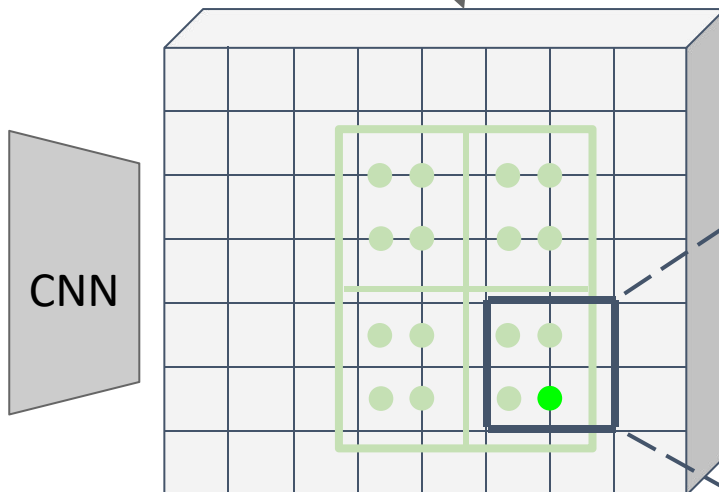
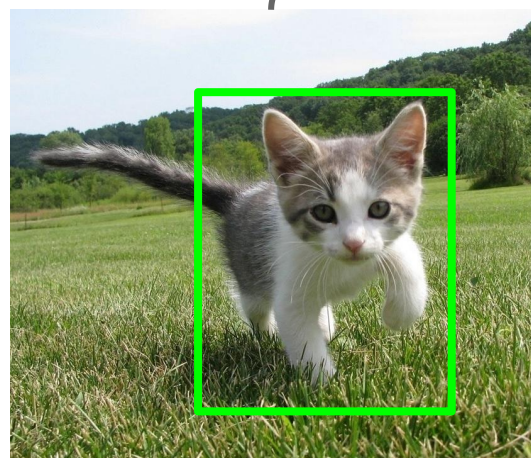
特征裁剪: RoI 对齐

划分为 2x2 区域
(不对齐到网格!)

没有“微调”!

在每个子区域使用双
线性插值得到等间隔
采样特征点

投影区域至特征
平面



点(x, y)的特征 f_{xy} 是邻域网格
点的线性组合

$$f_{xy} = \sum_{i,j=1}^2 f_{i,j} \max(0, 1 - |x - x_i|) \max(0, 1 - |y - y_j|)$$

$$\begin{aligned} f_{6.5,5.8} &= (f_{6,5} * 0.5 * 0.2) + (f_{7,5} * 0.5 * 0.2) \\ &\quad + (f_{6,6} * 0.5 * 0.8) + (f_{7,6} * 0.5 * 0.8) \end{aligned}$$

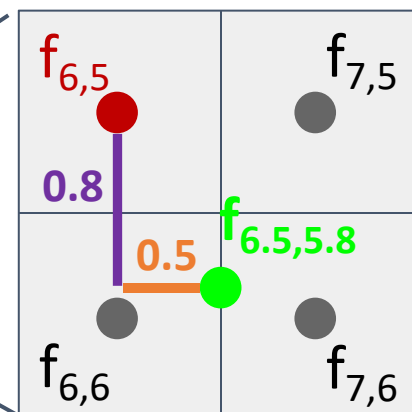
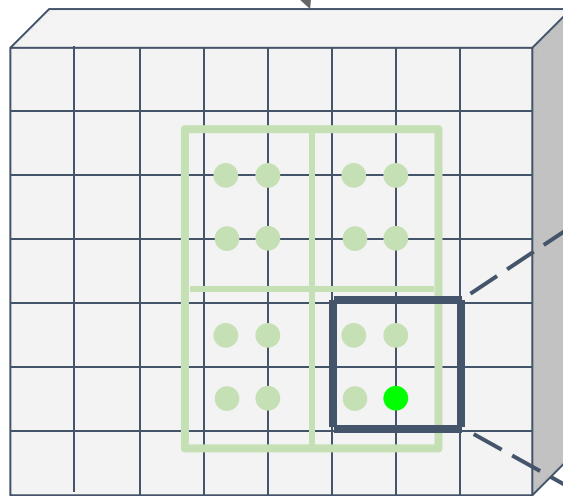
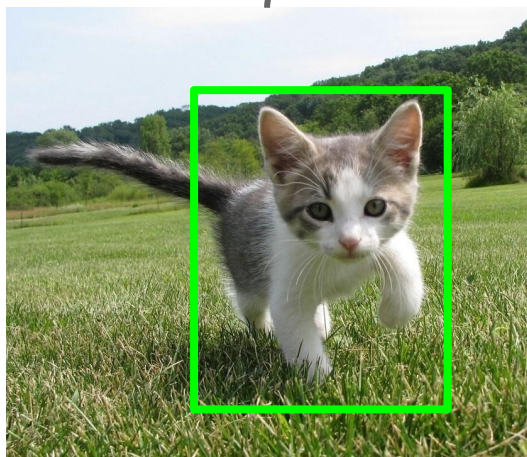
特征裁剪: RoI 对齐

划分为 2x2 区域
(不对齐到网格!)

没有“微调”!

在每个子区域使用双
线性插值得到等间隔
采样特征点

投影区域至特征
平面



点(x, y)的特征 f_{xy} 是邻域网格
点的线性组合

$$f_{xy} = \sum_{i,j=1}^2 \boxed{f_{i,j}} \boxed{\max(0, 1 - |x - x_i|)} \boxed{\max(0, 1 - |y - y_j|)}$$

$$\begin{aligned} \mathbf{f}_{6.5,5.8} &= (\mathbf{f}_{6,5} * \mathbf{0.5} * \mathbf{0.2}) + (\mathbf{f}_{7,5} * 0.5 * 0.2) \\ &\quad + (\mathbf{f}_{6,6} * 0.5 * 0.8) + (\mathbf{f}_{7,6} * 0.5 * 0.8) \end{aligned}$$

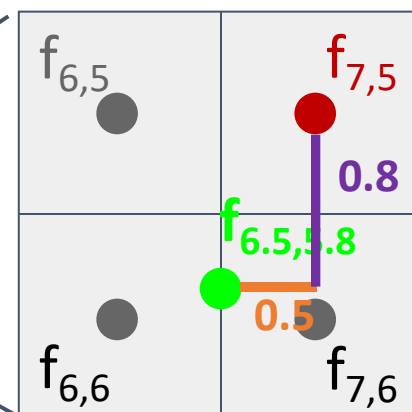
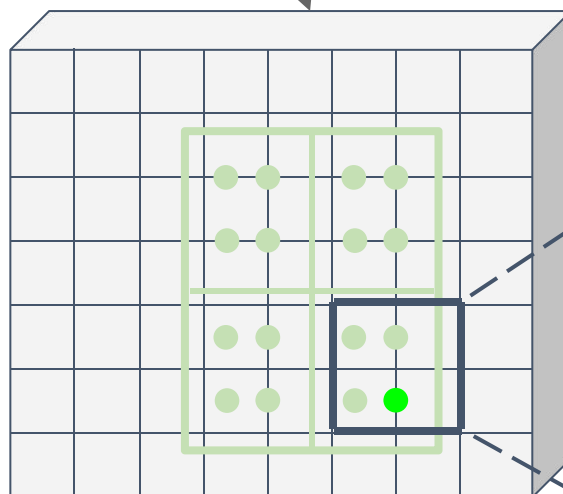
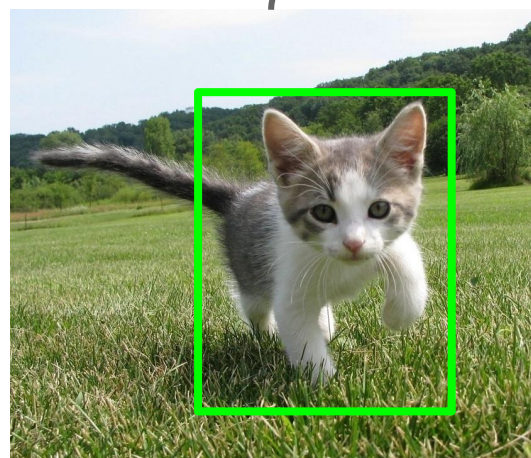
特征裁剪: RoI 对齐

划分为 2x2 区域
(不对齐到网格!)

没有“微调”!

在每个子区域使用双
线性插值得到等间隔
采样特征点

投影区域至特征
平面



$$f_{xy} = \sum_{i,j=1}^2 \boxed{f_{i,j}} \boxed{\max(0, 1 - |x - x_i|)} \boxed{\max(0, 1 - |y - y_j|)}$$

$$\begin{aligned} \mathbf{f}_{6.5,5.8} &= (\mathbf{f}_{6,5} * 0.5 * 0.2) + (\mathbf{f}_{7,5} * \mathbf{0.5} * \mathbf{0.2}) \\ &\quad + (\mathbf{f}_{6,6} * 0.5 * 0.8) + (\mathbf{f}_{7,6} * 0.5 * 0.8) \end{aligned}$$

点(x, y)的特征 f_{xy} 是邻域网格
点的线性组合

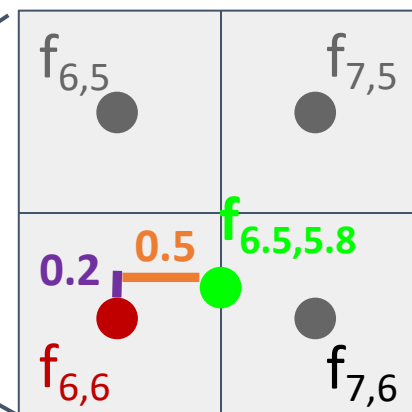
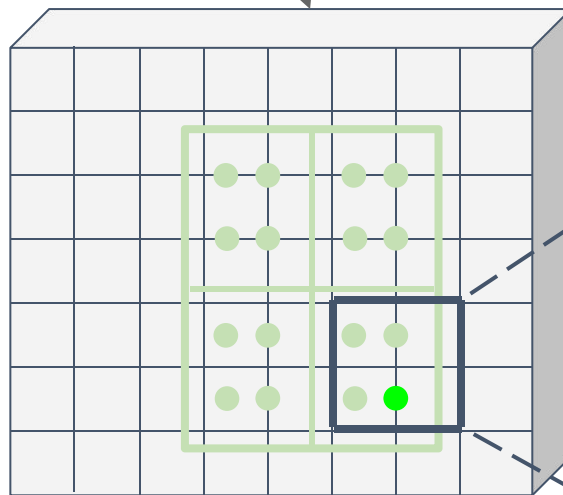
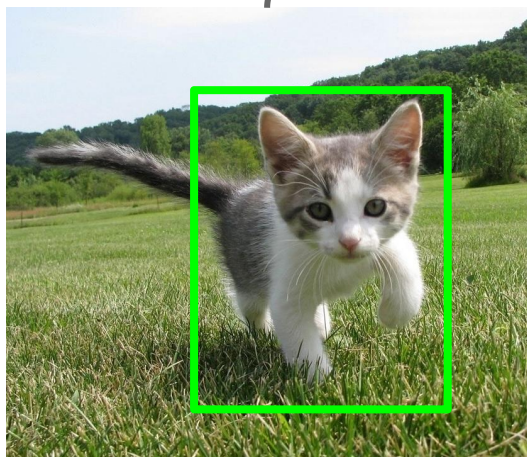
特征裁剪: RoI 对齐

划分为 2x2 区域
(不对齐到网格!)

没有“微调”!

在每个子区域使用双
线性插值得到等间隔
采样特征点

投影区域至特征
平面



点(x, y)的特征 f_{xy} 是邻域网格
点的线性组合

$$f_{xy} = \sum_{i,j=1}^2 \boxed{f_{i,j}} \boxed{\max(0, 1 - |x - x_i|)} \boxed{\max(0, 1 - |y - y_j|)}$$

$$\begin{aligned} f_{6.5,5.8} &= (f_{6,5} * 0.5 * 0.2) + (f_{7,5} * 0.5 * 0.2) \\ &\quad + (f_{6,6} * 0.5 * 0.8) + (f_{7,6} * 0.5 * 0.8) \end{aligned}$$

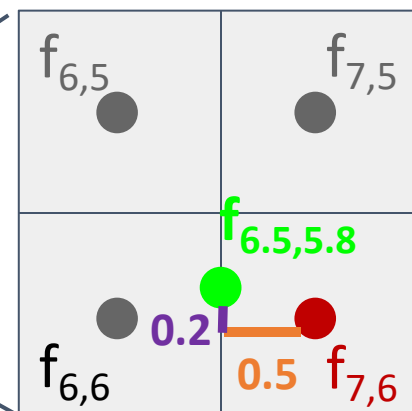
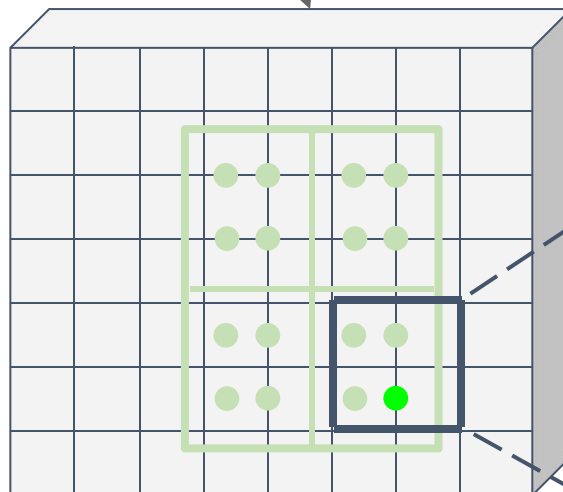
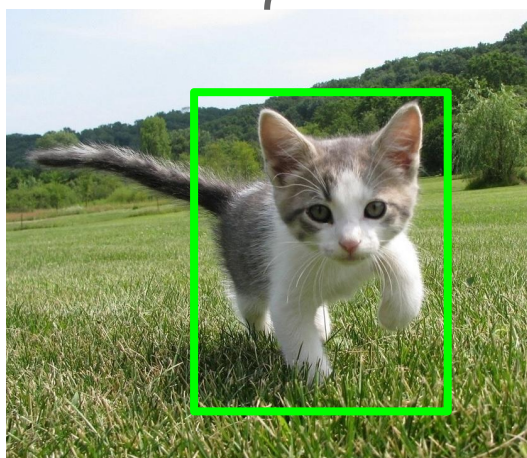
特征裁剪: RoI 对齐

划分为 2x2 区域
(不对齐到网格!)

没有“微调”!

在每个子区域使用双
线性插值得到等间隔
采样特征点

投影区域至特征
平面



点(x, y)的特征 f_{xy} 是邻域网格
点的线性组合

$$f_{xy} = \sum_{i,j=1}^2 \boxed{f_{i,j}} \boxed{\max(0, 1 - |x - x_i|)} \boxed{\max(0, 1 - |y - y_j|)}$$

$$\begin{aligned} f_{6.5,5.8} &= (f_{6,5} * 0.5 * 0.2) + (f_{7,5} * 0.5 * 0.2) \\ &\quad + (f_{6,6} * 0.5 * 0.8) + (f_{7,6} * 0.5 * 0.8) \end{aligned}$$

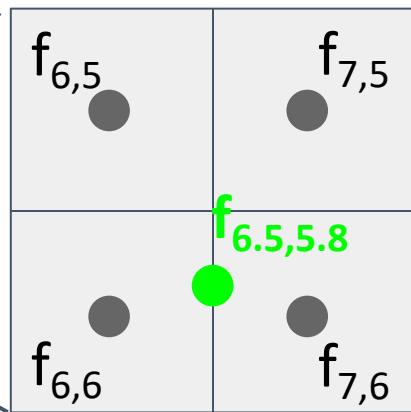
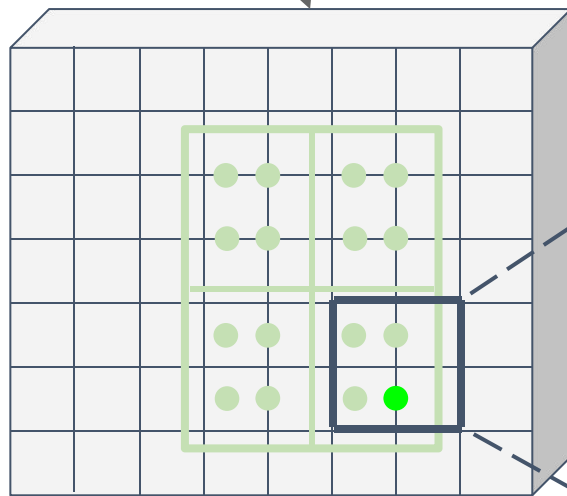
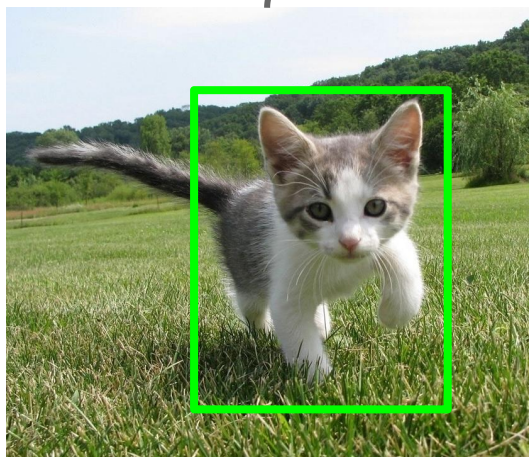
特征裁剪: RoI 对齐

划分为 2x2 区域
(不对齐到网格!)

没有“微调”!

投影区域至特征
平面

在每个子区域使用双
线性插值得到等间隔
采样特征点



$$f_{xy} = \sum_{i,j=1}^2 f_{i,j} \max(0, 1 - |x - x_i|) \max(0, 1 - |y - y_j|)$$

这个计算是可微分的! 采样特征的上游梯度将反向流动到4个最近邻的网格点之上

点(x, y)的特征 f_{xy} 是邻域网格点的线性组合

特征裁剪: RoI 对齐

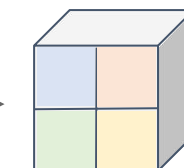
划分为 2×2 区域
(不对齐到网格!)

没有“微调”!

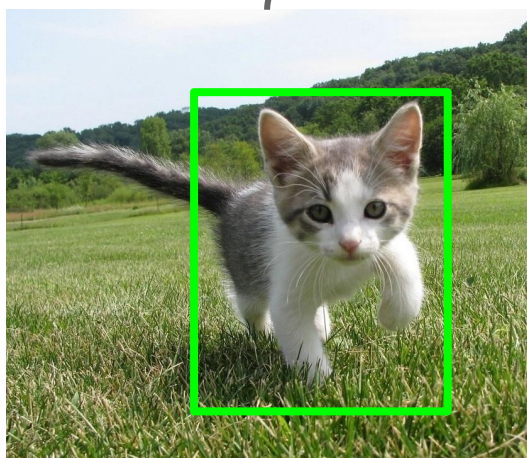
投影区域至特征
平面

在每个子区域使用双
线性插值得到等间隔
采样特征点

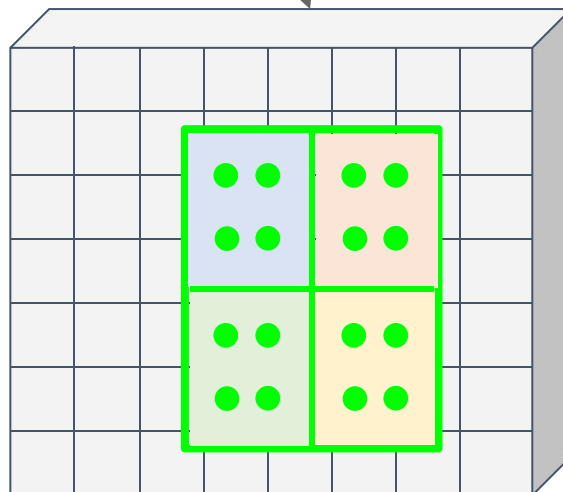
采样之后, 每个子区
域max-pool



区域特征
(这里 $512 \times 2 \times 2$;
实际上e.g $512 \times 7 \times 7$)



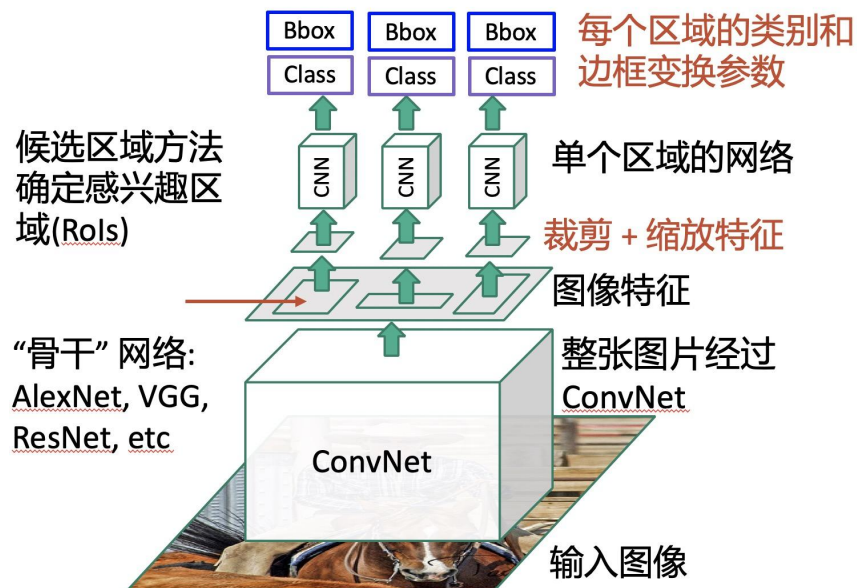
输入图像
(e.g. $3 \times 640 \times 480$)



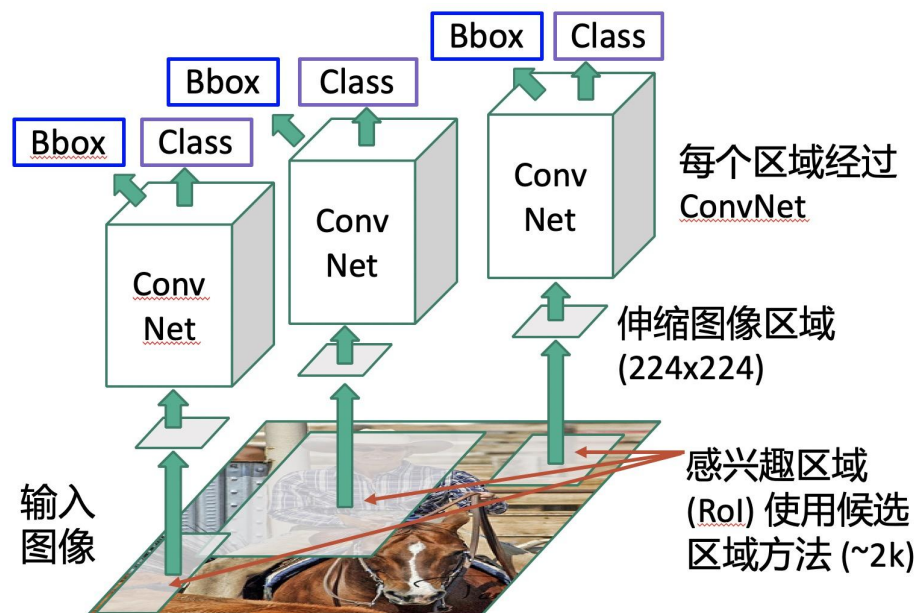
特征平面
(e.g. $512 \times 20 \times 15$)

Fast R-CNN vs “Slow” R-CNN

Fast R-CNN: 使用可微的裁剪处理全局图像特征

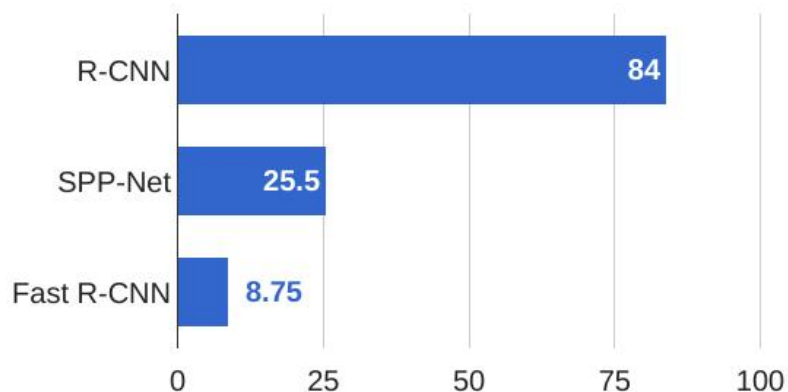


“Slow” R-CNN: 每个候选区域独立 CNN 处理

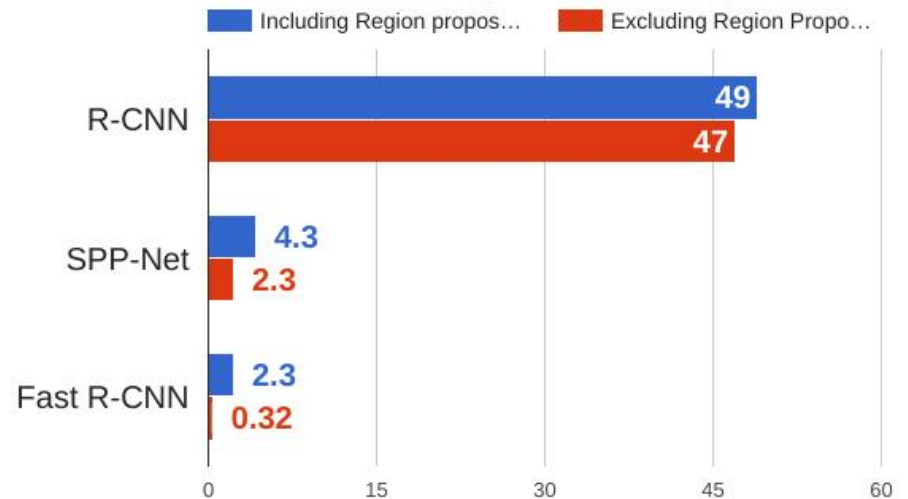


Fast R-CNN vs “Slow” R-CNN

Training time (Hours)



Test time (seconds)



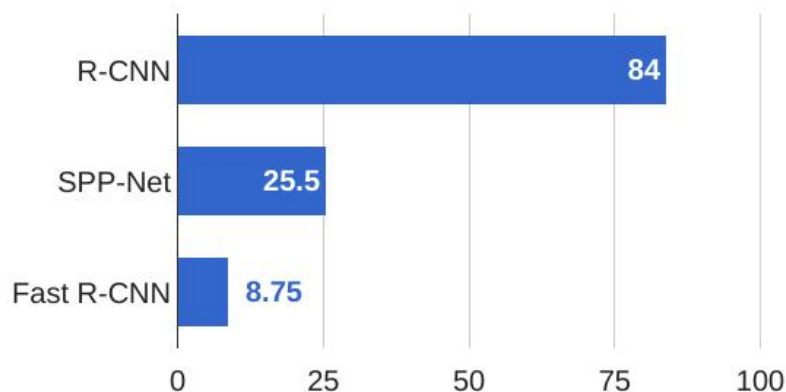
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.

He et al, "Spatial pyramid pooling in deep convolutional networks for visual recognition", ECCV 2014

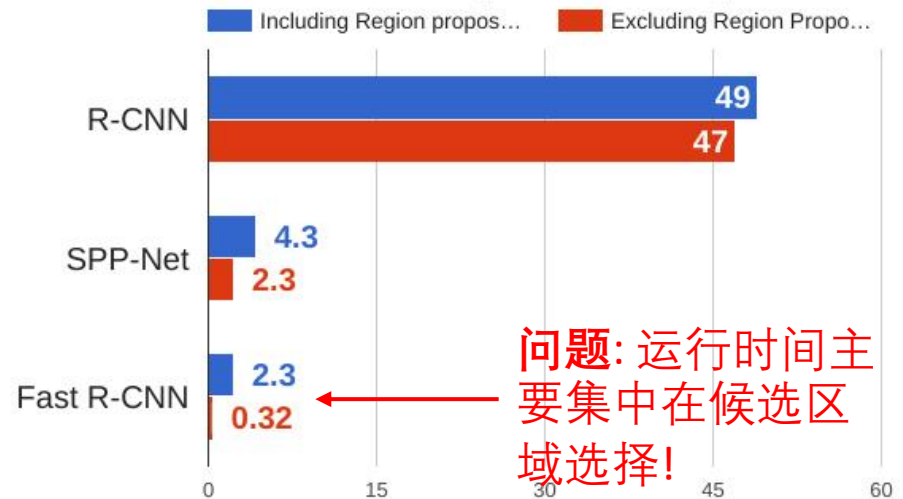
Girshick, "Fast R-CNN", ICCV 2015

Fast R-CNN vs “Slow” R-CNN

Training time (Hours)



Test time (seconds)

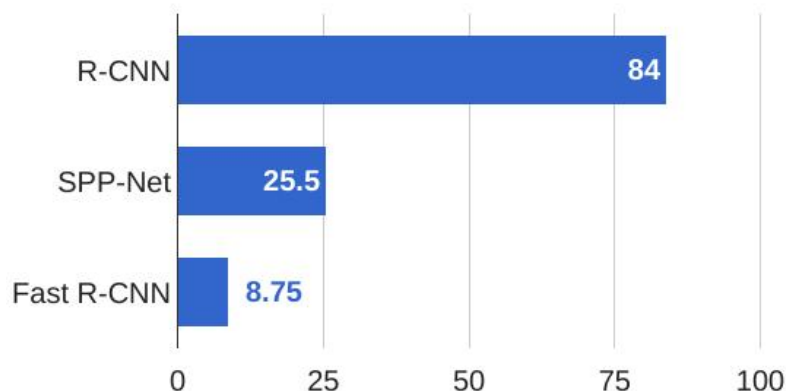


问题: 运行时间主要集中在候选区域选择!

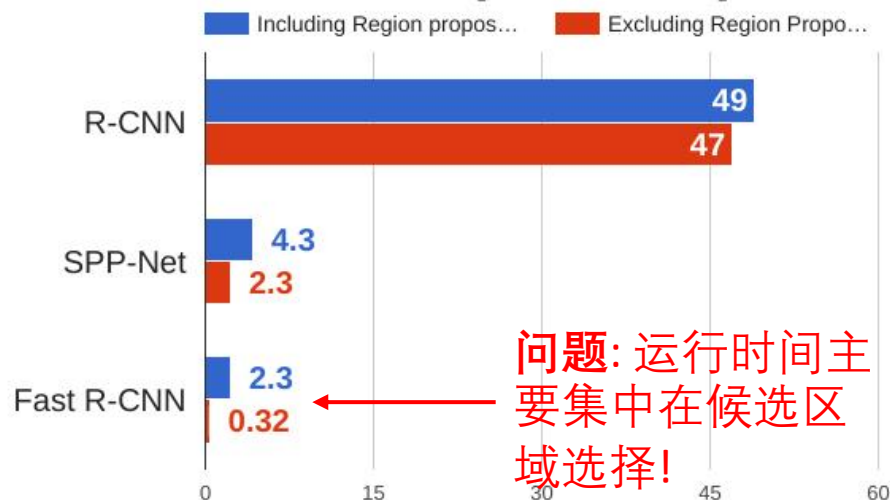
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
He et al, "Spatial pyramid pooling in deep convolutional networks for visual recognition", ECCV 2014
Girshick, "Fast R-CNN", ICCV 2015

Fast R-CNN vs “Slow” R-CNN

Training time (Hours)



Test time (seconds)



问题: 运行时间主要集中在候选区域选择!

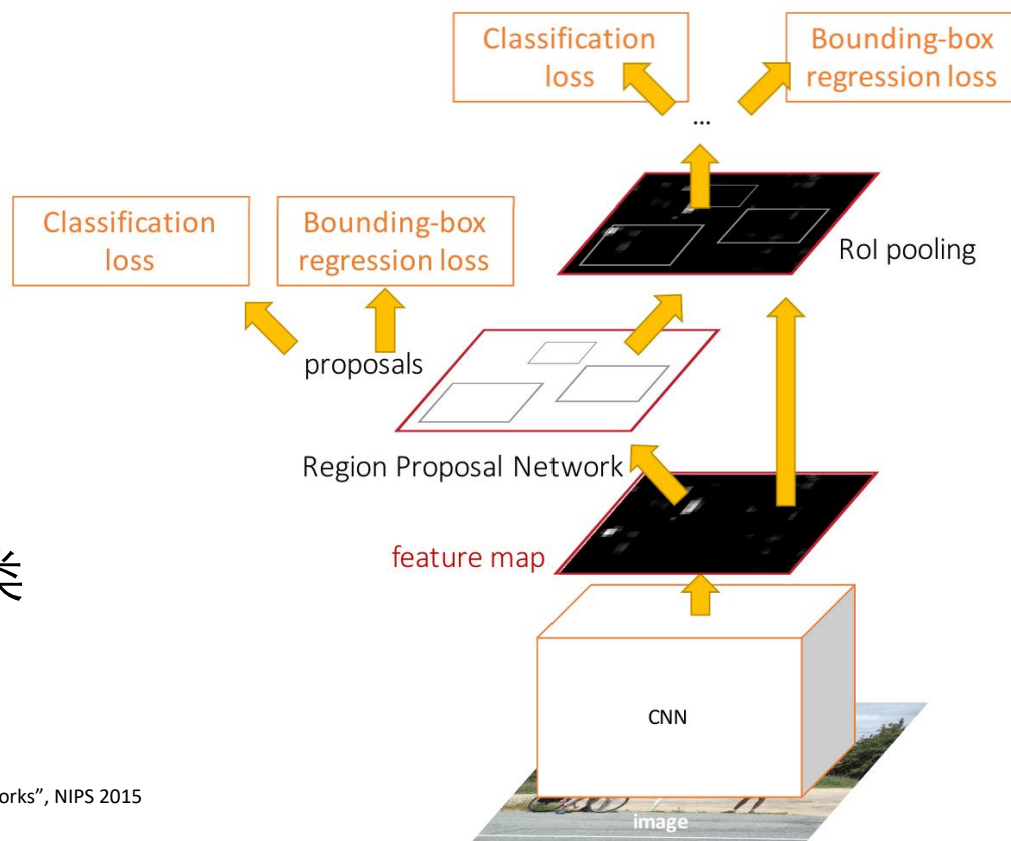
回顾: 候选区域选择使用“选择搜索”算法 – 使用CNN学习来替代!

Girshick et al, “Rich feature hierarchies for accurate object detection and semantic segmentation”, CVPR 2014.
He et al, “Spatial pyramid pooling in deep convolutional networks for visual recognition”, ECCV 2014
Girshick, “Fast R-CNN”, ICCV 2015

Fast_{er} R-CNN: 可学习的区域候选

插入区域候选网络**Region Proposal Network (RPN)** 从特征中预测候选区域

其他与Fast R-CNN一样: 裁剪特征得到每个候选, 逐个分类



Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015
Figure copyright 2015, Ross Girshick; reproduced with permission

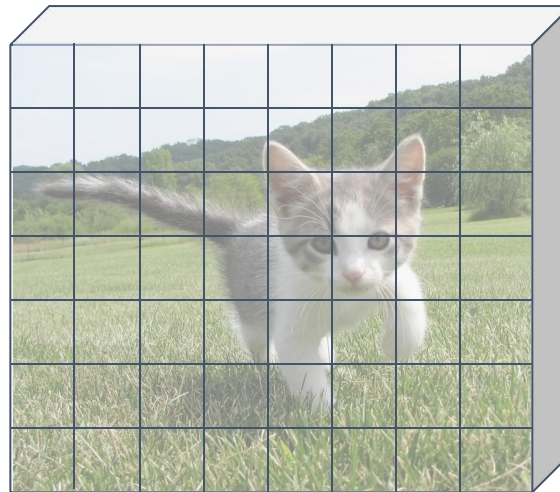
Region Proposal Network (RPN)

运行骨干CNN网络得到与输入图像对齐的特征平面



输入图像
(e.g. 3 x 640 x 480)

CNN



图像特征
(e.g. 512 x 20 x 15)

Region Proposal Network (RPN)

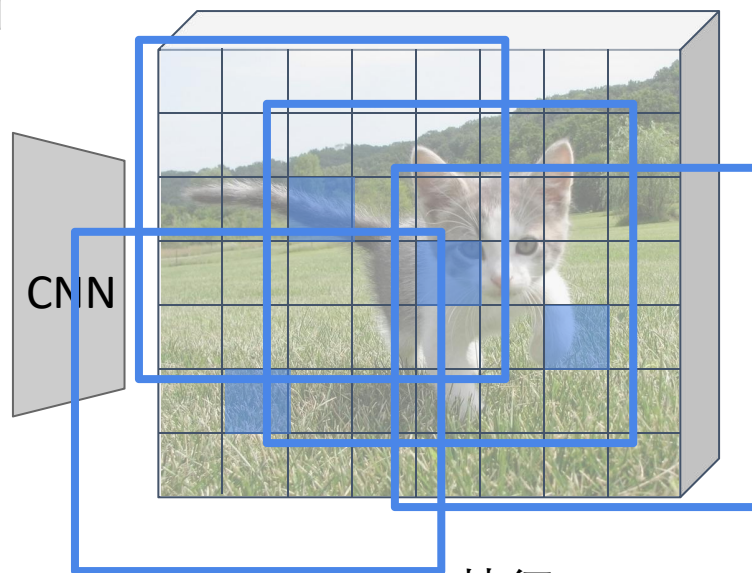
设定一个在特征平面每个点固定大小的锚框

anchor box

运行骨干CNN网络得到与输入图像对齐的特征平面



输入图像
(e.g. 3 x 640 x 480)



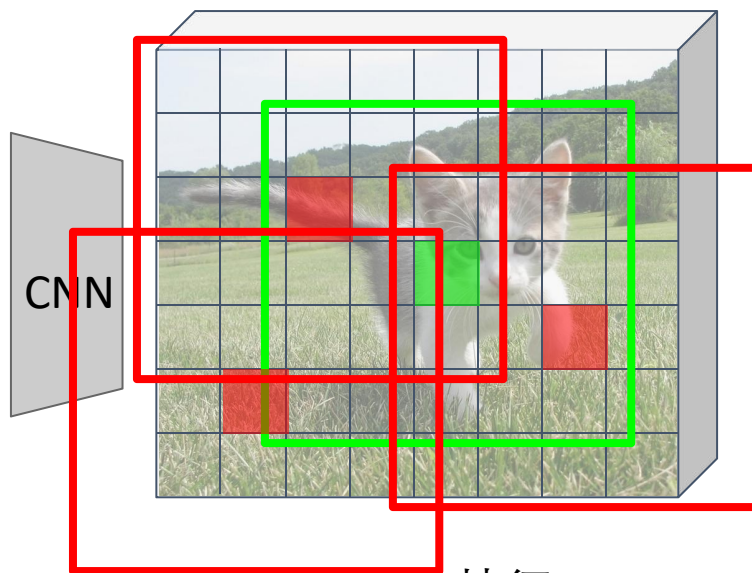
图像特征
(e.g. 512 x 20 x 15)

Region Proposal Network (RPN)

运行骨干CNN网络得到与输入图像对齐的特征平面



输入图像
(e.g. 3 x 640 x 480)



图像特征
(e.g. 512 x 20 x 15)

设定一个在特征平面每个点固定大小的锚框

anchor box

锚框是一个
目标?
1 x 20 x 15

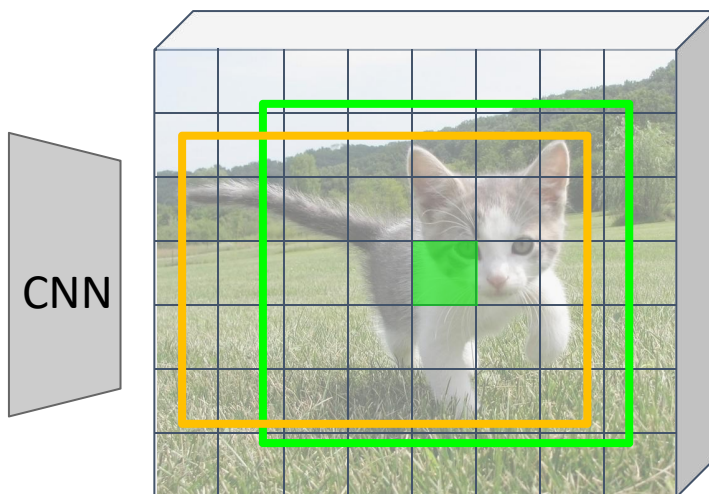
在每点, 预测对应的锚框是否包含一个目标 (每个单元的逻辑回归, 使用conv层预测分值)

Region Proposal Network (RPN)

运行骨干CNN网络得到与输入图像对齐的特征平面



输入图像
(e.g. $3 \times 640 \times 480$)



图像特征
(e.g. $512 \times 20 \times 15$)

设定一个在特征平面每个点固定大小的锚框

anchor box



锚框是一个目标?

$1 \times 20 \times 15$

框变换

$4 \times 20 \times 15$

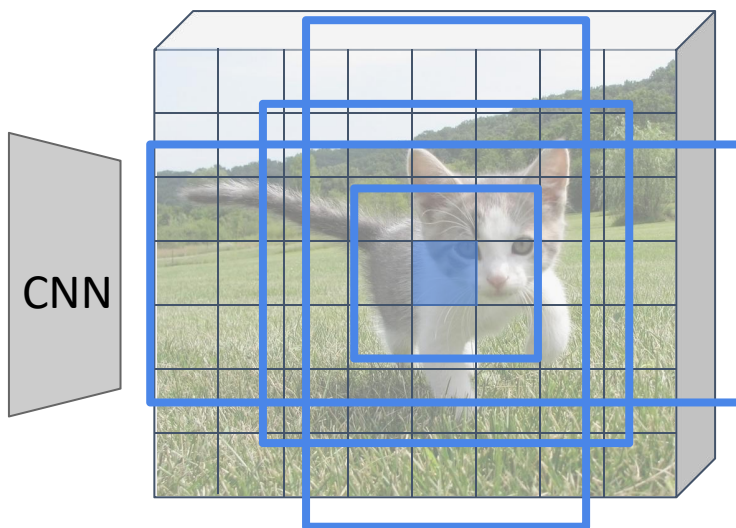
对于预测为正向的边框, 同样预测一个从**锚框**到**目标框**的边框的回归变换

Region Proposal Network (RPN)

运行骨干CNN网络得到与输入图像对齐的特征平面



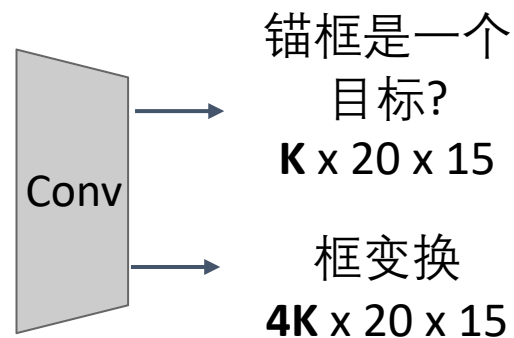
输入图像
(e.g. $3 \times 640 \times 480$)



图像特征
(e.g. $512 \times 20 \times 15$)

问题: 锚框Anchor box 可能有错误的大小/形状

方案: 每个点使用 K 个不同的锚框!

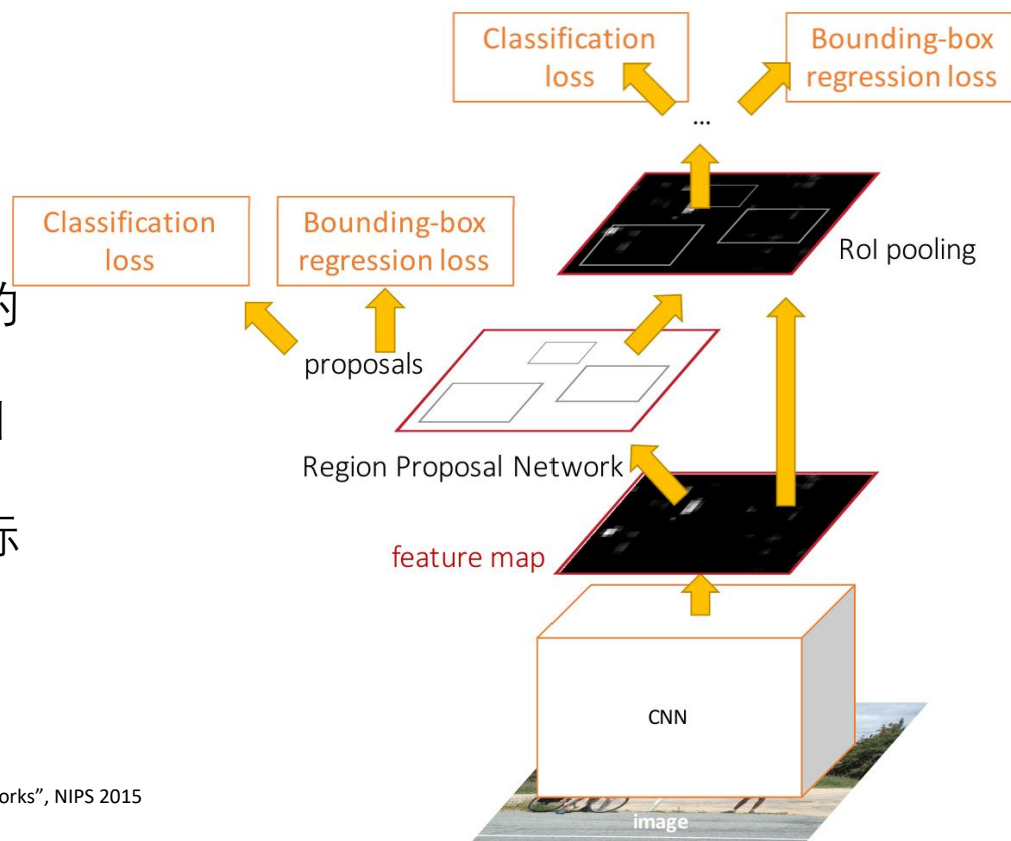


在测试时间: 按分值对所有 $K \times 20 \times 15$ 边框进行排序, 选择最高的~300 作为候选区域

Faster R-CNN: 可学习的候选区域方法

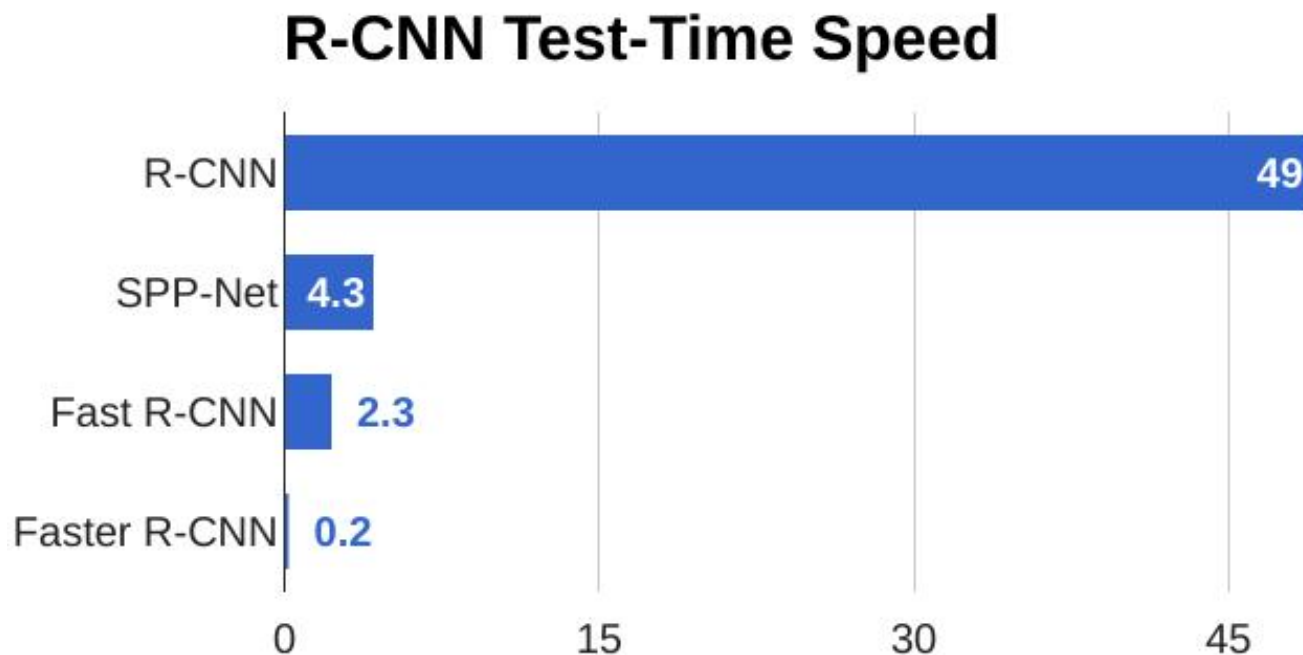
联合训练4类损失:

1. **RPN 分类**: 锚框是目标/ 还是不是目标
2. **RPN 回归**: 预测从锚框到候选框的变换
3. **Object 分类**: 分类候选是背景 / 目标
4. **Object 回归**: 预测从候选框到目标框的变换



Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015
Figure copyright 2015, Ross Girshick; reproduced with permission

Faster R-CNN:可学习的候选区域方法



Faster R-CNN:可学习的候选区域方法

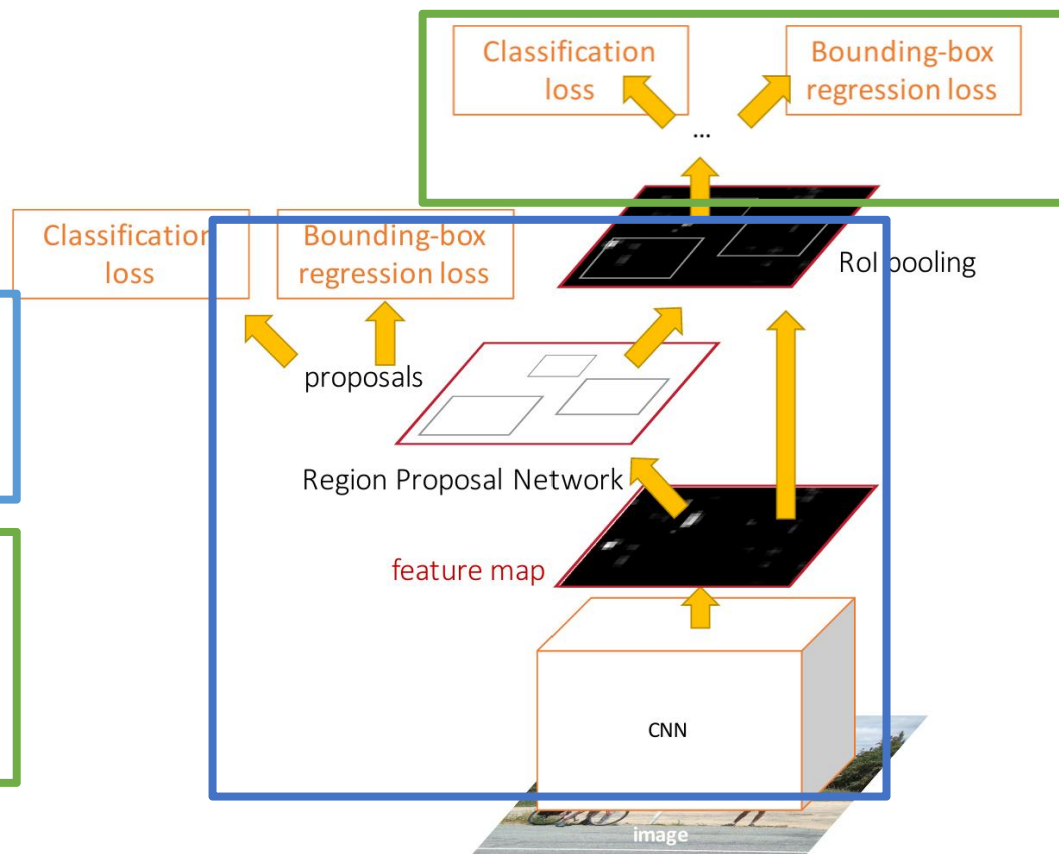
Faster R-CNN 是一个
两步目标检测子

第一步: 每张图片执行一次

- 骨干网络
- 区域候选网络

第二步: 每个区域执行一次

- 特征裁剪: RoI 池化 / 对齐
- 预测目标类别
- 预测bbox偏移



Faster R-CNN:可学习的候选区域方法

Faster R-CNN 是一个
两步目标检测子

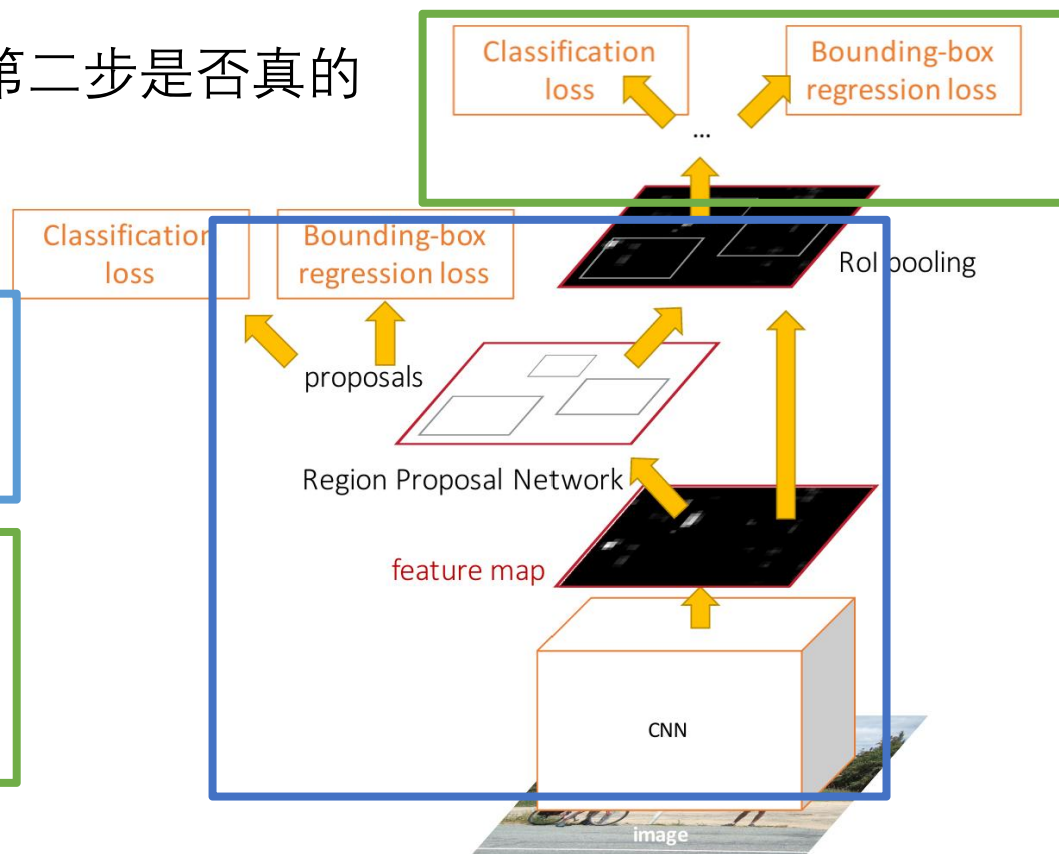
第一步: 每张图片执行一次

- 骨干网络
- 区域候选网络

第二步: 每个区域执行一次

- 特征裁剪: RoI 池化 / 对齐
- 预测目标类别
- 预测bbox偏移

问题: 第二步是否真的
需要?

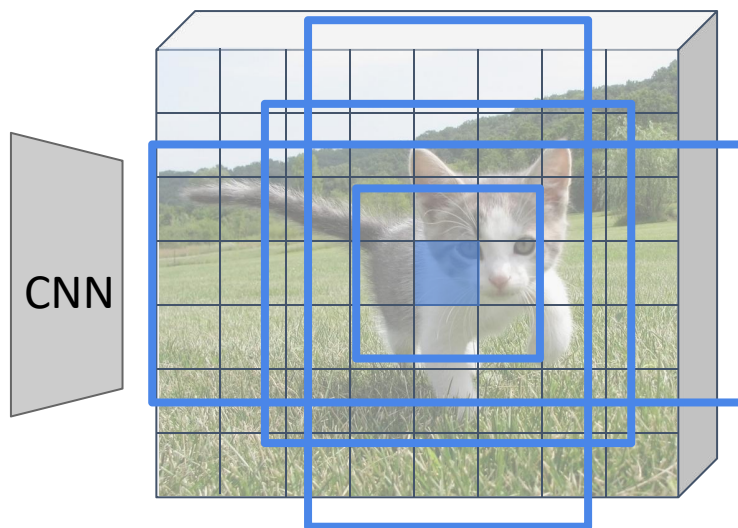


单步目标检测

执行骨干CNN网络获取与输入图像对齐的特征



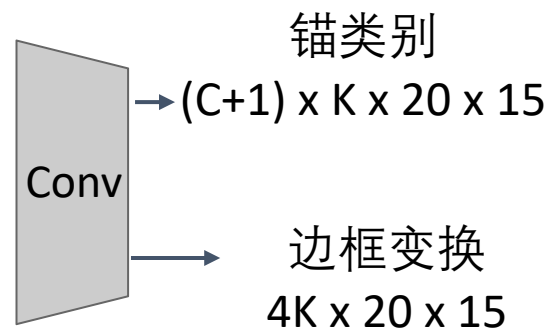
输入图像
(e.g. $3 \times 640 \times 480$)



图像特征
(e.g. $512 \times 20 \times 15$)

RPN: 分类单个锚框为目标/
不是目标

单步检测子: 分类每个目标为
C类目标 (背景) 中的一个



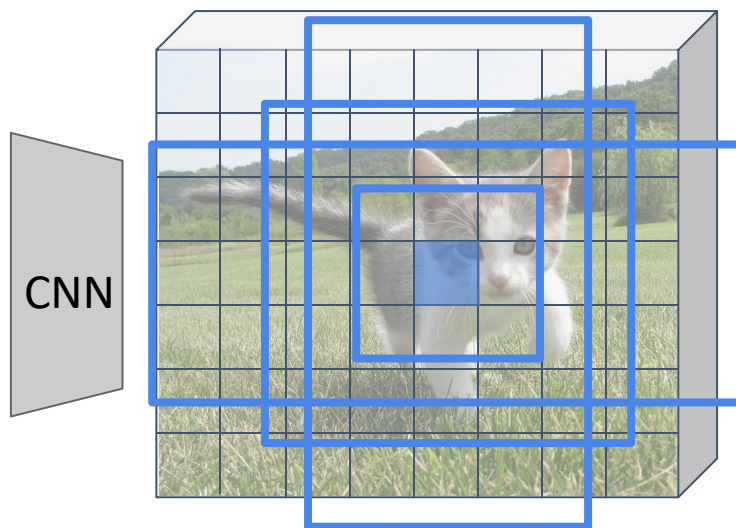
记住: 图像特征平面每个点有K个锚框

单步目标检测

执行骨干CNN网络获取与输入图像对齐的特征



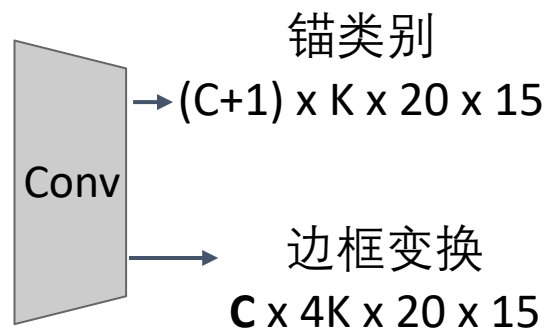
输入图像
(e.g. $3 \times 640 \times 480$)



图像特征
(e.g. $512 \times 20 \times 15$)

RPN: 分类单个锚框为目标/
不是目标

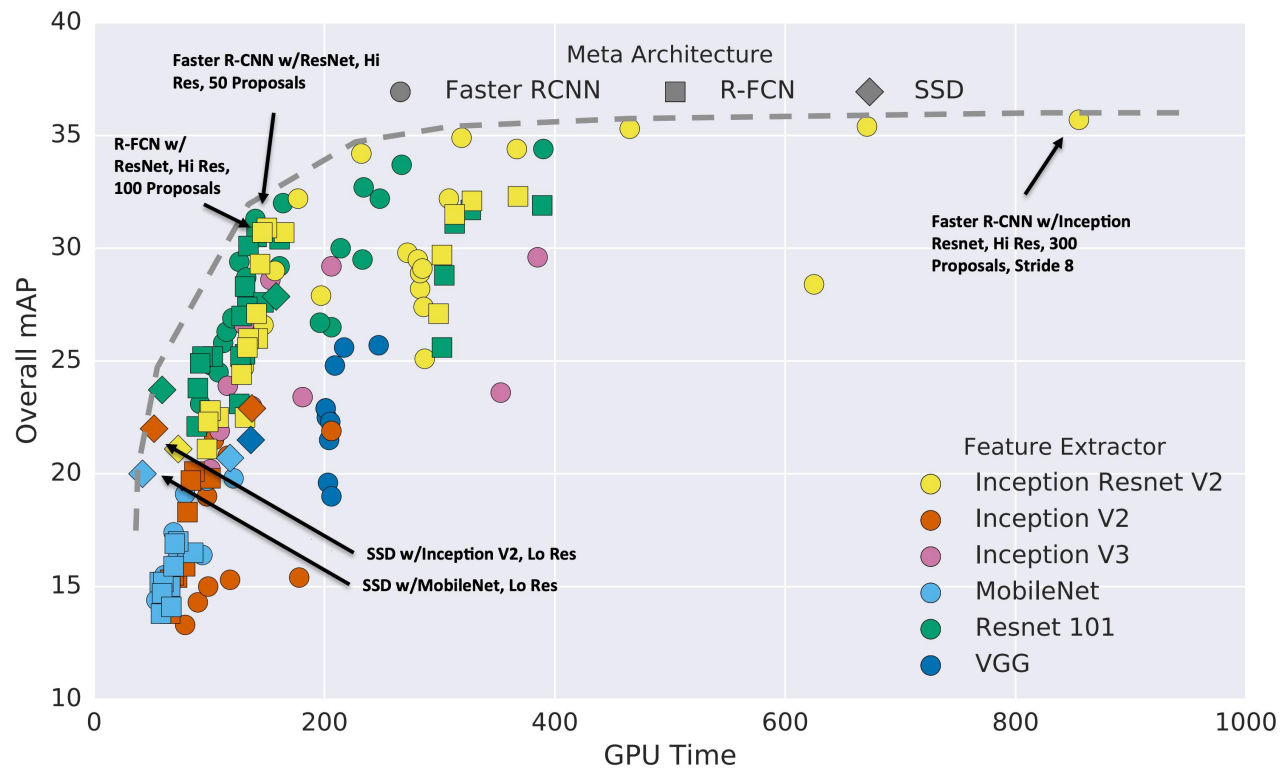
单步检测子: 分类每个目标为
C类目标 (背景) 中的一个



有时使用**指定类别回归**方法: 类别分别使用不同的网络每个预测框变换

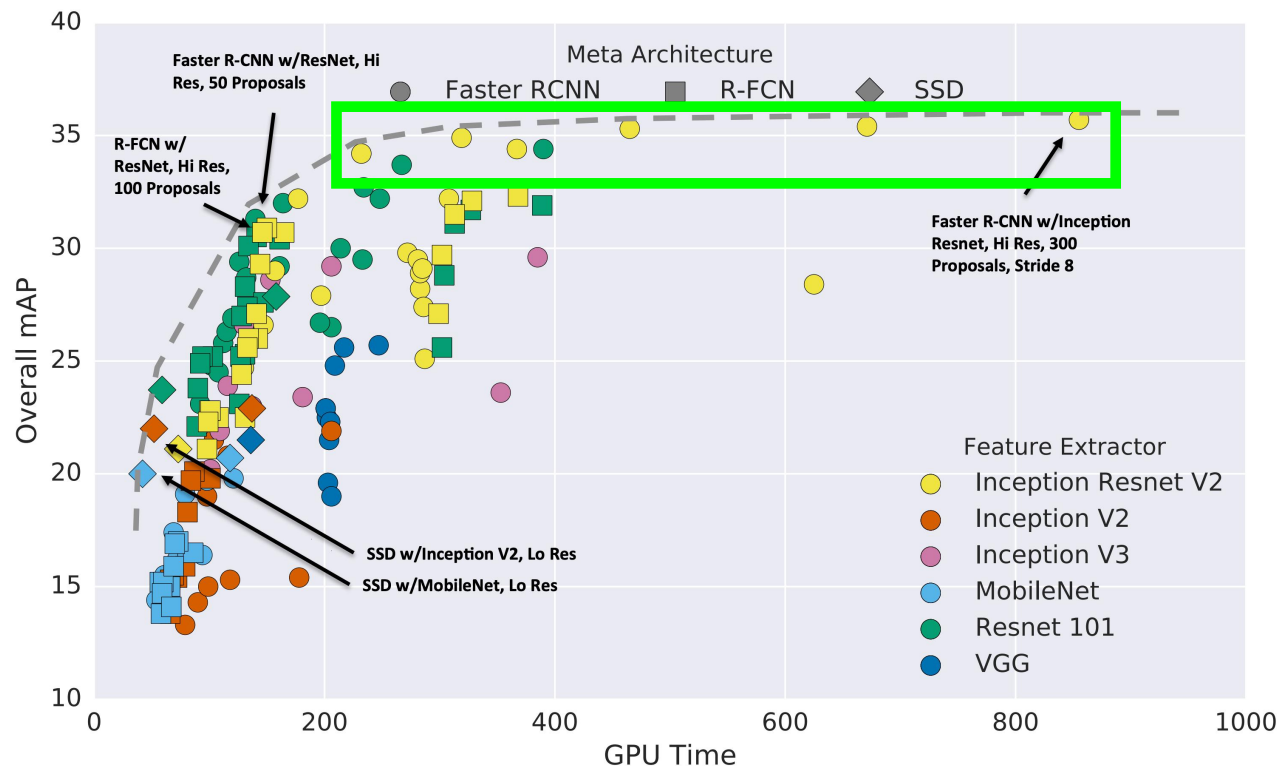
Redmon et al, "You Only Look Once: Unified, Real-Time Object Detection", CVPR 2016
Liu et al, "SSD: Single-Shot MultiBox Detector", ECCV 2016
Lin et al, "Focal Loss for Dense Object Detection", ICCV 2017

目标检测: 更多变种!



Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

目标检测: 更多变种!

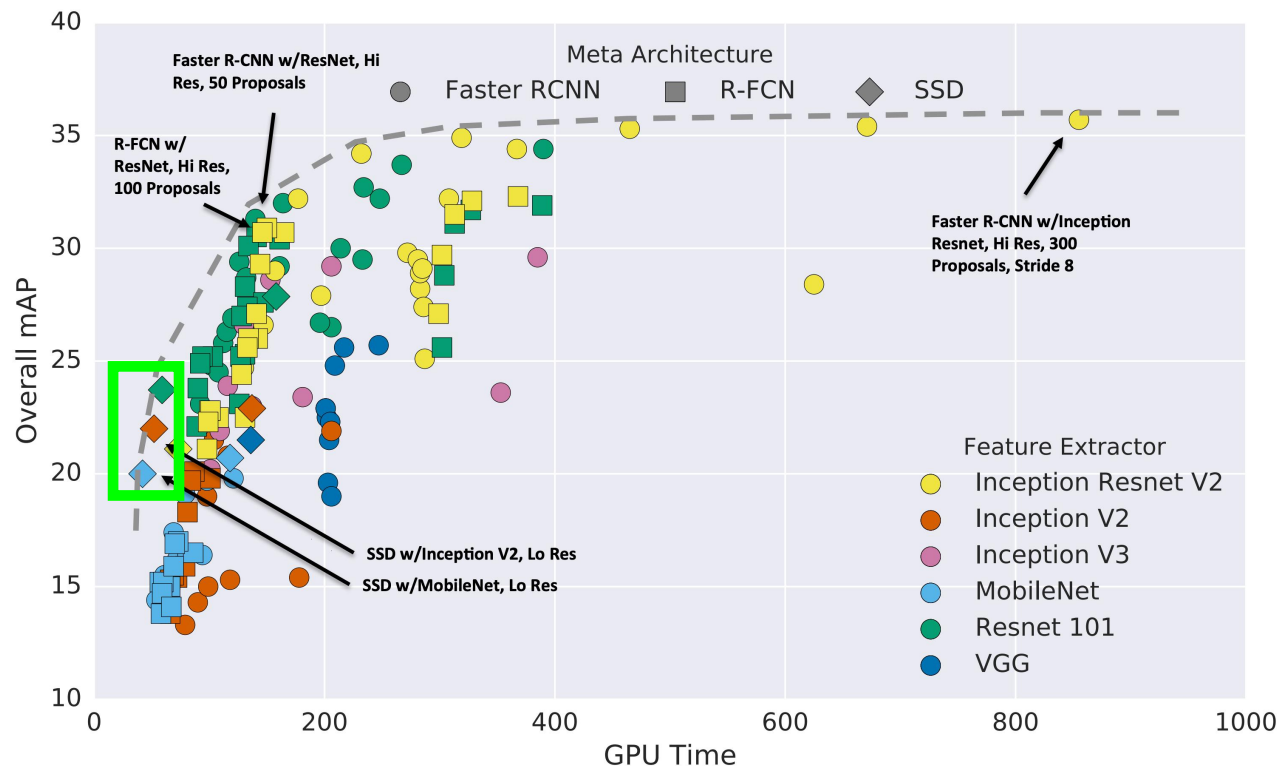


此外:

- 两步方法(Faster R-CNN)
性能更好, 但速度慢一些

Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

目标检测: 更多变种!

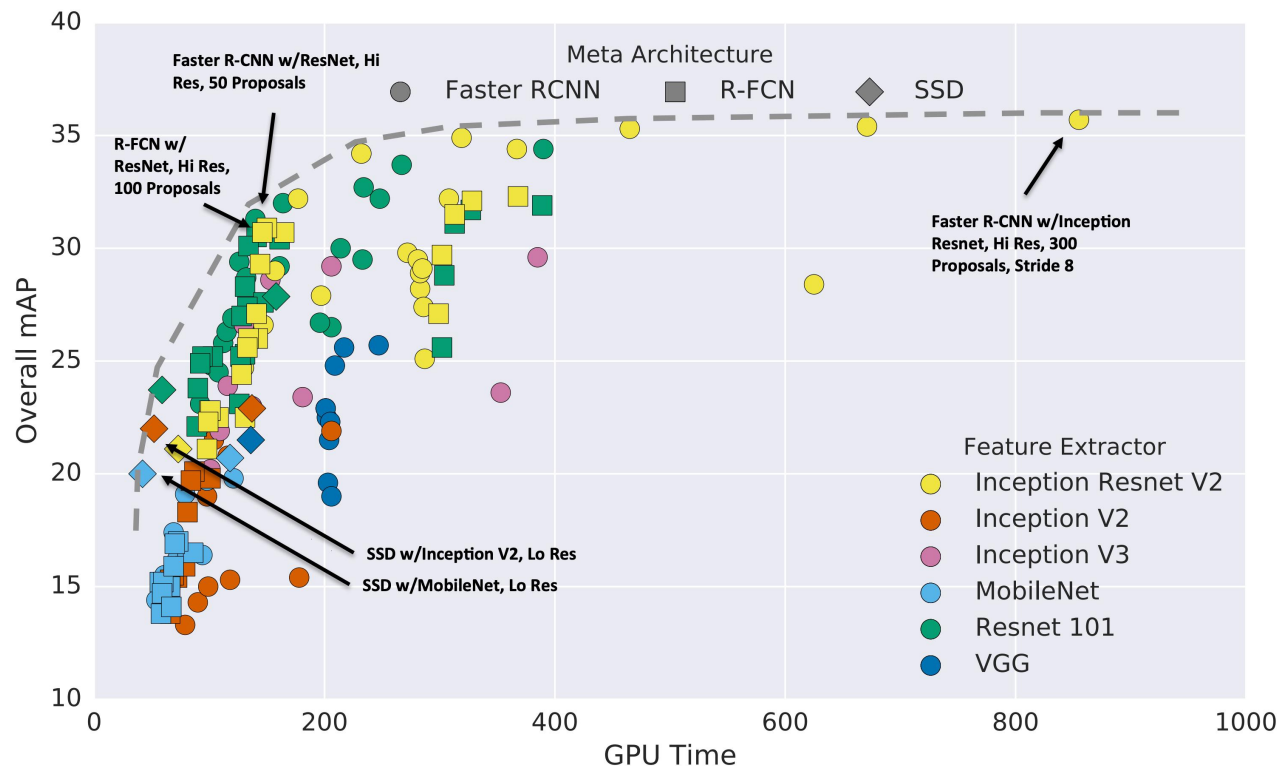


此外:

- 两步方法(Faster R-CNN) 性能更好, 但速度慢一些
- 单步方法(SSD) 更快, 但性能差一些

Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

目标检测: 更多变种!

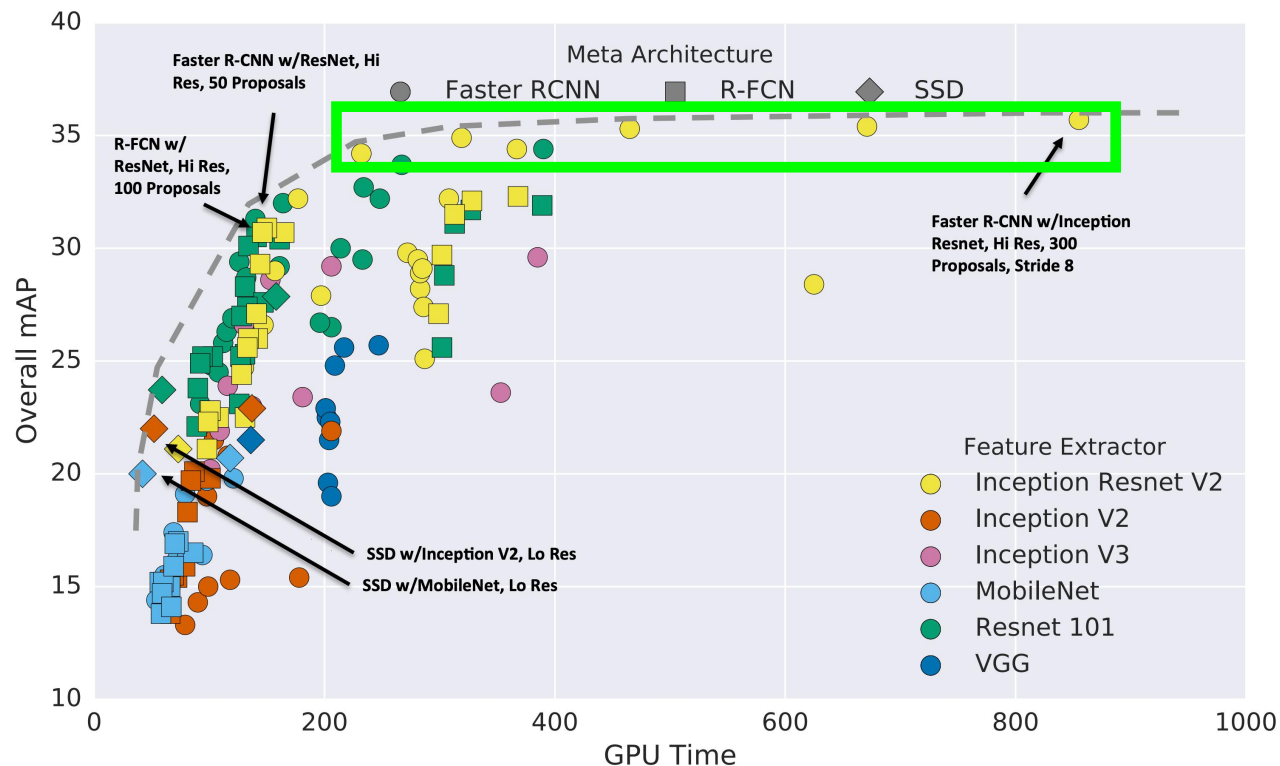


此外:

- 两步方法(Faster R-CNN) 性能更好, 但速度慢一些
- 单步方法(SSD) 更快, 但性能差一些
- 更大的骨干网络能够提升性能, 但速度慢一些

Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

目标检测: 更多变种!



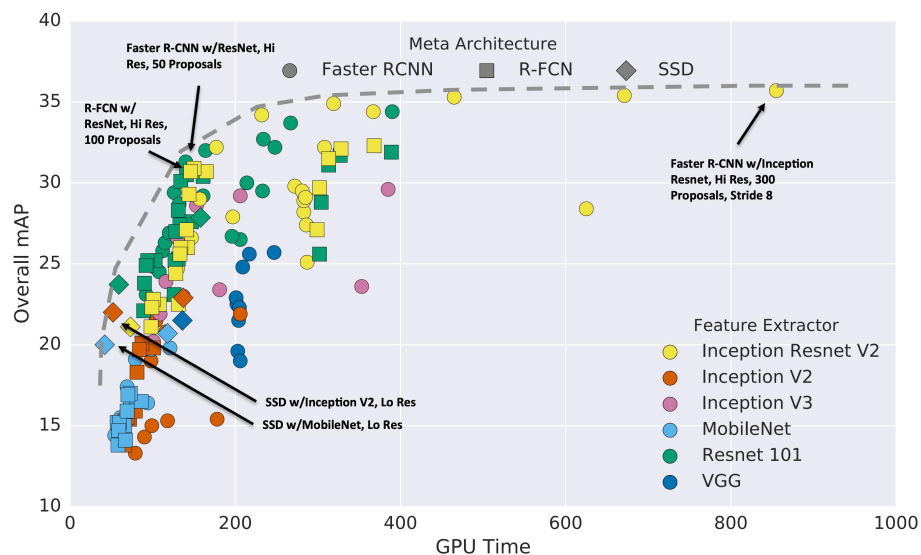
此外:

- 两步方法(Faster R-CNN) 性能更好, 但速度慢一些
- 单步方法(SSD) 更快, 但性能差一些
- 更大的骨干网络能够提升性能, 但速度慢一些
- 随着方法速度变慢性能收益减少

Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

目标检测: 更多变种!

这些都是最近几年的结果... 由于GPUs更新迅速, 我们使用了很多小技巧来提升各种方法性能:

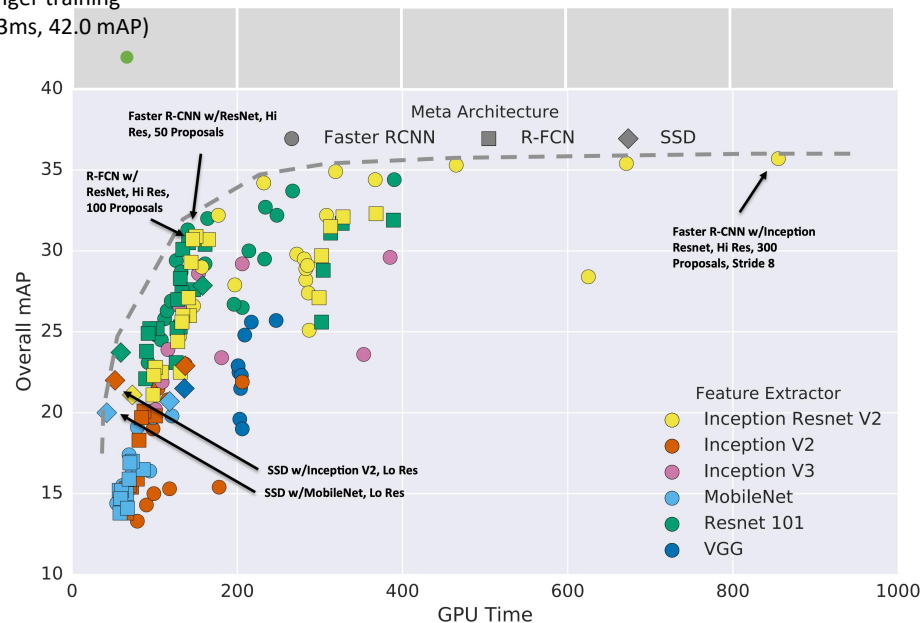


Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

Wu et al, Detectron2, GitHub 2019

目标检测: 更多变种!

Faster R-CNN
w/ResNet-101-FPN,
longer training
(63ms, 42.0 mAP)



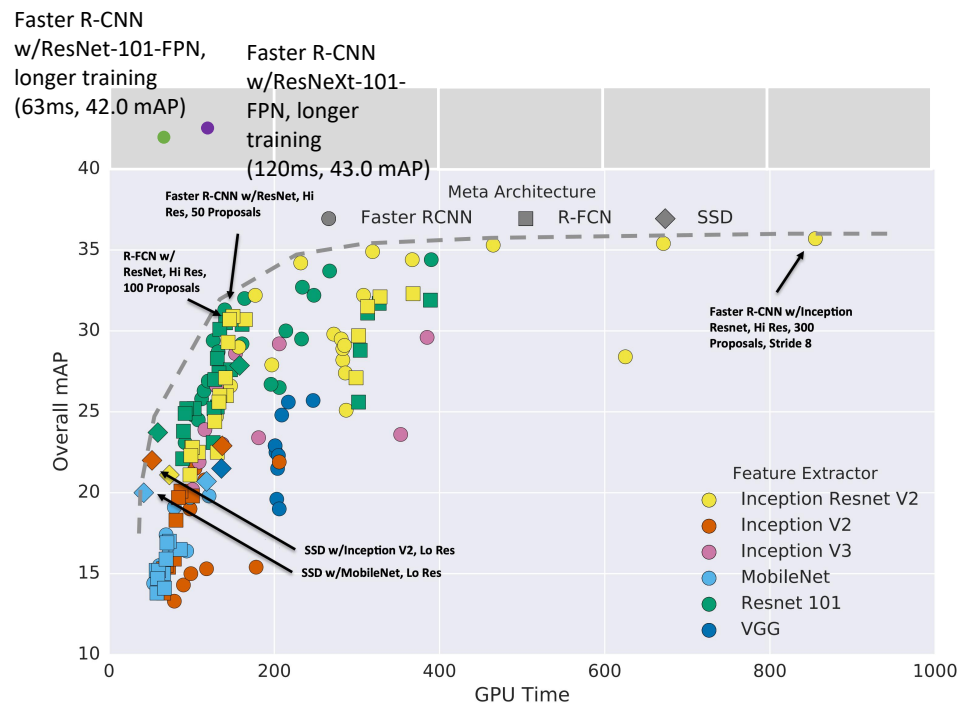
这些都是最近几年的结果... 由于GPUs更新迅速, 我们使用了很多小技巧来提升各种方法性能:

- 训练更长时间!
- 多尺度骨干网络: 特征金字塔网络

Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

Wu et al, Detectron2, GitHub 2019

目标检测: 更多变种!



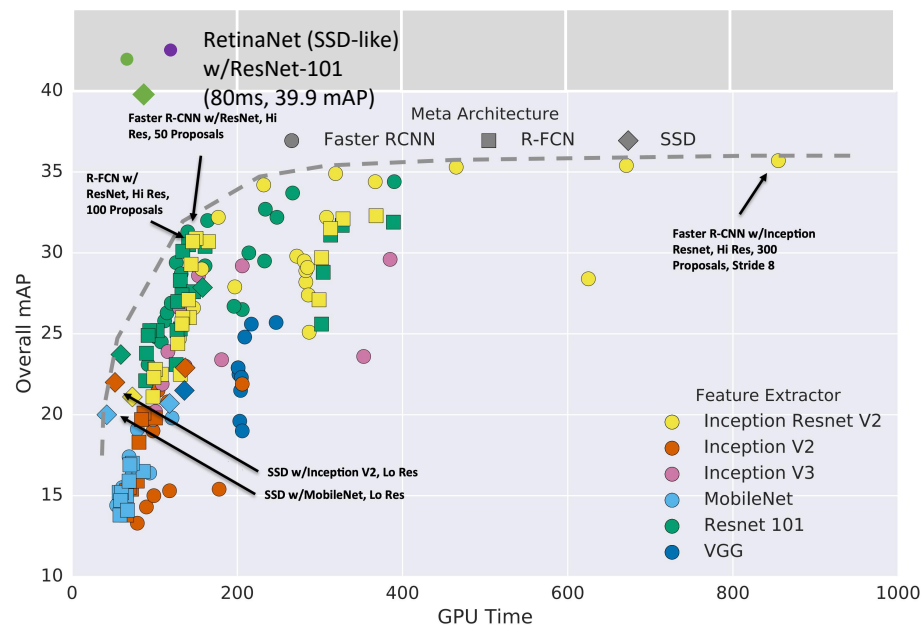
这些都是最近几年的结果... 由于GPUs更新迅速, 我们使用了很多小技巧来提升各种方法性能:

- 训练更长时间!
- 多尺度骨干网络: 特征金字塔网络
- 更好的骨干网络: ResNeXt

Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

Wu et al, Detectron2, GitHub 2019

目标检测: 更多变种!



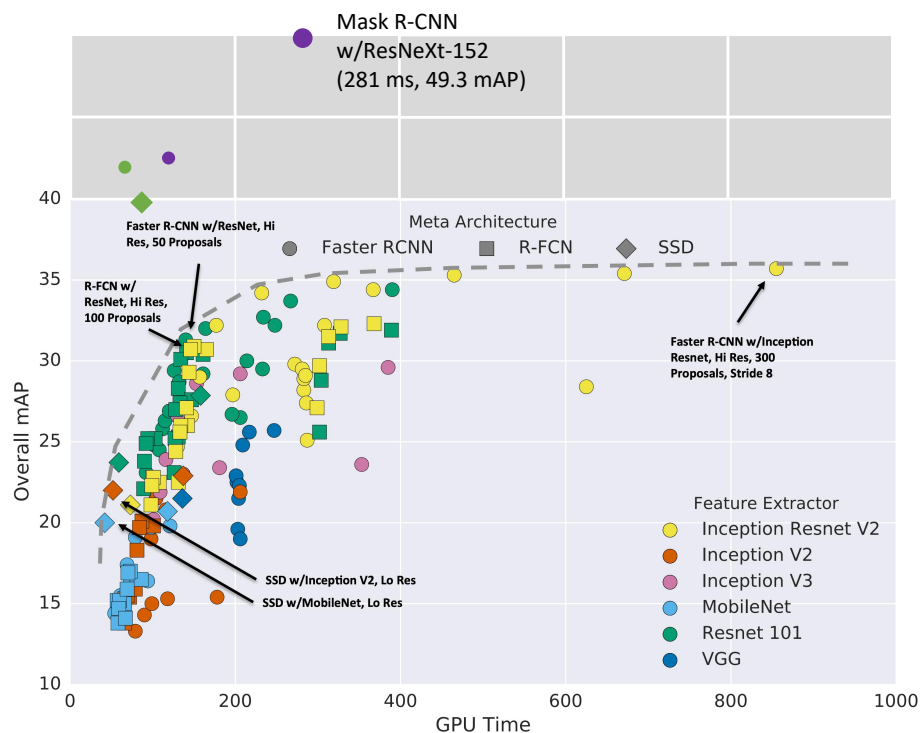
这些都是最近几年的结果... 由于GPUs更新迅速, 我们使用了很多小技巧来提升各种方法性能:

- 训练更长时间!
- 多尺度骨干网络: 特征金字塔网络
- 更好的骨干网络: ResNeXt
- 改进了单步检测方法

Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

Wu et al, Detectron2, GitHub 2019

目标检测: 更多变种!



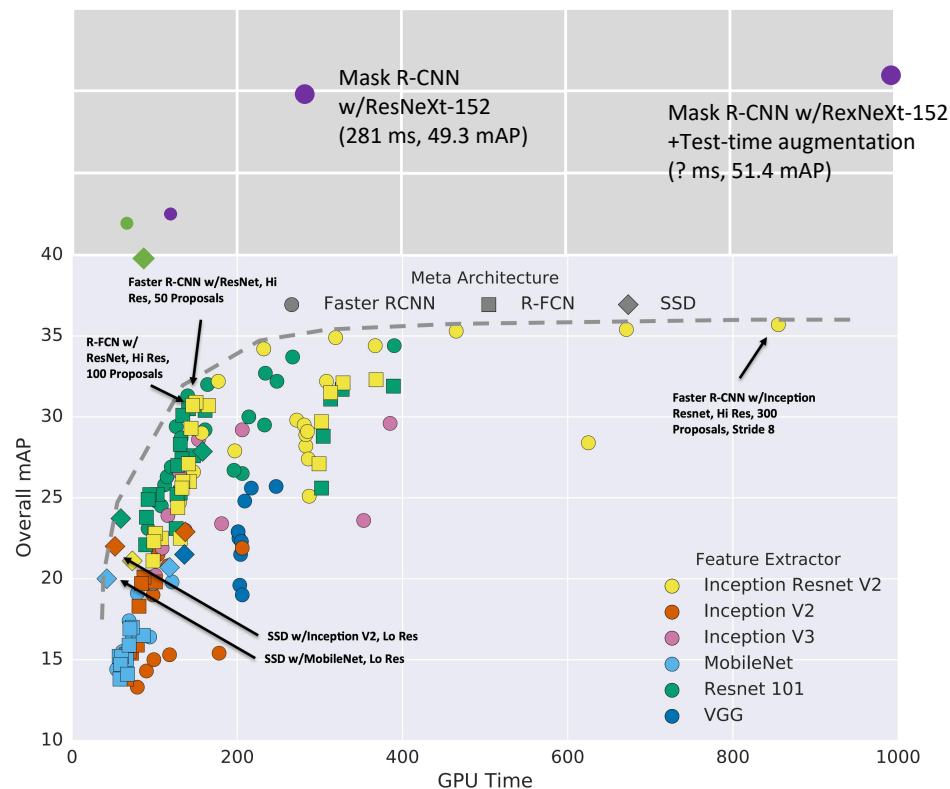
这些都是最近几年的结果... 由于GPUs更新迅速, 我们使用了很多小技巧来提升各种方法性能:

- 训练更长时间!
- 多尺度骨干网络: 特征金字塔网络
- 更好的骨干网络: ResNeXt
- 改进了单步检测方法
- 更大的模型性能更好

Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

Wu et al, Detectron2, GitHub 2019

目标检测: 更多变种!



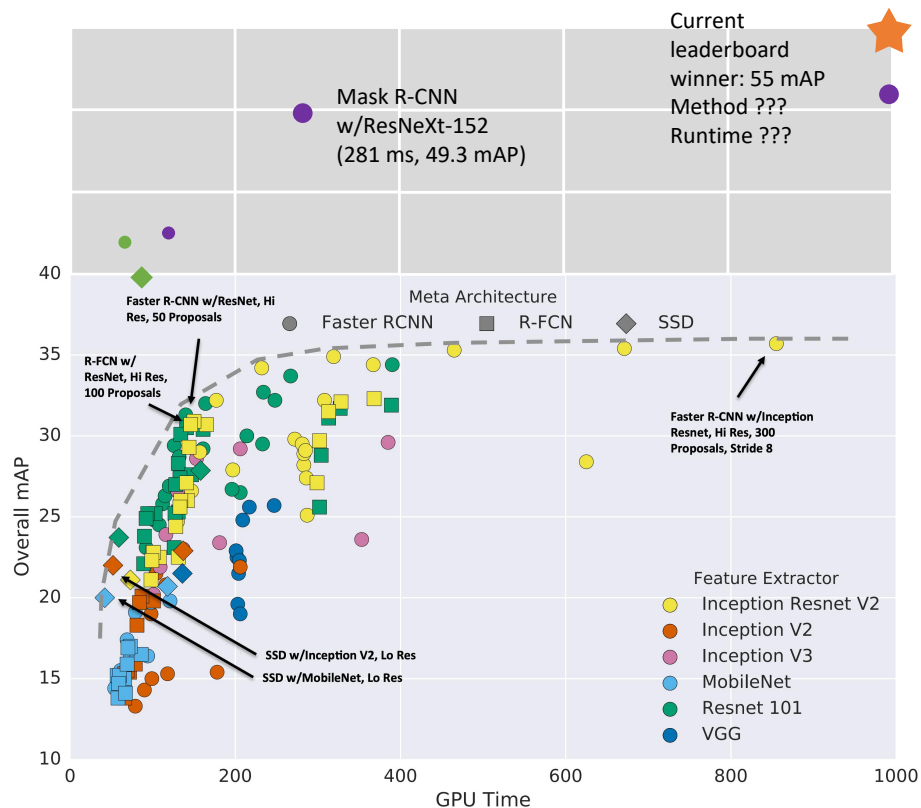
这些都是最近几年的结果... 由于GPUs更新迅速, 我们使用了很多小技巧来提升各种方法性能:

- 训练更长时间!
- 多尺度骨干网络: 特征金字塔网络
- 更好的骨干网络: ResNeXt
- 改进了单步检测方法
- 更大的模型性能更好
- 测试时间增加

Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

Wu et al, Detectron2, GitHub 2019

目标检测: 更多变种!



这些都是最近几年的结果... 由于GPUs更新迅速, 我们使用了很多小技巧来提升各种方法性能:

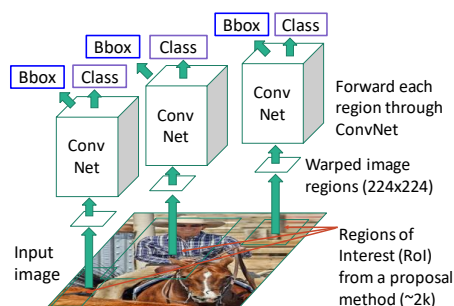
- 训练更长时间!
- 多尺度骨干网络: 特征金字塔网络
- 更好的骨干网络: ResNeXt
- 改进了单步检测方法
- 更大的模型性能更好
- 测试时间增加
- 集成, 更多数据, 等等

Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

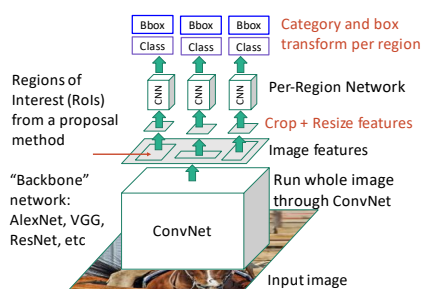
<https://competitions.codalab.org/competitions/20794#results>

总结

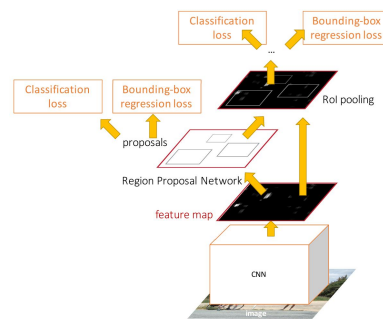
“Slow” R-CNN: 每个区域独立执行CNN



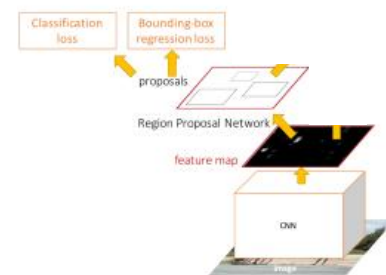
Fast R-CNN: 应用微小的裁剪方法处理共同特征平面



Faster R-CNN: 使用CNN计算候选区域



Single-Stage: 全卷积检测子



结束

下一次课

- 图像分割